

排序算法学习手册

像有人坐在旁边 从最小的动作开始讲

中文版 独立学习版

80 个学习条目 含算法家族 重要变体与辅助技术

2026 年 10 月 5 日

桌上有四张纸，分别写着 4、1、3、2。我们想把它们摆成 1、2、3、4。一张纸也不能少，重复的数字也要保留。这个目标很容易说清楚，难点是：下一步应该看哪张纸，什么时候换位置，怎样知道已经完成？排序算法就是把这些决定写成一套能反复照着做的步骤。这本书会先讲每个小动作，再把动作接成完整方法。

[可搜索正文](#) [可点击完整目录与书签](#) [原创步骤图](#) [参考来源](#)

第一次学，请从这里走

学习包里有一个总入口：00_从这里开始.html。第一次学，请双击它；用 Mac 上已有的 Safari、Chrome 或 Edge 打开即可。它把同样的完整内容分成小节，一次只读眼前的一小步。这份 PDF 是可以搜索、打印、反复查阅的完整手册，不要求你从第一页一口气读到最后。

总入口先解释四张纸、大小关系、比较和换位置，再学插入、选择、冒泡、归并、快速和计数。完整目录按可比较条件下的效率分组；入门路线按理解的先后安排，两种顺序各有用途。

读到“看对应动画”时再打开补充文件。初次只点“下一步”：先读这一步为什么这样做，再看长条。没看懂点“上一动作”。能说清一个动作以后，再点“自动慢播放”；播放中同一按钮变成“暂停”。“重看”回到开始。插入会有“手里暂存”，归并和计数有下方暂放或结果区；暂放的记录仍然存在。

看过不等于学会。暂停在下一动作之前，先猜会比较谁、会不会移动、哪个位置会空出来；再点一步检查。最后用四张写着数字的纸自己操作一遍，并用自己的话说出为什么可以停止。每个入门小节都有练习和解释。

六种基础方法有离线动画；全部 80 个学习条目都有完整步骤与数字示例，必要处有原创步骤图。高级方法先读本章“先把要用的小动作讲清楚”。遇到没有学过的树、堆、队列或二分，回到基础补充再继续。不必先记复杂度符号。

HTML 与两份 PDF 放在同一个文件夹。总入口的链接靠这些相邻文件工作，所以移动时搬整个文件夹，解压下载包后再打开。离线内容不需要安装软件、注册账号或联网；外部参考来源和 VisuAlgo 链接需要联网。

覆盖范围与计数方法

完整目录先于算法正文。目录共有 80 个编号学习条目；编号代表讲解单元，不等于 80 种互不相关的新算法。同名别称归到同一条目。具有不同操作步骤的重要变体单列，并标明“变体”；多个相关实现标明“家族”。

范围包括常见比较排序，整数与字符串的分布排序，自适应与缓存方法，少见的堆和树方法，外存调度，并行与网络排序，限制操作及趣味方法。置换选择明确是段生成阶段；间接排序和固定小网络明确是辅助技术。Grailsort 与 WikiSort 是同一类块归并的不同实现，并非彼此别名。

这个范围足够广，但不能穷尽所有已发表或新命名的方法。未展开的范围包括 AKS 等复杂网络构造、Columnsort、Cubosort、LearnedSort 的众多模型版本、特殊整数排序理论、全部缓存优化及磁带调度变体，也不逐一计数每一种枢轴、增量或阈值组合。它们需要额外的数学或机器模型，不应随意补一个通用复杂度。

Stalin sort 会删去不合次序的值，不能保持全部输入记录；miracle sort 没有保证发生的有效步骤。两者不计为本手册中的完整排序算法。拓扑排序排列依赖关系、Quickselect 只找某个名次，任务不同，亦不混入全量数值排序的速度排名。

本文固定从小到大排序。步骤图和例子为自制，数值小是为了看清动作，不作为实测性能证据。高级家族会给出可理解的具体操作与必要前提；平台级实现还需按文末作者资料补齐边界和工程细节。

完整目录

所有条目及别名先列于此。括号内说明计数类型，右侧为正文页码。

一 键结构与分布条件下的方法	14
01 计数排序 算法·计数式直方图排序；与鸽巢排序相关但不等同	14
02 鸽巢排序 算法·鸽笼排序；按完整键值分格的分配排序	15
03 低位优先基数排序 重要变体·最不重要位优先基数排序；LSD 表示 least significant digit	16
04 高位优先基数排序 重要变体·最重要位优先基数排序；MSD 表示 most significant digit	17
05 二进制快速排序 重要变体·基数交换排序；二进制 MSD 基数排序	19
06 美国旗排序 重要变体·原地多路 MSD 基数排序；名称来自连续色块般的分区	20
07 桶排序 算法家族·箱排序；bin sort；分桶策略是一族算法	21
08 闪排序 算法·FlashSort；按类别原地循环分配的直方图排序	22
09 字典树排序 算法家族·前缀树排序；trie 读作 try；数字树排序的相关形式	24
10 爆裂排序 算法家族·动态桶字典树字符串排序；含 C-burstersort、采样 Burstersort 变体	25
11 Spreadsort 混合分布排序 算法家族·混合基数/比较排序；含整数、浮点和字符串专用版本	26
12 多键快速排序 算法·三路基数快速排序；三向字符串快排；这些是同一方法的名称	27
二 利用已有顺序的自适应方法	29
13 自然归并排序 算法家族·自然合并排序；连续有序段自适应归并	29
14 TimSort 算法·Timsort；自然归并、二分插入与跳跃合并的混合算法	31
15 Powersort 重要变体·按节点层级安排归并的自然归并变体；含多路 Powersort	32
16 Peeksort 重要变体·探看中点附近自然段的归并排序	33
17 插入排序 算法·直接插入排序；线性插入排序	35
18 二分插入排序 重要变体·折半插入排序；用二分查找选插入位置	37
19 图书馆排序 重要变体·留空插入排序；gapped insertion sort	38
20 耐心排序 算法·纸牌堆排序；与最长上升子序列算法有关	39
21 平滑排序 重要变体·Dijkstra 的自适应原地堆排序	40

22 线段排序 算法·抽取有序子序列再归并；不是连续自然段归并	42
23 笛卡尔树排序 算法·基于笛卡尔树的自适应堆排序；Levcopoulos–Petersson 方法	43
三 通用比较排序与条件优化	45
24 自顶向下归并排序 算法·递归归并排序；Merge sort 的经典实现	45
25 自底向上归并排序 重要变体·迭代归并排序；归并排序变体	47
26 堆排序 算法·二叉堆排序；堆式选择排序	48
27 弱堆排序 算法·Weak-heap sort；经典 n 个反向位实现	50
28 锦标赛排序 算法·胜者树排序；淘汰赛排序	51
29 内省排序 算法·Introspective sort；快速排序与堆排序的混合	52
30 模式破坏快速排序 算法·pdqsort；模式抵抗快排	53
31 伸展树排序 算法·Splay sort；自调整树排序	55
32 最小最大堆排序变体 重要变体·最小最大堆提取排序；双端优先队列排序	56
33 块归并排序：Grailsort / WikiSort 算法家族·块排序；Grailsort 与 WikiSort 是相关实现，互非别名	57
34 快速排序：Lomuto 分区 算法·快排；本节采用 Lomuto 单向扫描分区	59
35 快速排序：Hoare 分区 重要变体·Hoare 原始分区；双向扫描快排变体	60
36 三路快速排序 重要变体·三向快排；肥枢轴快排；分区使用荷兰国旗思路	62
37 双枢轴快速排序 重要变体·双轴快排；Yaroslavskiy 风格双枢轴分区	63
38 样本排序 算法家族·抽样排序；Sample sort；多路分区家族	64
39 树排序与平衡树变体 算法家族·二叉搜索树排序；BST sort；平衡树排序是其变体	66
40 希尔排序与增量变体 算法·缩小增量排序；Shell sort	67
41 漏斗排序 算法·Funnel sort；缓存无关排序的一种	68
42 归并插入排序 算法·Ford–Johnson 算法；不是普通归并与插入的任意混合	69
43 间接排序 辅助技术·索引排序；指针排序；此处 tag sort 指附带索引	71
四 外存并行与受限输入的方法	72
44 外部多路归并排序 算法家族·多路外排序； k 路归并外排序	72
45 平衡多路外部归并 重要变体·平衡 k 路归并；外部归并的轮次调度变体	73

46 多相归并排序 重要变体 · 多相外部归并	74
47 置换选择生成有序段 辅助阶段 · 替换选择；外排序的段生成阶段	75
48 外部分布排序 算法家族 · 外部桶排序；外部范围分割排序	76
49 并行归并排序 重要变体 · 并行合并排序	77
50 并行快速排序 重要变体 · 并行快排	78
51 并行采样排序与 IPS4o 算法家族 · 原地并行超标量采样排序	80
52 GPU 基数排序 重要变体 · 图形处理器基数排序	81
53 双调排序网络 算法 · Bitonic sort；双调排序；有时泛称 Batcher sort	82
54 奇偶归并排序网络 算法 · Batcher 奇偶归并；不同于奇偶换位排序	84
55 固定小规模排序网络 辅助技术 · 小数组比较交换网络；本条采用四项五比较器网络	85
56 剪切排序 算法 · 蛇形网格排序	86
57 位移有界数据的堆排序 算法 · k 位移排序；近有序最小堆方法	87
五 工作较多或操作受限的方法	89
58 梳排序 算法 · 梳子排序；CombSort11 是间隔规则变体	89
59 圆圈排序 算法 · Circlesort；不是 circular sort 的所有同名方案	90
60 循环排序 算法 · 最少写入的比较式循环排序；另有已知目标位置的线性变体	91
61 秩排序（枚举排名） 算法 · 排名排序；枚举排序（不要与枚举全排列混淆）	93
62 选择排序 算法 · 直接选择排序	94
63 双向选择排序 重要变体 · 双选择排序；最小最大选择排序	95
64 侏儒排序 算法 · 花园侏儒排序；最初也称 stupid sort，名称有歧义	96
65 冒泡排序 算法 · 下沉排序；exchange sort 在部分资料中也指冒泡	97
66 鸡尾酒排序 重要变体 · 双向冒泡排序；摇摆排序；shaker sort	98
67 奇偶换位排序 算法 · 砖排序；奇偶排序（区别于奇偶归并）	99
68 直接交换排序 算法 · 两两交换排序；exchange/interchange 名称在文献中有歧义	100
69 基于旋转的原地归并排序家族 算法家族 · 原地归并；本节采用逐项插入式旋转归并	101

70 煎饼排序 算法 · 前缀翻转排序	102
71 烧焦煎饼排序 重要变体 · 带方向前缀翻转排序	104
72 Stooge 排序 趣味方法 · 三段重叠递归排序 (描述性译名)	105
73 慢排序 趣味方法 · Slowsort ; 勿与 Bogosort 的模糊别名混用	106
六 特殊模型与趣味方法	108
74 珠排序 (重力排序) 物理模型 · 算盘排序 ; Bead sort	108
75 意大利面排序 物理模型 · 面条排序 ; SAG (Spaghetti Analog Gadget)	110
76 睡眠排序 趣味方法 · 延时输出排序 (描述性名称)	111
77 穷举全排列排序 趣味方法 · 确定性排列排序 ; deterministic Bogosort 在部分资料中的称呼	112
78 Bogo 排序 趣味方法 · 愚蠢排序 ; 猴子排序 ; 随机排列排序	113
79 Bozo 排序 趣味方法 · 随机两格交换排序	114
80 Bogo-Bogo 排序 趣味方法 · 递归检查型 Bogobogosort ; 存在不统一变体	115

从四张纸和最小的动作开始

1 我们到底要得到什么

先把 1+1 想成把一张纸和另一张纸放在一起：现在共有两张纸。这里最重要的是分清“有多少张”和“纸上写着几”。纸上写着 7，也仍然只是一张纸。输入 [4,1,3,2] 有四张纸，不是十张。

从小到大表示：沿着一排纸往右看，右边的数不能比左边小。[1,2,3,4] 满足这个条件，[1,3,2,4] 不满足，因为 3 后面紧跟着 2。相等可以挨在一起，所以 [1,2,2,4] 也是排好的。

2 为什么不能把不顺眼的数拿掉

如果 [4,1,3,2] 最后只剩 [1,2,3]，它看起来从小到大，却不是正确排序：4 被丢了。正确排序要同时做到两件事：顺序正确，每张原来的纸也都还在。

纸不只有数字。它也可能写着一个人的名字和分数。按分数排序时，名字要跟着分数一起走。整张纸叫“记录”，决定位置的分数叫“键”。这两个词后面还会出现，含义就是这样。

3 看一眼和动一下是两件事

看到 4 和 1，我们可以问：4 比 1 大吗？答案是大。这叫比较。比较只得到一个判断，两张纸现在还在原位置。

如果规定左边不能比右边大，我们接着把它们对调：[4,1,3,2] 变成 [1,4,3,2]。这叫交换。一次比较可以不交换，例如 1 和 4 已经方向正确。算法必须告诉我们该比较谁，不能只说“比较到排好为止”。

4 为什么只换一次还没有完成

[1,4,3,2] 的头两张已经从小到大，但 4 后面仍有较小的 3 和 2。所以“这一对排好”和“整排排好”不是一回事。

很多算法每一步只保证一小件事。例如让最小数到最左边，或者让开头三张彼此有序。下一步靠这件已完成的小事继续推进。书中的绿色有时表示一个局部排好的小区间，文字会说明它是否已在最终位置。

5 怎样插入一个数 又不丢掉别的数

假设开头两张是 [1,4]，现在要加入 3。3 应放在 1 与 4 之间。先把 3 拿在手里，再把 4 向右挪一个位置，最后把 3 放进空位，得到 [1,3,4]。

电脑里的“挪动”也要按这个顺序。直接把 3 写到 4 的格子里，会覆盖 4；如果没有先保存或挪走它，就把记录丢掉了。文中的临时保存、空位和右移，都在解决这个问题。

6 位置编号只是用来指路

一排格子让我们能说“拿第几个”。为了避免每次都写长句，电脑通常从 0 编号。对于 [4,1,3,2]，位置 0 放 4，位置 1 放 1，位置 2 放 3，位置 3 放 2。

数字的大小与位置编号没有关系。位置 0 可以放最大的数。本文一般用“第 1 个”说话；出现下标公式时会说明从 0 数。连续几个位置叫区间，开头的一段叫前缀，末尾的一段叫后缀。

7 重复数为什么还可能有先后

两张纸都写 2，但一张属于甲，一张属于乙。我们用 2a、2b 记住纸的身份，字母不参与数字大小比较。输入 [2a,1,2b]，从小到大可以是 [1,2a,2b]，也可以是 [1,2b,2a]。

如果我们要求相等时仍按原先先后，第一种结果才符合要求，这叫稳定。稳定与数值是否排对是两个问题。每个条目会讲所述做法能否保留这种先后。

8 两排已经有序的纸 怎样合成一排

拿 [1,4] 和 [2,3]。每排最左边都是自己那排剩余纸里最小的，所以只比较两排的头就够了。1 比 2 小，先拿 1；现在比较 4 和 2，拿 2；再比较 4 和 3，拿 3；另一排空了，把剩下的 4 接上。结果 [1,2,3,4]。

这个动作叫归并。拿走一张后，只更新它所属那排的头，不重新扫描所有纸。它成立的前提是两排各自已排好；把两排无序数这样合起来，不保证正确。

9 大任务拆成小任务 为什么有用

要排四张纸，可以先分别排好前两张和后两张，再用刚才的归并合起来。排两张时，又可以把它看作两组各一张。一张纸本来就有序，不再拆。

让同一个做法处理更小的一组，叫递归。它像一个人暂停手上大任务，先解决里面的小任务，再回到原来的地方。必须规定什么时候不再继续，通常是只剩零张或一张；否则会不停重复。

10 栈与队列 只是两种取东西的顺序

把小盘子一层层叠起来，最后放的盘子最先取，这种规则叫栈。递归要记住“做完小任务后回哪儿”，可以把待回去的任务叠在这样的栈里。

排队买东西时，先来的人先走，这种规则叫队列。算法的队列也是先放先取。它们不是自动排序工具，而是安排下一步取谁的规则。

11 什么是树 它为什么能组织数字

画一个圆圈放 4，下面画两个圆圈放 1 和 6，用线相连。上面的圆圈叫父节点，下面的叫孩子。再在 1 下连 2 和 3，就有了树形关系。最上面的节点叫根；顺着线下去经过的所有节点是后代。

树不一定代表大小顺序。算法还必须规定关系。例如搜索树要求左边都较小、右边都较大；最小堆要求每个父节点不大于自己的孩子。两种规则不同，不能看到树就当作已排序。

12 堆怎样帮我们总是先拿到最小数

若每个父节点都不大于孩子，从根沿任意路径往下走，数都不会变小。所以根是全树最小的。这样的树叫最小堆。拿走根以后，把末尾的数补到根，再与较小的孩子比较，必要时向下交换，直到关系恢复。

加进一个新数时，先放树的末尾，与父节点比较；若新数更小，向上交换，反复做到根或不再需要交换。堆只保证根最小，兄弟之间不必有序。具体堆算法会继续讲怎样放到数组里、怎样找孩子。

13 什么是桶 与数位

桶就是分组。按“0 到 9、10 到 19、20 到 29”分组时，整个第一组一定在整个第二组前面，但第一组内部还可能乱。所以分组以后，常常仍需排每一组。

十进制的 21 表示两个十加一个一。个位是 1，十位是 2。按个位分组与按整个数分组不同：21 和 11 个位相同，却不代表它们相等。基数排序会解释为什么要把几轮分组按特定顺序接起来。

14 一排已经有序 怎样省力找位置

要把 5 插进 [1,3,6,8,10]。先看中间的 6，知道 5 在它左边，就不用再查 8 和 10；只看 [1,3]，最后找到 3 与 6 之间。这样每次排除约一半候选，叫二分查找。

它省的是找位置时的比较。真把 5 放进紧密排列的格子里，6、8、10 仍得往右挪。所以“很快找到了位置”不等于“整次插入也只做很少的动作”。

15 我们用什么办法说快或慢

先问具体在做什么：看多少张纸、比较多少次、交换多少次、挪多少张、读写多少块文件。有时比较很贵，有时搬整张大记录很贵，两者不能总混成同一个数字。

再问数据有多少。后面用 n 表示记录数。 $n=4$ 就是四张； $n=1000$ 就是一千张。走一遍每张看一次，大约做 n 件事。输入翻倍，这一类工作通常也约翻倍。

16 为什么会出现 n^2 和 $\log n$

若每张纸都与所有其他纸打交道，总次数接近 n 乘 n ，写成 n^2 。十张大约一百份这种工作，一千张大约一百万份。这是解释增长趋势，实际算法常只比较一半的成对组合。

若每次把候选减少一半，一千个候选约十次就缩到一个：1000、500、250、125，继续往下减。所需的减半次数写作 $\log_2 n$ ，读作“以 2 为底的对数”。每层看完整批，约十层，就有 $n \log n$ 的工作。先理解动作，符号只是给这种增长起个短名字。

17 最好 最坏 平均 不是同一件事

一排本来就有序，是某些方法的最好情况；完全倒着放，可能是最坏情况。但“最坏”不一定是倒序，取决于方法会在哪种输入上反复做最多的事。

说平均要先讲怎样抽取输入。把所有排列等可能抽一个，与“每天来的日志通常接近有序”，不是同一分布。随机算法还可能因为自己的随机选择，时快时慢。没有这些前提，就不能把一个平均数字当成保证。

18 空间 正确结束 与下一步怎样读

除了原来的纸，我们可能还要另一排空格、一张计数表、几张用来记任务的小纸条。这些是额外空间。原地排序通常表示不再准备另一份同样大的记录数组，但递归栈仍可能占地方。

每个方法都要能说清什么时候结束。正确结果必须有序、所有记录都保留，而且按所述条件能完成。接下来看算法时，沿着数字例子问三个问题：这一步改了哪里？为什么不会丢纸？完成了哪一件小事？复杂度部分可以在你能手动复现过程之后再读。

如何公平比较效率

我们已经把 n 、减半和 n^2 讲过了。现在才用一张表把这些工作量放在一起。表中比较的是增长趋势，不是秒数。默认只有一个工作者，数字已放在内存里，每次比较或搬动一个小记录的成本差不多。文件读写、多核和特殊数字条件要分开看。

增长量	朴素含义	n 为 1000 时的量级
n	像完整扫描一次；输入翻倍，工作约翻倍	1000
$n \log_2 n$	每层扫描，每次把任务约减半	约 10000
$n \log_2^2 n$	有约平方对数层的固定网络等	约 100000
n^2	每项可能与几乎所有项打交道	1000000
$n!$	尝试输入的全部排列，增加一项就再乘一次	大到不适合穷举

O 表示增长上界， Θ 表示同一增长量级， Ω 表示下界。它们通常忽略常数，不是秒数。 n 是记录数， k 可能是键域大小或位移上限， r 是基数， d 或 w 是位数；每个条目会给局部定义，不能跨条目把 k 当同一个量。 $\log$ 默认以 2 为底，其他固定底数只差常数。

最好指最有利输入，最坏指最不利输入，平均需要说明输入分布；期望也可能来自算法自己的随机选择。“以高概率”表示失败概率随规模变小，不等于任何随机结果都保证。无法可靠给出统一平均界时，正文明确保留条件或不给数值。

假设所有键都不同。两项比较只有“前者小”或“后者小”两个结果。问一次，最多分出 2 条结果路径；问两次，最多 4 条；问 t 次，最多 2^t 条。 n 条不同记录的可能次序有 $n!$ 种，其中 $n!$ 表示 $1 \times 2 \times \dots \times n$ 。要把这些次序分清，最坏路径必须足够长，使 $2^t \geq n!$ ，因此最坏至少要 $\log_2(n!)$ 次比较，即 $\Omega(n \log n)$ 。这不是说每一次比较都恰好把可能性减半。计数和基数排序还利用键的结构，所以不受纯比较的这个限制。

现实中比较字符串要检查多个字符，大记录移动很贵，缓存命中、分支、分配和磁盘读写也有成本。两种同为 $O(n \log n)$ 的算法仍可相差很多。小例能说明原理，不能证明常数性能。

外存按 B 条记录一块读取、 M 条记录内存来计算传输；并行同时报告总工作量 W 和跨度 S 。 p 个工作者的时间至少是 $\max(W/p, S)$ ，还会有同步与传输成本。物理装置和计时器方法则不能套用普通 RAM 排名。

有条件的效率层级

固定条件	优先层级 从较少工作到较多工作	限制
整数键域 $k=O(n)$ ，位数 d 固定	计数或基数的近线性层 → 通用 $n \log n$ 比较层 → 平方层	键域巨大、键很长会改顺序；桶类另需分布假设
已有很少逆序或很少长有序段	插入的 $O(n+I)$ 或自适应归并的近线性层 → 常规 $n \log n$ 层	I 为逆序对数；不同“近有序”含义不可混用
任意大数组，只准键比较	有保护的 $n \log n$ 层 → 无保护可能 n^2 的方法 → 固定平方层 → 超多项式层	普通快排平均很快，但最坏保障弱；同层靠实测比较
写入昂贵或记录巨大	间接排序、Cycle 的少写策略需单独评估	比较更多仍可能总用时更短
文件超过内存	少轮的大路数归并或均衡分布 → 更多整文件读写轮次	固定内存、磁盘、桶均衡和可用流数量
多核 GPU 或排序电路	固定 W 、 S 、线程数与带宽后评估	不能把并行层数当串行工作量；不作跨硬件名次
物理与随机趣味模型	条件成立的特殊模型另列；阶乘穷举和随机重试靠后	Sleep 不保证正确；随机方法没有有限最坏步数

阅读顺序按效率条件分组：先看可能近线性的专用方法和自适应方法，再看通用 $n \log n$ 方法；特殊机器条件单列；通常较慢的方法及趣味方法靠后。同组内先介绍有保障或较少工作的一层，再介绍附带条件、退化可能或研究型方法。同一效率层中的先后是学习依赖，不宣称前一个必定比后一个快。完整目录和 PDF 书签均可直接跳转。

如何使用每个条目

每个条目先把要用的小动作讲清楚，再接成完整方法。拿纸片照着例子做一遍：每次换、挪、分组或输出，都确认没有丢掉任何一张纸。复杂方法允许多读一遍；关键是每一步都有理由，而不是记住一个名字。

效率部分解释成本和成立条件；用途部分说明什么数据值得考虑。学习时先能复现例子，再替换为重复数、负数、逆序数和空数组。正文给出的稳定性仅针对所述版本，不能未经检查套到所有同名实现。

算法正文包含完整数字例、具体执行规则和必要工程注意；参考资料提供作者原论文、大学教材、NIST 定义和原实现。引用编号在两个语言版本中分别独立可用。

步骤图图例：橙色边框与 C 表示正在比较或分类；蓝色填充与 M 表示刚交换或移动；绿色填充与 S 表示文字明确指出的已排好区间或最终位置。S 若只是局部有序，并不宣称全局最终。白色是当前未标记项。图中有省略的内部比较时，正文会逐步解释；每一行都对应正文的一个状态。

步骤状态图 C 比较 M 移动 S 已有序

比较 4 与 1；绿色区 [2,3] 只是局部有序



交换后两项标蓝；这里只是动作图例



一 键结构与分布条件下的方法

先看小整数键域与固定位数可达到的近线性层，再看依赖分布或前缀的桶与字符串方法。同层并列；输入键域、字符总量、桶均衡度改变次序。二进制快速排序是 MSD 的变体，字符串三向快排使用字符比较，不能对任意长文本宣称 $O(n)$ 。

01 计数排序

类型 算法 别名或相关名称 计数式直方图排序；与鸽巢排序相关但不等同

先把要用的小动作讲清楚

先把“值”和“位置”分开：输入 $[3,1,2,1]$ 中有四条记录，但值 1 出现两次。排序不能删掉其中一个。若数字旁边还带着姓名，数字叫排序键，姓名必须随数字一起移动。

计数表是一排小格，每个格有固定标签。标签为 1 的格只记 1 出现几次，标签为 2 的格只记 2 出现几次；格里存的是数量，不是原输入的那条记录。

基本思路

如果要排的数只能是几个整数，为什么还要反复比较它们？我们先数清每种数有几份，就知道前面该留多少位置。这就是计数排序。比如已有两个 1，那么任何 2 都必须排在这两个 1 后面。

具体执行步骤

1. 先找输入中的最小键和最大键。给这个范围内的每个整数开一个计数格，全部写 0。若最小键是负数，可以用“键减最小键”当格号，例如 -2 对应格 0，-1 对应格 1。
2. 读一条记录，就把它对应格的数量加 1。读完后从小标签往大标签累加：把前面各格的数量加到当前格。这个累计值表示“当前键以及更小的键总共占多少位置”。
3. 另准备一排输出格。为了不把相等记录的先后顺序颠倒，从原输入右端开始处理。若某键的累计值是 t ，把当前记录放到输出下标 $t-1$ ，再把该键的累计值减 1。减 1 的意思是这个键的最后一个可用位置已被占用。
4. 一直处理到原输入最左端，输出就完成了。若只是裸整数，没有需要保留的其他字段，也可以按标签从小到大直接写出计数规定的份数；但这不是保存完整记录的实现。

跟着数字一步步做

1. 给两个 1 加标签以便观察：输入 $[3,1\text{甲},2,1\text{乙}]$ 。依次读取后，标签 1、2、3 的数量是 $[2,1,1]$ 。累计数量变成 $[2,3,4]$ ，所以输出的下标 0-1 给 1，下标 2 给 2，下标 3 给 3。
2. 先读右端的 1 乙：累计数为 2，放输出下标 1，得到 $[\text{空},1\text{乙},\text{空},\text{空}]$ ，键 1 的累计数降为 1。再读 2，放下标 2，得到 $[\text{空},1\text{乙},2,\text{空}]$ 。
3. 再读 1 甲，放下标 0；最后把 3 放下标 3。结果 $[1\text{甲},1\text{乙},2,3]$ 。甲仍在乙前面；去掉观察标签就是 $[1,1,2,3]$ 。

要做多少事 要用多少地方

实际工作是读输入、扫计数格、再写输出，各只做一遍。设 n 是记录数， k 是最大键减最小键再加 1； k 包括没有出现的整数。标准版本最好、通常、最坏都为 $\Theta(n+k)$ ，表示工作随这两项的和增长。额外空间 $\Theta(n+k)$ 用于计数及输出。完整记录版稳定，即相等键保留原先后。若十个数跨越几十亿个整数格，扫描空格会很贵。

为什么快或慢

快的原因是键直接指出计数格和输出区间，免去反复寻找。慢的情况是可能的键太多：少量输入并不能抵消巨大的计数表。

什么时候适合用

适合分数、年龄和有限编号。也适合充当基数排序的一轮，但那时一定要用保留相等项次序的版本。

最后把容易弄错的地方说清楚

空输入直接返回；单元素也无须建表。数值必须能准确对应离散整数格。只有“出现/未出现”的位图不能代替计数，它会丢掉重复项。把稀疏键压成连续编号需要先把不同键排好，这项准备工作也要计时。

参考来源 [1]

02 鸽巢排序

类型 算法 别名或相关名称 鸽笼排序；按完整键值分格的分配排序

先把要用的小动作讲清楚

把记录放进容器和给记录计数是两回事。一个容器里能保存完整数字、姓名等内容；相等数字仍是多条独立记录。这里容器可用列表：从尾部加入，稍后从头到尾读出。

巢的标签本来就有次序。先取标签 2 的全部记录，再取标签 3 的全部记录，就能保证不同键的大小次序；巢内只有同一种键。

基本思路

想象准备一排贴好整数标签的小盒子。每条记录只需走到自己的盒子，再按盒子标签从小到大收回。这个方法叫鸽巢排序。与计数排序的区别是：它先真的把记录装进去，而不只是统计份数再计算最终地址。

具体执行步骤

1. 找最小键和最大键，为中间每个整数建一个空巢。不要只准备“一个数字位置”：某个键可能有很多条记录，同一巢必须装得下它们。
2. 从左向右读输入。读到键 x ，就把整条记录追加到标签 x 的巢末尾。追加而不是放到最前面，是为了保留同键记录的原先后。
3. 从最小标签的巢开始，依次读出该巢的所有记录，追加到输出。这个巢空了再去下一标签；不用比较不同巢中的记录，标签已经决定它们的先后。
4. 最大标签的巢也收完后结束。整数为负时，把标签减去最小键就能转换成数组格号。如果只保存每巢的计数再重写裸整数，这就是它的简化键值变体。

跟着数字一步步做

1. 输入 [4,2甲,3,2乙]，建标签 2、3、4 的巢。读 4 后，巢 4=[4]；读 2甲后，巢 2=[2甲]；读 3后，巢 3=[3]；读 2乙后，巢 2=[2甲,2乙]。
2. 输出最初为空。收巢 2，得到 [2甲,2乙]；收巢 3，得到 [2甲,2乙,3]；收巢 4，得到 [2甲,2乙,3,4]。每次是搬一整条记录，不是只给输出改一个数字。

要做多少事 要用多少地方

每条记录搬进巢一次、搬出巢一次，还要走过全部巢。 n 为记录数， k 为从最小到最大键包含的整数格数，标准最好、通常、最坏均 $\Theta(n+k)$ 。存完整记录要 $\Theta(n+k)$ 额外空间；纯整数计数式可用 $\Theta(k)$ 。按输入顺序追加并收集的版本稳定。

为什么快或慢

小键域里，直接找标签很省事；巢很多时，空容器的建立和访问会浪费时间。多个容器的分配成本也可能高于一张连续计数表。

什么时候适合用

适合按少量整数类别整理完整记录。它也很好地说“先分配、后收集”的排序思路。

最后把容易弄错的地方说清楚

同一巢不能被后来的记录覆盖。普通桶排序一桶可以含多个不同键，鸽巢却一巢对应一个键。若用哈希表省去空巢，表的遍历顺序通常不是键的顺序；还得排列实际标签。空输入直接结束。

参考来源 [2] [1]

03 低位优先基数排序

类型 重要变体 别名或相关名称 最不重要位优先基数排序；LSD 表示 least significant digit

先把要用的小动作讲清楚

十进制数位的分量不同：21 的十位是 2、个位是 1。这里先把所有正整数写成同样宽度，7 可写成 07；加前导零不改变数值。

一轮“稳定分组”的操作是：按当前数位准备标签 0–9 的容器，从左向右追加记录，再从容器 0 到 9 顺序收回。数位相同的记录保留这一轮输入中的先后。这一点叫稳定性，不是可有可无的优化。

基本思路

先排个位真的能帮助排整个数吗？单独一轮当然不够。诀窍是下一轮按十位分组时，保住已经排好的个位顺序。这样十位先决定谁在前，同十位内部再由个位决定次序。这个从较小分量往较大分量处理的方法，就是低位优先基数排序。

具体执行步骤

1. 确定共同位数 d ，并决定一位的表示。十进制每位有 10 种值；程序也常把一个字节当一位，便有 256 种值。这里用十进制方便看过程。
2. 只看个位做一次稳定分组。不要看整数字大小，也不要同一桶里任意交换。收回结果只是按个位排好，可能看起来比原来更乱，这是正常中间状态。
3. 在这份新顺序上只看十位，再做同样的稳定分组。十位相等的记录按进桶次序收回，因此它们保留了上一轮的个位顺序。
4. 继续向左，每次在上一轮结果上处理下一位。处理完最左位才宣布完整有序。可用稳定计数排序实现每轮：计数、累计位置、保序写到输出缓冲，再交换两份数组的角色。

跟着数字一步步做

1. 输入 [21,13,12,11,22]。个位为 1 的记录依次是 21、11；个位为 2 是 12、22；个位为 3 是 13。收回得 [21,11,12,22,13]。注意 21 还在 11 前面，因此不是最终结果。
2. 在这个新顺序上看十位。十位 1 的记录现在依次是 11、12、13；十位 2 的是 21、22。收回得 [11,12,13,21,22]。11、12、13 的个位次序没有被第二轮打乱。
3. 如果第二轮把十位 1 的组倒过来，结果会含 [13,12,11]，上一轮成果就丢了。这解释了为什么每一轮必须稳定，而不是碰巧在本例稳定。

步骤状态图 C 比较 M 移动 S 已有序

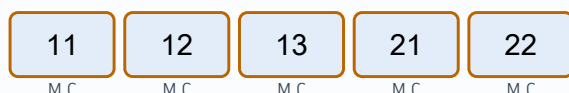
原始输入；先看个位



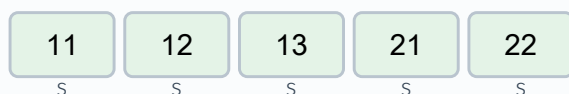
个位稳定分组后的中间结果



按十位稳定分组，保留个位顺序



两位都处理完，完整数值有序



要做多少事 要用多少地方

每轮读写全部记录，还要整理当前位的计数表。 n 为记录数， d 为位数， r 为一位可能取值数，完整版本最好、通常、最坏均 $\Theta(d(n+r))$ 。 d 和 r 固定时，工作随 n 线性增长。额外空间 $\Theta(n+r)$ 。各轮稳定，则整体稳定；已经有序通常也要跑所有轮。

为什么快或慢

一轮只判断一个小数位，免去反复比较完整键。键很长时，来回读写多轮也可能比比比较排序昂贵；基数过大还会浪费计数空间。

什么时候适合用

大量固定宽度整数、等长编号和可按字节编码的键。

最后把容易弄错的地方说清楚

负数不能直接把负号当普通数字排；需把键转成保持数值次序的固定宽度编码。有符号机器整数可翻转符号位后按无符号顺序处理。变长字符串的字典序另需结束标记或规范表示。重复键各自参加所有轮，不能合并成一条。

参考来源 [3] [4]

04 高位优先基数排序

类型 重要变体 别名或相关名称 最重要位优先基数排序；MSD 表示 most significant digit

先把要用的小动作讲清楚

本例把整数写成相同位数，所以最高位较小，完整数字一定较小。字符串则按字典序：先看第一字符，不同就决定大小；相同再看下一字符。

一个组的位置可以用起点和结束点记住。例如下标 $[0,2)$ 包含 0、1，不含 2，称为半开区间。递归的意思是在某个较小区间里再次执行相同分组动作，并先检查它是否已经无需处理。

基本思路

与其给所有记录每一位都排一遍，不如先看最重要的一位。最高位不同的记录已经分清前后，后面的工作只留给同组记录。这就是高位优先基数排序。一个单元组已经没有同伴需要比较，可直接停止，即使它还有很多位未读。

具体执行步骤

1. 在当前区间统计最高待处理位的各类数量。累计数量得到每组应占的连续输出区间；准备一份临时输出，保证不会在读取原记录时把它覆盖。
2. 按当前输入顺序把记录放入各组区间，再复制回当前区间。这样同位值的记录聚在一起；按顺序放也使这一版稳定。
3. 依次处理各组：空组和单元素组停止；已经到末位的组也停止。其余组把待处理位向右移一位，再执行上述统计、分配、处理组的步骤。不同高位组之间绝不混回去。
4. 变长字符串需要一个“字符串在这里结束”的特殊类别，排在任何实际字符前面。例如 '2' 与 '21' 共有开头 2，但 '2' 到此结束，应先输出；结束类别不再递归，否则会不停读不存在的字符。

跟着数字一步步做

1. 输入 [31,12,32,11,22]。十位 1 收 [12,11]，十位 2 收 [22]，十位 3 收 [31,32]，分配后 [12,11,22,31,32]。第一组只占下标 0-1，第二组占 2，第三组占 3-4。
2. [22] 只有一项，立即停止。处理 [12,11] 的个位：个位 1 的 11 先于个位 2 的 12，变 [11,12]。处理 [31,32] 的个位，仍是 [31,32]。
3. 把已处理的小区间放在原组的位置，得 [11,12,22,31,32]。没有必要拿 11 和 32 比较：十位已经保证第一组在第三组之前。

步骤状态图 C 比较 M 移动 S 已有序

原始输入；比较十位



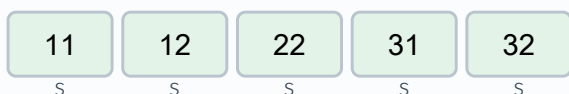
十位分组：1 | 2 | 3



只在十位为 1 的组内排个位



各组完成，合起来有序



要做多少事 要用多少地方

工作取决于实际读了多少字符，以及处理了多少组。 n 为记录数、 d 为最大键长、 r 为一位的取值数、 l 为执行分组的节点数，通用上界 $O(nd+rl)$ 。固定小字母表时常写 $O(nd)$ 。最好一轮就分开各键， $\Theta(n+r)$ ；通常快慢依赖共同前缀。缓冲与逐层计数信息用 $O(n+rd)$ 空间。这个保序缓冲版稳定。

为什么快或慢

高位一旦分开，后续就不用为两组互相比；单元素组还可早停。大量键有长共同前缀时，要逐位读过同样的开头，优势会减小。

什么时候适合用

适合字符串、长编号和容易从开头分组的键。

最后把容易弄错的地方说清楚

'12' 按字典序在 '2' 前面，按数值序却在后面；要先决定哪种次序，数值字符串可补同宽前导零。完全相同的键处理到结束位就停，但保留每条记录。小组可调用插入排序：保有序左部，逐个插入剩余项，减少分组开销。

参考来源 [5] [6]

05 二进制快速排序

类型 重要变体 别名或相关名称 基数交换排序；二进制 MSD 基数排序

先把要用的小动作讲清楚

二进制只是用 0、1 写数。三位从左到右分别值 4、2、1：101 就是一个 4、零个 2、一个 1，共 5；010 是 2；111 是 4+2+1=7；001 是 1。这些权重来自 $1+1=2$ 、 $2+2=4$ ，一位向左就翻倍。

先看最高位时，以 0 开头的三位数都比以 1 开头的小，因为前者最多 3，后者至少 4。最高位相同才需要继续看下一位。左、右指针只是记住当前在检查哪两个格子。

基本思路

先按最高位的 0、1 分成两边，再各自按下一位分。没有用一个数值枢轴比较全键，而是看一个固定数位。每次分组后，跨组顺序确定，组内还要继续看低位。

具体执行步骤

1. 先限定同宽非负整数，或把原键转成保持次序的同宽编码。选最高位开始。负数原补码的大小规则不同，不能直接照非负示例排。
2. 左指针从区间左端走：当前位是 0 就向右推进，直到遇到 1 或走完。右指针从右端走：当前位是 1 就向左推进，直到遇到 0 或与左指针交叉。
3. 两指针还没交叉时，左停的是应去右边的 1，右停的是应去左边的 0，把两条交换，左右各推进一格，再继续扫描。
4. 交叉后，左边当前位全是 0，右边全是 1。对两个区间分别用下一低位做相同步骤。这个过程不是每次重看最高位。
5. 区间只剩零条或一条时立即结束；所有位都处理完也结束，重复数保留所有副本。最后按整个数组读出结果。

跟着数字一步步做

1. 输入 [5,2,7,1]，三位编码 [101,010,111,001]。最高位应 0 左、1 右。左指针在 5 停住，右指针在 1 停住。
2. 交换 5、1，得 [1,2,7,5]。左跳过 2 的最高位 0，右跳过 7 的最高位 1，指针交叉。分区为 [1,2] | [7,5]。
3. 左区看第二位：1=001 是 0，2=010 是 1，位置已经正确，拆成两个单项。右区 7=111 是 1，5=101 是 0，交换成 [5,7]，也拆成单项。
4. 所有分支已经单项，无需继续扫描最后一位。输出 [1,2,5,7]。如果组中有两个相同键，就可能一直看到最后一位才结束，但两个记录都留下。

要做多少事 要用多少地方

每一位上，各未完成组合起来最多扫描 n 条；键宽 w 位时，总上界 $O(nw)$ ， n 是记录数。早早成为单项的分支会少扫几位；全等键的朴素版本仍扫 w 层。定宽机器字才允许把 w 当固定常数并说 $O(n)$ 上界。分组直接交换原数组，深度优先递归需 $O(w)$ 栈。通常不稳定。一次只看一位，不要把它和一轮处理多位的基数实现混成同一轮数。

为什么快或慢

每轮只分两个桶，无需大计数表；一次只处理 1 位，扫描轮数往往比多位基数排序多。

什么时候适合用

额外数组空间少、定宽整数键的环境，或演示按位分区。

最后把容易弄错的地方说清楚

负整数可以采用合适的保序变换；例如有符号定宽整数用翻转符号位的编码作排序键，原记录本身仍保持原值。quick 来自分区形状相似，它不是数值枢轴快排的新名称。

参考来源 [7] [3]

06 美国旗排序

类型 重要变体 别名或相关名称 原地多路 MSD 基数排序；名称来自连续色块般的分区

先把要用的小动作讲清楚

分好组不等于组内有序。我们先保证十位 1 的记录都在十位 2 前，再处理各组个位。这里仍从最高位开始。

交换必须保住被挤走的值：先暂存一个位置的记录，再将另一个写过去，最后写回暂存记录。next[c] 只是记录“第 c 组下一格尚未确认的位置”的整数，不是额外一份数据。

基本思路

高位分组常用一整排临时输出。内存紧时，能否直接在原数组里把各组摆成连续区间？美国旗排序的办法是先数清每组需要几格，再把放错组的记录交换到正确组的空位。最后的组像一条条色带，名称由此而来。

具体执行步骤

1. 统计当前位各类数量，按小位值到大位值划出互不重叠的区间。记固定起止边界，再给每组一个 next，初始指向该组起点。
2. 从第一组的 next 位置检查记录。若它确实属于本组，这一格可确认，next 加 1。若它属于别组，就与目标组 next 位置上的记录交换，并把目标组 next 加 1：至少一个记录已送进正确组。
3. 刚换回当前格的记录还没有确认，必须继续检查，不能直接跳过。若它又属于另一组，再送过去。当前组所有格确认后处理下一组；桶边界和 next 能告诉我们哪些位置已经完成，无须为每条记录添标记。
4. 所有组分配完后，在含两条以上记录且仍有下一位的组内继续同样过程。单元素、最后一位、字符串结束类别都是停止条件。可以用一张待处理区间清单代替函数递归。

跟着数字一步步做

1. 输入 [21,12,11,22]。十位 1 与 2 各有两条，所以组 1 的下标是 [0,2)，组 2 是 [2,4)，next 为 [0,2]。
2. 检查组 1 的下标 0：21 属于组 2，与其 next=2 上的 11 交换，得 [11,12,21,22]，组 2 的 next 变 3。再检查当前格 11，它属于组 1，所以组 1 的 next 变 1。
3. 下标 1 的 12 也正确，组 1 完成。组 2 的 next=3 上是 22，也正确。接着在 [11,12]、[21,22] 中分别看个位，均无需改变。最终 [11,12,21,22]。

要做多少事 要用多少地方

主要成本是每层统计、交换、扫桶表。n 为记录数、d 为最大键长、r 为基数、l 为处理的分组数，上界 $O(nd+rl)$ ，固定小基数时 $O(nd)$ 。最好一层便分开时 $\Theta(n+r)$ ，通常依赖共同前缀。深度优先实现的桶表和任务信息可用 $O(rd)$ 空间，无需第二份 n 条记录的数组。通常不稳定：远距离交换可改变同键记录先后。

为什么快或慢

它用原数组内的交换省去大输出缓冲。不过交换的内存访问不如连续写出规律，许多小组也带来管理成本，所以省内存并不自动等于更快。

什么时候适合用

适合可以逐位读取的长键、内存受限且不要求稳定的任务。

最后把容易弄错的地方说清楚

next 不得越过本组终点。已确认的格不能再当空位使用。原地表示没有按 n 分配第二份记录数组，不表示没有计数表或任务清单。实际实现通常用插入排序完成很小的组；字符串结束类别须排在字符之前并停止处理。

参考来源 [6] [8]

07 桶排序

类型 算法家族 别名或相关名称 箱排序；bin sort；分桶策略是一族算法

先把要用的小动作讲清楚

桶是一个能保存多条记录的容器。不同于计数格或鸽巢，一个桶可装多个不同值，例如区间 10–19 的桶装 11 和 13。

一个桶的区间必须在下一桶区间之前；如果前桶最大可能值小于后桶最小可能值，两个桶的记录就不必交叉比较。但同桶的 13 和 11 仍要整理。

基本思路

如果知道数字的大致分布，可以先把大问题切成许多小问题：小数放小范围桶，大数放大范围桶，再在每个桶里整理。这叫桶排序。省下的工作来自每个桶很短，并不是“进桶就排完”。

具体执行步骤

1. 先定义有序的数值区间及边界归属。例如整数 0–99 可按十位分十桶。也可用不等宽区间，让常见范围分得更细。每个值必须恰好属于一个桶。
2. 从左到右读输入，按区间把整条记录追加到相应桶。记录与附带信息一起存；重复值也全部加入。
3. 把每个桶内部排好。本例用插入法：第一项暂当有序；取下一项，向右挪动比它大的前项，再把它插入空出的正确位置。重复到桶尾。只有一条的桶不需要这个步骤。
4. 从最小区间桶开始，顺序输出其排好的全部记录，再输出下一桶。这样桶内有序、桶间也有序。追加保序，且同值插到旧值之后时，整套方法保持稳定。

跟着数字一步步做

1. 输入 [42,13,35,11,41]，用 10–19、20–39、40–49 三个桶。42 入最后桶；13 入首桶；35 入中桶；11 追加到首桶；41 追加到最后桶。得到 [13,11] | [35] | [42,41]。
2. 首桶取 11，13 比它大，先把 13 右移，再写 11，变 [11,13]。中桶 [35] 不动。最后桶取 41，把 42 右移，变 [41,42]。
3. 依桶顺序连接 [11,13]、[35]、[41,42]，得到 [11,13,35,41,42]。若省掉桶内整理，连接出来的 [13,11,35,42,41] 仍错。

要做多少事 要用多少地方

分配和收集都走一遍数据，另有各桶内部的工作。 n 是记录数， b 是桶数， m_i 是第 i 桶大小。插入法给上界 $O(n+b+\sum m_i^2)$ ；按桶内逆序对 l_i 更精确是 $O(n+b+\sum l_i)$ 。最好 $O(n+b)$ 。独立均匀分布且 b 与 n 同阶时，期望也为 $O(n+b)$ 。最坏所有记录挤一桶并逆序， $O(n^2+b)$ 。额外空间 $O(n+b)$ ，稳定性取决于桶内方法。

为什么快或慢

分散得好，每个桶只有很少记录，小整理相加也少。分散得差，大桶就保留了原来的大难题。换成最坏 $O(m \log m)$ 的桶内排序可保护最坏情况，但成本模型随之改变。

什么时候适合用

已知范围、分布比较平滑的数值或空间坐标。先确认范围假设，再选择桶数。

最后把容易弄错的地方说清楚

最大值必须进最后桶，不能算出不存在的下标。仅按哈希值分桶不能直接按桶编号输出，因为哈希编号没有大小意义。桶过多会增加空桶费用。特殊值如 NaN 要先规定顺序，不能假设普通大小比较有效。

参考来源 [9] [10]

08 闪排序

类型 算法 别名或相关名称 FlashSort；按类别原地循环分配的直方图排序

先把要用的小动作讲清楚

数组是一排按下标定位的格子，本例下标从 0 数。类别区间 [2,4) 是下标 2、3。它们只是属于某一数值范围的两格，格里的两个值仍可能倒着。

start 是每类固定起点，end 是移动的结束指针。向 end-1 写入一条正确类别记录后，把 end 减 1；这样右侧已放好的部分扩大，左侧待分配的部分缩小。暂存被挤出的记录能防止覆盖导致丢数据。

基本思路

如果不想把每个桶另放在一份大数组里，可以先算出各类应占的区间，再在原数组里做一串搬运：一个数占用自己的类位置，同时接过被挤走的数，接着安置它。Flashsort 用数值范围估计类别，并用这种闭合搬运循环完成分配。它只把数据放到大致正确的区域，最后仍须整理区域内部。

具体执行步骤

1. 找最小值 min 和最大值 max。若相等，已经全体同值，结束。否则选 m 类，用 $c(x)=\text{floor}((m-1)(x-\text{min})/(\text{max}-\text{min}))$ 分类；floor 是向下取整，比如 1.4 变 1。每类只代表一段范围，不代表某个记录的最终名次。
2. 数各类数量，累计得到 end；从这些累计数得到固定 start。若 $\text{start}[c]<\text{end}[c]$ ，该类还有待分配的格。从第一个这样的区间起点取出一条记录，暂存在手中，该位置成为循环起点。
3. 看手中记录所属类，把它写到该类 end-1，先保住那里原来的记录，再将 end 减 1。现在手中换成被挤出的记录，按它的类别继续。写回最初起点时，循环闭合，不再重复处理起点旧值。
4. 继续寻找还有待分配格的类。若取出的数本来就在其类最后一个待填格，确认这一格并减 end 即可。所有 $\text{end}=\text{start}$ 时类别分配完成。接着对各类用插入排序：逐个插入，右移比当前值大的前项。未经这步不能说排序完成。

跟着数字一步步做

1. 输入 [40,20,32,10,31]，min=10，max=40，m=3。10、20 属于类 0，31、32 属于类 1，40 属于类 2。数量 [2,2,1]，start=[0,2,4]，end=[2,4,5]。
2. 从下标 0 暂取 40。40 写入下标 4，接过 31，end[2]=4；31 写入下标 3，接过 10，end[1]=3；10 写入下标 1，接过 20，end[0]=1；20 写回下标 0，循环闭合，end[0]=0。数组为 [20,10,32,31,40]。
3. 类 0、2 的 end 已等于 start。类 1 还剩 [2,3)；32 正好占这类最后待填位置，确认后 end[1]=2。注意类别正确但仍有 20>10、32>31。分别插入 10 和 31 后才得到 [10,20,31,32,40]。

步骤状态图 C 比较 M 移动 S 已有序

原始输入；暂取下标 0 的 40



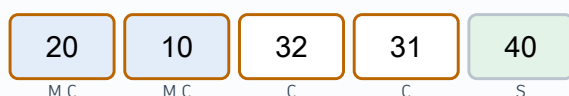
40 到下标 4，手中变成 31



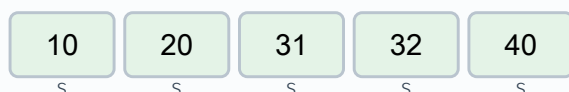
31 到下标 3，手中变成 10



10 到 1，20 回到 0；32 确认，分类完成



类内插入排序收尾，全体有序



要做多少事 要用多少地方

分类要读全部记录，分配也逐条安置，再付类内整理费用。 n 为记录数、 m 为类数，前两阶段 $\Theta(n+m)$ 。 m 与 n 同阶且数据适当分散时，典型期望接近 $\Theta(n)$ ；最好 $\Theta(n+m)$ ，全同值可 $\Theta(n)$ 结束。插入收尾最坏 $\Theta(n^2+m)$ 。 $start/end$ 表用 $\Theta(m)$ 空间；循环搬运通常不稳定。线性速度依赖分布，不能推广到任意数值输入。

为什么快或慢

估计类别能把全局混乱限制到小范围，循环搬运又省去整份记录缓冲。离群值会拉大数值范围，让多数数据落入少数类，导致收尾很重。

什么时候适合用

适合分布较平滑的大批数值，尤其在不要同键保序且希望节省记录缓冲时。

最后把容易弄错的地方说清楚

本说明保留 $start$ 与 end 两张小表，使未分配区域容易判断；更省表的实现有专门领头规则。要防 $max=min$ 除零、浮点舍入和类别下标越界。Flashsort 的名称曾用于其他算法，这里专指 Neubert 的数值分类方法。

参考来源 [11] [12]

09 字典树排序

类型 算法家族 别名或相关名称 前缀树排序；trie 读作 try；数字树排序的相关形式

先把要用的小动作讲清楚

字符串就是按次序排的一串字符，例如 '21' 有字符 '2'、'1'。字典序先比较开头，不同便决定顺序；相同再往后看；前面全相同则较短字符串在前。它与数值大小不是同一规则。

树由节点和连接组成，根是起点。这里每条连接写一个字符。沿根到某节点的连接读出来，就是一个前缀；同样开头的字符串共用这条路。节点还要保存“哪些记录在这里结束”，不能只保存孩子。

基本思路

如果很多键开头一样，能不能把共同开头只安排一次？把每个字符当作岔路选择，所有字符串就能装入前缀树，也叫字典树。输出时沿较小字符的岔路先走，自然就得到字典序。

具体执行步骤

1. 建立一个空根。每个节点要能按字符找到孩子，并有一个终点记录列表。孩子可用固定字符格，也可用带标签的连接；无论哪种，输出时必须能按标签大小访问。
2. 插入一个字符串，从根读第一字符，找对应孩子；不存在就新建。再读下一字符继续，直到字符串结束，把整条记录追加到当前节点终点列表。重复键共享路径，但终点列表保存每条记录。
3. 输出时先读当前节点的终点列表，再从最小字符孩子走到最大字符孩子。对孩子重复同样动作。先终点后孩子，是因为 '2' 应在 '21' 前，不是因为终点数值更小。
4. 节点没有终点也没有孩子就返回上一级，继续下一个未访问孩子。根及所有分支走完结束后。可以把“还要回来走哪条分支”记在任务栈里；不允许只保留一条路径然后忘记其他分支。

跟着数字一步步做

1. 插入 ['21','12','2','12']。'21' 建根→2→1，在末节点存一条记录；'12' 建根→1→2；'2' 不再建新路，只在根的孩子 2 存终点记录；第二个 '12' 在原终点追加。
2. 输出先走字符 1 路，到 '12' 终点连续输出两份。再走字符 2 路，先输出这个节点的 '2'，再走孩子 1，输出 '21'。得 ['12','12','2','21']。
3. 若要数值大小，先规范为两位 ['21','12','02','12']。分支 0 先输出 '02'，再分支 1 输出两份 '12'，最后 '21'，代表 [2,12,12,21]。

要做多少事 要用多少地方

主要动作是读取字符、建或走节点，再输出记录。S 为全部输入字符数，n 为记录数；固定小字符集且按字符直接找孩子时可 $O(S+n)$ 。若每个节点都有 r 个格且遍历全部格，还需 $O(rV)$ ，V 是节点数。稀疏孩子若要排序，也另有费用。空间通常 $O(S+n)$ ，密集节点可更大。终点保序列列表使它稳定。

为什么快或慢

共同前缀共享结构，免去反复完整字符串比较。但节点和指针占空间，分散访问也可能比连续数组慢。

什么时候适合用

适合字符串排序和同时需要前缀查找的任务。

最后把容易弄错的地方说清楚

空字符串结束在根节点，输出时先取。只存唯一键的集合式字典树会丢重复项，必须改为计数或记录列表。只有一个孩子的长路径可以压成一条带多字符标签的边，但比较和输出规则仍相同。

参考来源 [13] [14]

10 爆裂排序

类型 算法家族 别名或相关名称 动态桶字典树字符串排序；含 C-burtsort、采样 Burtsort 变体

先把要用的小动作讲清楚

字典树的一个节点表示共同前缀，按下一字符分岔。桶是暂时存许多记录的列表；记录指针只是指向原字符串的位置，不必复制整串文字。

高速缓存是处理器附近较小但读得快的存储。数据能集中在小范围里处理，往往比到大内存里来回找更省时间。桶容量阈值是决定何时拆桶的数量，不是排序正确性的条件。

基本思路

完整字典树可能建太多节点；直接比较字符串又可能一遍遍看相同开头。Burtsort 先只建少量前缀节点，下面挂桶。桶不大就先收着，太大才按下一字符拆成更细的桶，这个拆开动作叫“爆裂”。最后每个小桶单独排序。

具体执行步骤

1. 开始时只有一个根桶。把字符串沿已经存在的前缀分支走到叶桶，追加其记录指针。这个桶里的字符串已共享沿路前缀，桶内比较以后可以从前缀之后开始。
2. 桶内记录数超过阈值时，把该桶换成一个下一字符分支节点。依次读桶中每个键尚未处理的字符，按字符送到孩子桶。键已在这里结束的，存终点列表；不能把不存在的下一字符继续分下去。
3. 如果某个孩子桶仍很大，继续拆它。全部输入加入后，按字符从小到大遍历分支，终点记录先于其延长字符串输出。
4. 遇到保留的叶桶时，把它的未处理后缀完整排好再输出。可用比较归并：分成小份至单项，然后反复比较两份当前最前项、取较小者；相等保原序。这里只要正确后缀排序，不得把桶直接当成有序。

跟着数字一步步做

1. 输入 ['21','12','11','22','12']，教学阈值 2。最初 '21'、'12' 都进根桶。加入 '11' 后有三项，根爆裂：字符 1 桶=['12','11']，字符 2 桶=['21']。
2. 加入 '22' 后，字符 2 桶=['21','22']。加入第二个 '12' 后，字符 1 桶变三项，再按第二字符拆成路径 11=['11'] 与路径 12=['12','12']。重复终点保留两条，不再为它们无限拆桶。
3. 遍历先输出 '11'，再两份 '12'。字符 2 的叶桶共享开头 2，只需比较后缀 1、2，输出 '21','22'。结果 ['11','12','12','21','22']。

要做多少事 要用多少地方

实际费用有两块：沿树读共同前缀，以及给各叶桶排后缀。D 是前缀处理动作数， m_i 是桶大小， c_i 是一次后缀比较的字符读取成本，比较收尾可写 $O(D + \sum c_i m_i \log m_i)$ 。n 为记录数、d 为最大键长时，常规比较收尾上界可达 $O(nd \log n)$ 。最好和通常不能仅由 n 决定，取决于键长、前缀和阈值。桶、指针和节点一般有 $O(n+S)$ 空间上界，S 是总字符数。基本实现通常不承诺稳定。

为什么快或慢

它减少重复读前缀，并让最后的局部工作更容易留在缓存。桶阈值、机器缓存、字符串分布不合适时，拆桶和分配节点也可能抵消收益。

什么时候适合用

适合大型字符串集合，尤其是字符读取和慢内存访问占主要成本时。

最后把容易弄错的地方说清楚

保序分配加稳定桶内排序可构造稳定版。C-burtsort 会复制剩余后缀以改善局部性；采样变体先估计分支结构。论文中的倍数加速是具体实验结果，不是每种输入都固定快几倍。

参考来源 [15] [16]

11 Spreadsor 混合分布排序

类型 算法家族 别名或相关名称 混合基数/比较排序；含整数、浮点和字符串专用版本

先把要用的小动作讲清楚

把整数右移 s 位，就是丢掉二进制末尾 s 位；对非负整数等同除以 2^s 后向下取整。例如右移 3 位就是按每 8 个整数一组分段。低位仍决定组内次序。

混合算法是在不同规模选择不同动作。这里大区间按键的高位分配，小区间调用有最坏保证的比较排序。比较归并可把区间分至单项，再把两份有序记录从头逐项取较小者合成一份。

基本思路

固定每轮处理同样多位，有时桶太多，有时又分得太粗。Spreadsor 先看这一批数到底跨多大范围、还剩多少条，再选择分组位数。它也会承认小任务没必要建桶，直接用比较法结束。

具体执行步骤

1. 扫描当前区间，找最小键、最大键并记录大小。全部键相同就停；区间很小，就调用比较收尾。调用意味着对当前这一段得到完整有序结果，而不是只再分一次组。
2. 大区间根据大小和不同高位范围选右移量 s ，让桶表不过大。按“右移后的键减去右移后的最小键”编号，统计每桶需要几格，再累计成桶的连续区域。
3. 在原数组里交换错放记录：当前记录送目标桶的下一未空格，被挤回的记录继续安置；推进相应位置计数，直到桶区域正确。不同桶的高位关系已决定它们的先后。
4. 对仍很大的桶处理剩余不同位。某桶继续分配不划算时，换用有 $O(m \log m)$ 最坏保证的比较方法完成它。最后每个桶区间都有序，全数组才有序。选择位数和切换阈值是具体实现的策略，不是任意设置都能继承同一个复杂度证明。

跟着数字一步步做

1. 教学输入 [17,2,18,1,9,8]，右移三位即除以 8 后取整，得到桶 0 收 2、1，桶 1 收 9、8，桶 2 收 17、18。各有两项，所以它们的区域为 [0,2)、[2,4)、[4,6)，下一未填编号为 [0,2,4]。
2. 第 0 格的 17 属于桶 2，与该桶下一格第 4 格的 9 交换，成为 [9,2,18,1,17,8]，桶 2 的下一位置改成 5。第 0 格带回的 9 属于桶 1，与第 2 格的 18 交换，成为 [18,2,9,1,17,8]，桶 1 的下一位置改成 3。
3. 仍看第 0 格：18 去桶 2 的第 5 格，换回 8，得到 [8,2,9,1,17,18]；8 再去桶 1 的第 3 格，换回 1，得到 [1,2,9,8,17,18]。现在第 0 格的 1 和第 1 格的 2 都属于桶 0，确认即可。另两桶的未填位置也已走到各自末尾，分配结束。
4. 桶 1 仍是 [9,8]，内部比较后换为 [8,9]；另两桶无需改动。拼成 [1,2,8,9,17,18]。真实实现可能直接对这么小的输入使用比较排序；本例是把大输入才会触发的分配动作放大展示。

要做多少事 要用多少地方

实际工作是在每层统计和搬运，再对剩余小问题比较。 n 为记录数， K 为固定整数键位数。Boost 文档给最好 $O(n)$ 、平均性能模型 $O(n\sqrt{\log n})$ 、最坏 $O(n \cdot \min(\log n, K))$ ；平均模型并非任意分布保证，也不自动适用于任何自造阈值。桶表和递归信息随键长、配置变化，不是 n 条稳定输出缓冲。普通版本不稳定；字符串另计字符读取。

为什么快或慢

大任务可以靠键结构快速分开，小任务避免建表管理费用，比较后备保护困难情况。缓存和位提取成本仍能影响实际速度。

什么时候适合用

适合大批整数、可正确编码的浮点或字符串，而且不要求同键记录保持原先后。

最后把容易弄错的地方说清楚

自定义位提取必须与比较次序一致：不能比较按一种规则，桶编号却按另一种。浮点须用数值有序编码，并规定 NaN、正负零。Sortsort 是具体混合族；普通“分桶后小组排序”不能直接采用其文档性能界。

参考来源 [17] [18]

12 多键快速排序

类型 算法 别名或相关名称 三路基数快速排序；三向字符串快排；这些是同一方法的名称

先把要用的小动作讲清楚

字符串按字典式规则排：先看首字符，不同就决定先后；相同才看下一位。因此“13”在“2”前，和整数 $13 > 2$ 是两套规则。

字符位置 d 从 0 开始。交换的是整条记录引用或地址，不是在字符串内部交换字符。

字符串在 d 结束时用小于任何真字符的结束标记；“2”这个前缀就在“20”前。

小、等、大三组是比较当前字符的结果。只有等组共同确定了这一位，能移到下一位。

基本思路

每次只看一位字符做三路分组。小、大两组仍在这一位继续排；等组才看下一位。这样共同前缀不必在每次完整字符串比较时从头重复读。

具体执行步骤

1. 空组、单条组直接完成。其余组从一条字符串取位置 d 的字符作标准 c ，若已结束就取结束标记。整排从 $d=0$ 开始。
2. lt 、 i 从本组第一格开始， gt 从最后格开始。 lt 之前当前字符小； $lt..i-1$ 等； $i..gt$ 未检查； gt 之后大。
3. 看 i 的字符：小于 c ，交换整条引用与 lt ，前进 lt 、 i ；等于 c ，只前进 i ；大于 c ，与 gt 交换、后退 gt ， i 不动，必须再查新换来的记录。
4. 未知区空后，小组和大组继续用相同 d ，因为它们这一位的内部顺序没定。中组若 c 是真字符，用 $d+1$ ，因为这一位已共同确认。
5. 若 c 为结束标记，中组全在这里结束，没有下一位，直接完成，仍保留全部记录。随机选标准可降低持续偏分区的机会。
6. 小组也可直接插入：拿住下一引用，从 d 起逐字符比，到第一处不同或结束才决定大小；右移更大引用再插入，不必重新比已经确定的前缀。

跟着数字一步步做

1. 输入字符串 [“21”, “13”, “11”, “20”, “2”], $d=0$ ，标准取“21”的首字符“2”。第一条相等；“13”小，交换到左，成 [“13”, “21”, “11”, “20”, “2”]；“11”也小，成 [“13”, “11” | “21”, “20”, “2”]，后三条首字符都等于“2”。
2. 左组在 $d=0$ 都为“1”，前进 $d=1$ 。标准取“13”的“3”，“11”的“1”更小，交换得 [“11”, “13”]，单条组停止。
3. 原等组在 $d=1$ 为 [“21”, “20”, “2”]，标准“1”。“20”的“0”小，换前成 [“20”, “21”, “2”]；“2”的结束标记更小，再换成 [“20”, “2” | “21”]。
4. 小组 [“20”, “2”] 仍在 $d=1$ ，标准“0”；结束标记更小，成 [“2”, “20”]。结合单条“21”，为 [“2”, “20”, “21”]。
5. 合起来 [“11”, “13”, “2”, “20”, “21”]。五条都各一次，排的是字符先后，不是整数值。

要做多少事 要用多少地方

设 n 为条数、 L 为最长字符数，假定按位置读字符是常数成本。随机标准期望 $O(n \log n + nL)$ ，最坏可 $O(n^2 + nL)$ ：不等字符分组可很偏，而相等分支逐位前进最多总计约 nL 次。全同短串最好可 $\Theta(n)$ ，全同长度 L 串需 $\Theta(nL)$ 。交换引用临时 $O(1)$ 空间；直接调用栈期望 $O(\log n + L)$ ，最坏 $O(n + L)$ 。不稳定。复制字符串本体或预先解码另计成本。

为什么快或慢

共同前缀沿等分支只逐位确认，后面不再从第一字符重复读。前缀少或标准持续偏时收益小；结束标记使前缀与长串关系也正确。

什么时候适合用

共享前缀字符串和字典序多字段键。要整数顺序时，不能随便拿不等长数字串代替。

最后把容易弄错的地方说清楚

明确字符按字节、Unicode 码点还是语言规则排。等宽、补前导零的非负数字串才能同时符合普通数值顺序。虽有 radix 别名，这里用字符比较，不是计数桶。

参考来源 [\[19\]](#) [\[20\]](#) [\[21\]](#)

二 利用已有顺序的自适应方法

在少数长有序段条件下先看自适应归并；在少逆序条件下看插入。它们依据不同“近有序”指标，不互相给固定名次。随后看附带空间或随机条件的方法，以及仍可退化为平方工作的提取方法。最坏保障与自适应能力分别说明。

13 自然归并排序

类型 算法家族 别名或相关名称 自然合并排序；连续有序段自适应归并

先把要用的小动作讲清楚

连续段是数组中挨在一起的记录。这里的有序段允许相等，例如 [1,1,4]；一旦后一个数更小，这一段就结束。合并不是把两段直接拼接。各看最前面的未用记录，取较小者并推进它那边的位置；某边用完，再复制另一边的尾巴。

基本思路

有些乱序数组不是处处混乱，它们已经有几段排得很好。例如 [1,4,2,3,0,5] 里，1 到 4、2 到 3、0 到 5 都在上升。自然归并排序先认出这些现成的连续有序段，不把它们拆散，再两两合并成更长的段，直到只剩一段。

具体执行步骤

1. 从左边开始扫描。只要下一个数不小于当前数，就继续当前段；遇到下降，记下段的起点和终点，开始新段。扫完最后一格也要记下最后一段。这样得到的每段已经内部有序。
2. 本轮依次配对相邻段：第 1 与第 2 段、第 3 与第 4 段……。对一对段，准备输出缓冲区和两个位置。比较两个位置上的记录，把较小者写到缓冲区末尾，只前进被选中一边。相等时取左段，使原来靠左的相同值继续在前。
3. 一边到头后，另一边的尾部已经有序，直接接上。将合并结果写回原来的两段区域，记成一个新段。若段数为奇数，本轮最后一段没有伙伴，就原样带到下一轮。
4. 完成这一轮后，段数大致减半。按相同办法做下一轮。只有一段且覆盖整个数组时停止。本条用简单的两两配对；TimSort、Powersort、Peeksort 会更细致地安排哪些相邻段先合并。

跟着数字一步步做

1. 输入 [1,4,2,3,0,5]。1≤4，但 4>2，因此第一段 [1,4] 到此结束。2≤3，但 3>0，第二段是 [2,3]。最后的 [0,5] 为第三段。段状态：[1,4] | [2,3] | [0,5]。
2. 第一轮合并前两段：1 与 2 比，取 1；4 与 2 比，取 2；4 与 3 比，取 3。右段已用完，接上 4，得到 [1,2,3,4]。最后的 [0,5] 没有伙伴，保留。状态：[1,2,3,4] | [0,5]。
3. 第二轮先比 1 与 0，取 0；随后左段的 1、2、3、4 都小于右段当前的 5，依次取走；左段用完，接上 5。状态成为 [0,1,2,3,4,5]，现在只有一个完整段，结束。
4. 已排段不等于最终位置：第三段本来有序，但其中的 0 仍在最后一次合并时搬到全数组最前面。

步骤状态图 C 比较 M 移动 S 已有序

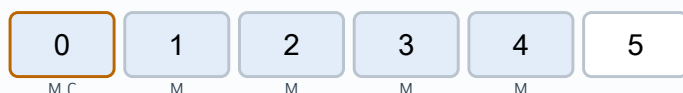
三个连续有序段：[1,4] | [2,3] | [0,5]



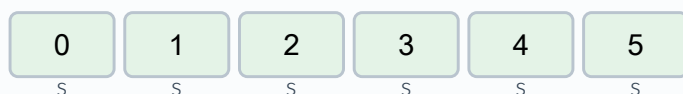
合并前两段，0 所在段仍待处理



下一轮先取右段的 0



一个段覆盖全体，排序结束



要做多少事 要用多少地方

找初始段要看一遍输入。每轮合并中，每个记录最多读写一次，花 $O(n)$ ； r 个初始段经过约 $\log_2 r$ 轮变成一个，所以总上界 $O(n + n \log r)$ 。 n 是记录数， r 是初始有序段数。只有一段时最好 $\Theta(n)$ ；无明显顺序的典型输入和最坏情形为 $O(n \log n)$ 。普通数组合并需 $\Theta(n)$ 额外缓冲区，链表可用改连接来合并。上述左段先取相等值的版本稳定。

为什么快或慢

现成的长段减少了需要合并的轮数。相比从单元素开始归并，它省掉了早已正确的小范围工作。但简单配对遇到长度差异很大的段时，可能让同一个长段被多搬几次。

什么时候适合用

适合已有连续顺序的记录、链表和按段合并的文件处理。它也是理解自适应归并家族的入口：自适应表示会利用输入原有的顺序。

最后把容易弄错的地方说清楚

可扩展扫描器，识别严格下降段后反转；必须严格下降，含相等记录的一段不能整体反转，否则破坏稳定性。本例只扫描不下降段。合并必须处理所有尾部记录；若采用从后向前写的优化，遇到相等值要先写右段，才与左段优先的前向规则一致。

参考来源 [22] [23]

14 TimSort

类型 算法 别名或相关名称 Timsort；自然归并、二分插入与跳跃合并的混合算法

先把要用的小动作讲清楚

有序段是一段相邻且不下降的记录。合并两段时各看最前面的未用项，输出较小者；相等时先取左段。这个动作本身和自然归并相同。

栈可以想成一摞纸。新纸压在顶上，取纸也从顶上拿。这里栈里放的是段的起点和长度等说明，不是把整段记录搬到纸堆里。

二分插入是先不断比较有序部分的中间值来找位置，再倒着右移记录腾出空位。它用于补长短段，不能省掉实际搬动。

基本思路

实际数据常常有现成的顺序，但段可能有长有短。把一个很长的段同许多短段一条条合并，会反复搬动长段。TimSort 一方面保留已经有序的段，一方面给过短的段补几条记录，再用一套规则决定何时合并。合并时如果一边连续提供较小值，它还能一次找出一整块直接复制。先理解这三个动作：找段、安排合并、成块复制。它不是一种只靠神奇名称就能完成的比较。

具体执行步骤

1. 从未处理位置找下一段：连续相等或上升就继续；若一开始严格下降，则继续找严格下降的末尾，再从两头交换把它反转。含相等值的下降段不能这样反转。每一段最后都成为不下降段。
2. 选定一个短段目标 minrun，真正实现通常选几十个记录。如果找到的段不足 minrun，就把后面几条尚未处理记录逐条二分插入这个段，直到达到目标或输入结束。插入时找旧相等值后面的位置，再从右向左搬动。因此补长后的段仍有序且稳定。
3. 把段的起点和长度压入栈。下面是修正后的经典合并规则：将栈顶四个长度按从老到新叫 W、X、Y、Z。若 $X \leq Y+Z$ 或 $W \leq X+Y$ ，需要合并；让 Y 与两边较短的 X 或 Z 合并。如果前两个条件都不成立，但 $Y \leq Z$ ，则合并 Y、Z。缺少某段时忽略含它的条件。两段合并后，用新长度和起点替换它们，重新检查，直到没有条件要求继续合并。
4. 合并的必须是相邻段。最直接的方法是各设一个当前位置和输出缓冲区，反复取较小值，最后接上尾部。常见节省空间办法只暂存较短段：若左段较短，从左往右写；若右段较短，从右往左写，比较较大的末尾值。前向遇到相等先取左，后向遇到相等先写右，两种方向都保持原顺序。
5. 如果连续多次都是同一边胜出，先不逐个比较，改用跳着找边界：检查距离 1、2、4、8……处的值，直到发现不能继续胜出的值或到达末尾；再在最后两次检查之间二分查出准确边界，把已确认的连续块复制过去。这称为 galloping，也叫疾驰搜索。随后继续普通合并；实际实现会调整启用它的门槛。
6. 读完输入后，即使栈上长度条件没有触发，也必须完成剩余合并。经典版本持续在栈顶选择相邻伙伴，通常让倒数第二段与两边较短的一段合并，直到栈上只有一段。

跟着数字一步步做

1. 为让小例子显示段结构，教学目标取 minrun=2，输入 [1,4,2,3,0,5]。先找到 [1,4]，长度栈 [2]。再找到 [2,3]，栈 [2,2]。此时最新两段一样长，满足 $Y \leq Z$ ，因此合并。
2. 合并时先取 1，再取 2、3，最后接上 4，数组前四格成为 [1,2,3,4]，栈变成 [4]。随后找到 [0,5]，栈为 [4,2]。4 不小于等于 2，不需立即合并。输入已经结束，于是强制最终合并，取 0、1、2、3、4、5，得到 [0,1,2,3,4,5]。
3. 补长动作可以单独看 [2,5,1,4]，教学目标为 4。现成段只有 [2,5]。取 1，找位置后把 5、2 右移，成为 [1,2,5,4]；再取 4，右移 5，成为 [1,2,4,5]。这四格才可以作为一个完整段入栈。
4. 疾驰动作另看 [1,2,3,4,8] 与 [5,6,7,9]。右边当前值是 5。左边第 1、2、4 项分别是 1、2、4，都能先输出；第 8 项不存在，于是将搜索限制在第 4 项之后到末尾。二分确认 8 是第一个不能先出的值，一次复制 [1,2,3,4]。剩下继续按头比较。小例只展示动作，不保证真实门槛会在四次胜出后启动。

要做多少事 要用多少地方

找段先读一遍数据。补长只在短片段里做插入；合并规则防止许多长段被无节制地重复搬动。每个记录经过的合并层数受到控制，典型与最坏总时间 $O(n \log n)$ ；一个现成段最好 $O(n)$ 。以 r 个初始段衡量，可给出 $O(n+n \log r)$ 自适应上界， n 为记录数。疾驰减少某些合并中的逐项比较，但不能免去输出记录的动作。常见额外空间 $O(n)$ ，包括较短段缓冲区；栈只保存段说明。算法稳定。

为什么快或慢

现成顺序不用重排，短段补长减少管理开销，长度规则减少长段的反复搬家。疾驰适合一边连续占优的合并；两边交替胜出时，它未必省事，所以会调整门槛。

什么时候适合用

适合一般记录、文本、已有部分顺序且需要稳定输出的数据。理解它时可先掌握普通合并和二分插入，再学习长度规则；初学者不必先背栈上的不等式。

最后把容易弄错的地方说清楚

本条解释修正后的经典 TimSort 调度，检查四段条件很重要，旧的不完整规则出现过栈问题。现代 CPython 保留找段、插入和疾驰机制，但合并顺序改用 Powersort，不应把所有实现都当作旧规则。minrun=2、4 是教学值。空输入直接返回；最后不足目标长度的段保留实际长度。

参考来源 [24] [25] [26]

15 Powersort

类型 重要变体 别名或相关名称 按节点层级安排归并的自然归并变体；含多路 Powersort

先把要用的小动作讲清楚

合并两条已排队列时，比较各自最前未用项，输出较小者；一边用尽后接上另一边。合并 a 与 b 个记录大约要处理 $a+b$ 个记录，因此先后顺序会影响累计搬动量。

栈只保存尚待合并的左段和一个边界编号。压入是在顶上加一项，弹出是取走顶上一项。段的真实记录留在数组中。

相对位置是把整条数组当作长度 1 的尺子。下文的中点说的是一段在尺子上的位置，不是那段数值的平均数。floor 表示向下取整，如 $\text{floor}(2.7)=2$ 。

基本思路

想把短队伍和长队伍并成一支大队伍，先并哪两支会影响大家要重新走几次。Powersort 用各段在输入里的长度和位置来安排，而不是比较段中数值的大小。它把整条数组想成反复对半划分的尺子：两段的中点在哪一层分到不同格子，就给它们之间的边界一个层数。这个层数叫 power。层数大表示靠下的小范围合并，应较早做；层数小表示靠上的大合并，应稍后做。

具体执行步骤

1. 本条基础版只找最大的不下降连续段。记段的起点 s 和长度 L ，编号从 0 开始。总长为 n 时，相对中点 $\mu=(s+L/2)/n$ 。保留最近发现的原始段，下一条边界的层数必须用它和下一原始段计算，不是拿已经合并后的大段重算。
2. 为相邻两个原始段找 power。先把尺子分成两格，编号 0、1；看两个中点是否分在不同格。若不是，再分成四格，然后八格，直到第一次分开。分成 2^p 格时，格子编号为 $\text{floor}(2^p \mu)$ ，第一次不同的正整数 p 就是 power。两个不同位置总会被足够细的格子分开。
3. 实现栈调度时，另保留一个当前段 C 。开始 C 是第一段。发现下一段 N ，算它与前一原始段之间的 p 。只要栈顶保存的旧 power 大于 p ，就弹出该项，把它保存的左段与 C 稳定合并，并让合并结果成为新的 C 。这样较深的小合并 in 较浅的大边界进入前先完成。
4. 没有需要弹出的项后，将一项 (当前 C, p) 压栈，再把 C 改成新段 N 。继续读下一原始段。栈底到栈顶的待办层数保持递增；记录的是待合并边界，不是元素优先级。

5. 输入结束时，栈项仍是当前段左边的待办段。逐项从顶上弹出，将它的左段与 C 合并，更新 C。最后 C 覆盖全数组。普通稳定合并用两头指针，遇到相等先取左边记录。
6. 计算层数可用整数避免小数舍入：把中点写成 $(2s+L)/(2n)$ ，按每一层比较两个整数商 $\text{floor}(2^p(2s+L)/(2n))$ 。实现中也可逐位处理这些分数，避免整数溢出；数学上的格子规则不变。

跟着数字一步步做

1. 输入 [1,8,2,0,3,4,5,6]。只扫描不下降段，得到 $R1=[1,8]$ 、 $R2=[2]$ 、 $R3=[0,3,4,5,6]$ ，长度分别为 2、1、5，起点为 0、2、3，总长 $n=8$ 。中点是 $1/8$ 、 $2.5/8$ 、 $5.5/8$ 。
2. $R1$ 与 $R2$ 的中点都在左半边，分两格时编号都是 0；分四格时分别落在编号 0、1，所以第一条边界 $\text{power}=2$ 。 $R2$ 在左半边， $R3$ 在右半边，第二条边界 $\text{power}=1$ 。这里比较的是位置，绝不是拿 8 与 2 比较来算层数。
3. 开始 $C=R1$ 。发现 $R2$ 时，栈为空，将 $(R1,2)$ 压入， $C=R2$ 。发现 $R3$ 时，新 $p=1$ ，栈顶的 2 更大，先弹出 $R1$ 与 $C=[2]$ 合并：取 1、2、8， $C=[1,2,8]$ 。然后压入 $([1,2,8],1)$ ，将 C 改为 $R3$ 。
4. 已经没有输入，弹出最后一项并合并。右段的 0 先出，然后左段的 1、2，再右段的 3、4、5、6，最后左段的 8。得到 $[0,1,2,3,4,5,6,8]$ 。两次合并共处理 $3+8=11$ 个记录；若先合并后两段，则处理 $6+8=14$ 个。

要做多少事 要用多少地方

读入和识别段要 $O(n)$ 。每次合并都要走过参与的记录，因此主要成本是所有合并长度之和。Powersort 将这份成本安排得接近最佳的相邻段合并计划，典型与最坏 $O(n \log n)$ ，一个段时最好 $\Theta(n)$ 。更细的上界为 $O(n(1+H))$ ，其中 n 是记录数， L_i 是第 i 段长度， r 是段数， $H=\sum(L_i/n)\log_2(n/L_i)$ 。这只是按各段长度给出的层数尺度：一段时 $H=0$ ， r 段等长时 $H=\log_2 r$ ，总有 $H \leq \log_2 r$ 。常规缓冲区 $O(n)$ ，调度栈 $O(\log n)$ 。稳定。接近最佳仍允许额外 $O(n)$ 合并成本，并非每个例子都绝对最优。

为什么快或慢

它让小范围合并先做，再逐渐做大范围合并，避免只因扫描先后而把某个长段反复卷入合并。长段在尺子上占更大位置，这个长度信息自然进入层数安排。

什么时候适合用

适合已经掌握普通合并之后，研究不同长度有序段如何高效合并。它可与 TimSort 的补长和疾驰机制组合；初学者先懂普通归并，再查此条。

最后把容易弄错的地方说清楚

本例固定只认不下降段。如果正式实现还识别并反转严格下降段，那么 $[2,0]$ 会成为另一种段，例子的段边界和 power 也要重算。相等值不得因反转而交换。多路 Powersort 把二格划分推广为多格，并做多路合并；那是另一种调度变体。

参考来源 [27] [28] [29]

16 Peeksort

类型 重要变体 别名或相关名称 探看中点附近自然段的归并排序

先把要用的小动作讲清楚

递归就是把同一个任务交给更小的一块，等小块处理完再组合结果。必须说明何时不用再拆，否则会无休止调用自己。

有序前缀是当前区域最左端已知有序的一段，有序后缀是最右端已知有序的一段。这里只记录它们的边界，避免下次重复扫描已经确认的记录。

一次稳定合并使用两个当前位置，反复输出较小值，相等时先取左边；只需合并相邻区域，不能跨过中间未参与记录。

基本思路

普通自顶向下归并总在中间切一刀，可能把本来很长的有序段切碎。Peeksort 先向中间看一眼，沿两边找出中间所在的完整有序段，再把刀移到它较近的一侧边界。已经排好的一段就保留下来。它从整个问题往小问题拆；Powersort 则从左往右收集段并用栈决定合并，两者是不同的安排办法。

具体执行步骤

1. 处理一个区域 $[l, r]$ ，编号包括两端。若空、只有一个数，或已经知道一条有序段覆盖整个区域，直接返回。调用时同时携带左边已知有序前缀的末尾 e 、右边已知有序后缀的开头 s ；初始可只认最左和最右的单元素段。
2. 取中间编号 $m = \text{floor}((l+r)/2)$ 。若 m 已落在已知前缀里，不必重新找它的段：保留 $[l, e]$ ，只递归处理 $[e+1, r]$ ，再合并。若 m 落在已知后缀里，则只处理 $[l, s-1]$ ，再与 $[s, r]$ 合并。边界保证这次切分没有拆开已知的整段。
3. 否则 m 在中间未知的部分。向左检查邻居，只要左值 $\leq$ 右值就扩展；到已知前缀边上时，只比较它的末项与新段首项：若能接上，就把新段起点直接改成 l ，无需再走过整个前缀。向右同理，到已知后缀边上只检查接缝，能接上就直接把终点改成 r 。遇到下降则停止。这样找到包含 m 的完整不下降段 $[i, j]$ ，也跳过了已经确认的内部记录。
4. 比较 $m-i$ 与 $j-m$ 。若左边距离较短，即 $m-i < j-m$ ，就在 i 前切，得到 $[l, i-1]$ 和 $[i, r]$ ；否则在 j 后切，得到 $[l, j]$ 和 $[j+1, r]$ 。若这一段接触区域外端，只用另一侧的内部边界；若覆盖整个区域，直接返回。不得切出同样大的问题再次调用。
5. 递归处理左右两块。把新发现的整段边界交给拥有它的那一块：切在 i 前时，它成为右块的已知前缀；切在 j 后时，它成为左块的已知后缀。其余已知边界随各自子区域传下去。于是后续调用可以跳过重复扫描。
6. 两块分别排好后，用稳定合并将它们写回原区域。到最外层合并结束时，整个数组有序。实现可设一个共用缓冲区，递归只改变处理边界，不必每层复制完整输入。

跟着数字一步步做

1. 输入 $[1, 8, 2, 0, 3, 4, 5, 6]$ ，区域编号 0 到 7，只识别不下降段。中间 $m=3$ ，值为 0。向左遇到 $2 > 0$ ，立即停；向右 $0 \leq 3 \leq 4 \leq 5 \leq 6$ 一直延伸，找到 $[i, j] = [3, 7]$ ，内容 $[0, 3, 4, 5, 6]$ 。
2. 中间到左端距离为 0，到右端为 4，所以在第 3 格前切。左块 $[1, 8, 2]$ 尚需排序；右块完整地是一条已知段，不必再拆。把右块的已知前缀末尾设为 7，下次调用立刻返回。
3. 左块区域 0 到 2，中间编号为 1，值为 8。它的完整不下降段是 $[1, 8]$ ，编号 0 到 1，接触左端，因此在第 1 格后切。左块 $[1, 8]$ 已知有序，右块 $[2]$ 是单元素。合并时取 1、2、8，得到 $[1, 2, 8]$ 。
4. 返回最外层，将 $[1, 2, 8]$ 与 $[0, 3, 4, 5, 6]$ 合并：先 0，再 1、2，再 3、4、5、6，最后 8。得到 $[0, 1, 2, 3, 4, 5, 6, 8]$ 。长度为五的右段只参加必要的大合并，没有被拆成五个单元素问题。

要做多少事 要用多少地方

寻找段要检查邻居，合并要走过参与记录。携带已知前缀和后缀避免反复扫描，靠近中间的段边界控制合并层数。一个现成段最好 $O(n)$ ，典型无结构输入和最坏 $O(n \log n)$ 。更细的界是 $O(n(1+H))$ ， n 为记录数， L_i 为初始段长度， $H = \sum (L_i/n) \log_2(n/L_i)$ 。一段时 $H=0$ ， r 段等长时 $H = \log_2 r$ ， $H \leq \log_2 r$ 。相邻段合并成本接近最佳，允许额外 $O(n)$ 。普通数组合并缓冲区 $O(n)$ ，标准调度需 $O(\log n)$ 递归记录。稳定。

为什么快或慢

它把切口移到已有顺序的边缘，避免先破坏长段再花钱恢复。已知边界还防止找段时一遍遍走过同一条长段。

什么时候适合用

适合在普通自顶向下归并之后学习如何利用现成段。若还不熟悉递归，可先看自然归并的两两合并，再回到此条的切块规则。

最后把容易弄错的地方说清楚

本条和例子固定只扫描最大不下降段。支持反转严格下降段的实现会将 $[2,0]$ 识别为另一段，例子边界也随之变化，不能混用。中间位置用来找段，不一定成为实际切口。所有子区域必须比原区域小；完整段、空区域、单元元素是停止条件。

参考来源 [30] [28]

17 插入排序

类型 算法 别名或相关名称 直接插入排序；线性插入排序

先把要用的小动作讲清楚

把数组想成一排有编号的小格子。本章格号从 0 开始：第 0 格是从左数第一格，第 1 格是从左数第二格，依次类推。格号是位置，不是里面的数值。移动一个数，是把它写入另一格；交换两个数，必须先暂存其中一个，否则第一次写入就会盖掉原值。

这里的已排部分只是内部顺序正确。后来出现的小数仍可能挤到它前面，因此它还不是最终位置。

基本思路

你已经把手中的几张牌从小到大摆好了，又拿到一张新牌。最直接的办法是找出它该站的位置，把挡路的牌往右挪一格，再把新牌放进去。插入排序就重复这个动作：左边始终保持有序，右边每次交出一个数。它不需要先知道全体数据的范围，也不需要额外建一整排格子。

具体执行步骤

1. 只有一个数时，它自己就是有序的，所以先把第一格当作已排部分。从第二格开始处理；把当前数 x 暂存在手里。它原来的格子现在可视为一个空位，虽然实际数组里可能还留着旧数。
2. 看空位左边的数。如果它比 x 大，它站在 x 左边就错了，便把它复制到空位，空位向左走一格。继续做同样的比较。必须从右往左移动，才能先保住尚未搬走的数。
3. 遇到一个不大于 x 的数，就不再往左走：它和左边那些数都可以留在 x 前面。如果已经走到数组开头，也停止。把手里的 x 写进当前空位。
4. 现在左边又多了一个数，而且仍然有序。转向右边下一个数，重复取出、让位、放入。处理完最后一格，已排部分就覆盖了整个数组。为了保留相同值的原来顺序，只移动严格大于 x 的数，不能把等于 x 的也挤过去。

跟着数字一步步做

1. 输入 $[4,2,3,1]$ 。先认定 $[4]$ 有序，取出 2。4 比 2 大，于是把 4 从第 0 格搬到第 1 格，形成 $[\text{空位},4,3,1]$ 。空位已到开头，把 2 放进去，得到 $[2,4,3,1]$ 。
2. 接着取出第 2 格的 3。4 比 3 大，右移为 $[2,\text{空位},4,1]$ 。空位左边是 2，它不大于 3，停止移动，填入 3： $[2,3,4,1]$ 。这里不是交换一次就结束，而是在为新数腾出位置。
3. 最后取出 1。依次把 4、3、2 往右搬，空位经过第 2、1、0 格。填入 1，得到 $[1,2,3,4]$ 。每次完成一次插入，已排部分恰好多一个数。
4. 若输入中有 2A 和 2B，A 原来在前。插入 2B 时遇到 2A 就停止，因为两者相等；2B 放在 2A 后面，所以不会打乱相同值的先后。

步骤状态图 C 比较 M 移动 S 已有序

取出 2，和前面的 4 比较



2 插入后，前两项内部有序



3 插入：4 右移一格



1 插入到开头，全体才最终有序



要做多少事 要用多少地方

主要花费来自向左比较和向右搬动。已经有序时，每个新数只看一次左邻居，几乎不搬，最好 $\Theta(n)$ 。反序时，第 2 个数搬 1 个、第 3 个数搬 2 个，直到搬 $n-1$ 个，总量约为 $n^2/2$ ，所以最坏 $\Theta(n^2)$ ；随机排列通常也要搬动平方数量级的数。更准确地说是 $\Theta(n+l)$ ： n 是记录数， l 是逆序对数，即原来排在前面却比后面大的两条记录组成的对数，每搬一次恰好消除一对。额外空间 $\Theta(1)$ ，只需暂存值和几个位置编号。上述严格大于才移动的版本稳定。

为什么快或慢

它快不快，取决于每张新牌要挤过多少张旧牌。接近有序时，很多新数已经在合适位置，比较一下就能继续；反序时，每个新数都得走到最左边，前面的数一遍遍搬家。

什么时候适合用

适合数量不多、基本有序，或者边收到记录边维持顺序的场景。很多大型排序会让它收拾短小片段，因为简单的循环在这种规模下很省事。

最后把容易弄错的地方说清楚

空数组和单元数组直接结束。可以把是否已排的边界记成一个编号，不必复制左半边。数组中插入意味着实际搬动；即使提前找到位置，这些搬动也不会凭空消失。链表可通过改连接来插入，但寻找位置仍要花时间。

参考来源 [31] [32]

18 二分插入排序

类型 重要变体 别名或相关名称 折半插入排序；用二分查找选插入位置

先把要用的小动作讲清楚

二分查找不是逐格寻找，而是利用已经有序这一点：看中间的数，一次排除肯定不可能的半边。只有待查部分有序时，这个推理才成立。

下文用从 0 开始的编号。区间 $[lo, hi)$ 包含 lo ，不包含 hi ；当两个边界相等，区间里没有待检查的格子。

基本思路

插入排序搬东西很容易理解，但找位置时可能一格一格向左看很久。既然左半边已经有序，就可以先问中间的数：新数应该在它左边还是右边？不断把寻找范围缩成一半，就是二分插入排序。它改进的是找位置的比较次数；为了插入，新数后面的记录仍要搬开。

具体执行步骤

1. 按普通插入排序的办法，把左边保持有序，暂存下一个数 x 。如果左边有 i 个数，就设 $lo=0$ 、 $hi=i$ ，搜索的只是这 i 个数，不包含 x 的旧位置。
2. 取 $mid = \text{floor}((lo+hi)/2)$ ，也就是两个边界的平均值向下取整。若 x 小于第 mid 格的值，插入点不能在它后面，将 hi 改成 mid 。若 x 大于或等于它，则将 lo 改成 $mid+1$ 。把相等情况送到右边，才能把新来的相同值放在旧值后面。
3. 只要 $lo < hi$ ，就继续检查新的中间位置。每轮至少去掉一个格子；最终 $lo=hi$ ，这个编号就是插入位置。它可能是 0，也可能是左半边末尾的后一格。
4. 从左半边最右格开始，倒着把插入位置及其右边的数各搬一格。再把 x 写到插入位置，扩大已排边界，处理下一个数。可以先比较 x 和左半边最后的值：若 x 已经不小于它，直接留在末尾，省掉搜索。

跟着数字一步步做

1. 完整输入 $[1, 3, 5, 7, 4]$ 。前四个数依次加入时都可以直接接在末尾。处理最后的 4 时，左边是 $[1, 3, 5, 7]$ ，设 $lo=0$ 、 $hi=4$ 。
2. 第一轮中间编号为 2，那里是 5。4 应在 5 前面，于是 $hi=2$ ，剩余范围 $[0, 2)$ 。第二轮中间编号为 1，那里是 3；4 应在它后面，于是 $lo=2$ 。现在 lo 和 hi 都是 2，搜索结束。
3. 找到第 2 格不等于已经插进去了。先把 7 搬到第 4 格，再把 5 搬到第 3 格，形成 $[1, 3, \text{空位}, 5, 7]$ ，最后把 4 写入空位，得到 $[1, 3, 4, 5, 7]$ 。
4. 若待插入的是另一个 3，遇到已有的 3 时继续搜右半边，最后插在所有旧 3 之后。不同记录不会因为值相同而丢失。

要做多少事 要用多少地方

每次搜索把范围减半，长度 i 的左半边只需约 $\log_2 i$ 次比较； n 次插入通常共 $\Theta(n \log n)$ 次比较。但每次可能还要搬 i 个记录，典型和最坏总搬动量 $\Theta(n^2)$ ，因此整个数组算法的典型与最坏时间仍为 $\Theta(n^2)$ 。若不先检查末尾，即使原来有序也做 $\Theta(n \log n)$ 次比较；加上末尾检查，最好可为 $\Theta(n)$ 。 n 为记录数。额外空间 $\Theta(1)$ 。按相等向右的规则稳定。

为什么快或慢

比较昂贵时，少问很多次很有价值，例如比较长字符串。但数组里的空位必须靠搬动腾出来。二分不能让夹在中间的 4 穿过 5 和 7，所以它没有消除平方级移动。

什么时候适合用

适合短片段、比较比搬动更贵的记录，以及需要稳定插入的混合排序。TimSort 常用它扩展较短的有序段。

最后把容易弄错的地方说清楚

始终用同一种边界约定，尤其不要把 h_i 当成包含在内的格子。向后搬动可以避免覆盖尚未保存的记录。若使用普通有序数组，不应仅凭比较次数把总时间写成 $O(n \log n)$ 。

参考来源 [33] [24]

19 图书馆排序

类型 重要变体 别名或相关名称 留空插入排序；gapped insertion sort

先把要用的小动作讲清楚

空位不代表某个特别大的数或小的数，而是格子附带的空/非空标记。读取时只看非空格，按它们从左到右的顺序判断是否有序。

随机打乱到达顺序是把记录以随机次序逐个插入；它不改变最终应按什么值排序。概率保证是在这种随机到达和固定预留比例等条件下成立。

支撑位置是一次重新铺开时已有记录所占的格子编号。在下一轮里，这些编号一直有记录，虽然该处的记录可能因搬动而改变。

基本思路

一排书挤得满满的，插一本书常常要搬半个书架。若在书与书之间留些空格，新书通常只需把附近几本推到一个空格。图书馆排序把这种办法用于数组：非空记录始终从小到大，定期把它们重新分散，让空位回到各处。不过，留下空位本身并不保证快；原论文还让记录随机到达，并按安排好的轮次重铺。

具体执行步骤

1. 先知道最终记录数 n ，选择一个固定备用比例 $\epsilon > 0$ ，预留约 $(1+\epsilon)n$ 个格子。随机排列到达次序。按轮次工作：已有 m 个记录时，本轮最多再插 m 个，人数大致翻倍；最后一轮可提前在 n 个处结束。
2. 每轮开始，把已有 m 个记录按顺序均匀铺在能容纳本轮目标数量的活动区域里。记下这些已有记录的格子编号，称为支撑位置。二分查找这些位置上的当前值，找到新记录 x 夹在哪两个支撑值之间；再在两位置之间从左到右检查非空记录，找出准确的前驱和后继。两端没有支撑值时，使用活动区域的边界。
3. 若前驱和后继之间有空格，把 x 放进去即可。否则搜索附近的空格。空格在右边时，从它左边开始向右搬连续记录，直到在后继原位腾出格子，再写 x ；空格在左边时，向左搬挡路的连续记录，最后把 x 写在前驱后面。每次都保留所有非空记录的顺序。
4. 这些搬动不会使原来占用的支撑位置变空，因此本轮还能继续二分查它们的当前值。达到本轮的目标数量后，扩大活动范围并均匀重铺，重新记录支撑位置。扩大的只是预先保留数组的使用范围，不是让定长数组无条件长大。
5. 全部插入后，从左到右把非空记录抄到连续位置，得到普通有序数组。若要稳定输出，比较键必须同时包含原来的编号；否则最初的随机打乱会改变相同值的顺序。

跟着数字一步步做

1. 让记录按 [4,1,7,3,2] 到达。4 先进入空架。插 1 时找到 4 左边的位置，让出一格后得到有序记录 [1,4]；7 放在它们右边，记录顺序变成 [1,4,7]。书架上的空格不是记录。
2. 为了看清插入动作，画出这三个记录的一次均匀铺开状态 [1,空,4,空,7,空]，支撑编号为 0、2、4。此图展示机制，不声称正式轮次恰在插完三个时重铺。3 位于支撑值 1 和 4 之间，第 1 格为空，直接放入：[1,3,4,空,7,空]。
3. 接着插 2。支撑编号上的当前值仍是 1、4、7，所以二分把范围缩在 1 和 4 之间；顺序检查发现它应该在 1 与 3 之间。这里没有空格，右侧第 3 格有空位。先把第 2 格的 4 搬到第 3 格，再把第 1 格的 3 搬到第 2 格，最后写入 2：[1,2,3,4,7,空]。
4. 若下一次安排重铺，可以变成 [1,空,2,空,3,空,4,空,7,空]；最后去掉空位就是 [1,2,3,4,7]。为五个记录保留十格可示意 $\epsilon=1$ 。实际轮次按数量近似翻倍结束，所需容量从开始就预留好。

要做多少事 要用多少地方

查找支撑值要约 $\log n$ 次比较，重铺时每个记录只搬一次。关键问题是两次重铺之间要找多远的空位、搬多长的连续段。在随机到达、固定 ε 和论文轮次规则下，这些局部拥挤受到概率控制，期望总时间 $O(n \log n)$ ，并且以高概率也是 $O(n \log n)$ ；高概率指失败机会随规模增长而变小，不表示绝不失败。逆着这些条件安排到达次序，或遇到罕见的坏随机结果，仍可有 $\Theta(n^2)$ 工作。普通每次二分的版本最好一般也用 $O(n \log n)$ 工作，额外空间 $\Theta(n)$ 。 n 为记录数。基本随机版不稳定；加入原编号作为相等值的第二比较键可稳定。

为什么快或慢

它把整排大搬迁改成了走到附近空位的小搬迁。随机到达有助于避免总向同一小段挤书；定期重铺把已经被吃掉的空隙重新分散。二分搜索仍保留对数成本。

什么时候适合用

适合研究带空隙的有序数组和概率型数据结构。当随机化、备用空间、重铺和空位管理的复杂度值得承担时可以考虑；普通小数组往往用插入排序更简单。

最后把容易弄错的地方说清楚

不能把空位作为普通值参与二分，也不能只写有空就插而省略定位方法。支撑位置二分之后仍需检查局部区间。备用比例必须固定且大于零；所有数组范围和最后不足一轮的情况都要受预留容量约束。变体的额外索引、轮次或空间策略需要自己的分析，不能自动继承论文结论。

参考来源 [34]

20 耐心排序

类型 算法 别名或相关名称 纸牌堆排序；与最长上升子序列算法有关

先把要用的小动作讲清楚

一摞牌只能从顶上取。我们会保证每摞从顶到底不下降，所以摞顶就是这摞尚未输出的最小值。

优先队列是一组候选记录，支持放入和取出最小项，不要求里面从左到右全部有序。二叉最小堆是一种实现：每个父位置的值不大于两个子位置，因而堆顶一定最小。

二分找摞不是在摞内找，而是在已经从左到右排好的摞顶列表里找第一个不小于新值的摞顶。

基本思路

像发牌一样把数放到几摞中。新数只要不大于某摞顶部，就能盖在上面；优先选最左边能放的摞，否则在右边开一摞。这样每摞自上而下有序，摞顶从左到右也有序。但这还没有排完：接下来必须在所有摞顶中不断选最小值，取走后露出同摞的下一张，直到所有牌都输出。

具体执行步骤

1. 开始没有摞。读入 x ，在摞顶列表中二分寻找第一个顶值 $\geq x$ 的位置：看中间，若顶值 $\geq x$ 就保留含它的左半边，否则只保留右半边，直到确定第一个位置。找到便把 x 压在该摞顶上；找不到就在最右边新建一摞。记录整摞的所有牌，不只留顶部。
2. 为什么摞顶仍有序？被选摞左边的顶值都小于 x ，否则会更早被选；被选摞右边的顶值不小于该摞原顶，更不小于 x 。因此换上 x 后，左右关系仍正确。该摞原顶 $\geq x$ ，所以摞内从顶到底也仍不下降。
3. 发完牌，将每摞顶部作为一个候选项放入最小优先队列，每项还记着属于哪一摞。下面可用二叉堆数组：第 i 格的父编号为 $\text{floor}((i-1)/2)$ ，两个孩子为 $2i+1$ 、 $2i+2$ ，编号均从 0 开始。不存在的孩子不参与比较。
4. 往堆里放候选项时，先加到末尾；若它小于父项，就交换并继续向上，直到父项更小或已到第 0 格。取最小时，拿走第 0 格，将最后一项搬到第 0 格；反复与较小孩子比较，若孩子更小就交换向下，直到关系恢复。这样无需每次重新排列整队候选。
5. 每次从队列取出最小摞顶，把那张牌加入最终输出，并从它原来的摞取走。若该摞还有牌，将新露出的摞顶入队；否则不再给这摞放候选。队列为空时全部结束。

跟着数字一步步做

1. 输入 [4,2,3,1]。4 建第一摞。2 找到摞顶 4，把自己盖在它上面，摞一从顶到底 [2,4]。3 比目前唯一摞顶 2 大，没有可放摞，于是在右边建摞二 [3]。1 找到最左的摞顶 2，盖上。最终摞一 [1,2,4]，摞二 [3]，顶值列表 [1,3]。
2. 候选队列开始只有 (1,摞一)、(3,摞二)。取 1，输出 [1]；摞一露出 2，入队。现在候选为 2、3，取 2，输出 [1,2]；同摞露出 4，入队。
3. 现在队列最小是 3，不是刚露出的 4。取 3，输出 [1,2,3]，摞二已空。最后取 4，输出 [1,2,3,4]，队列也空。若先整摞倒完摞一，再倒摞二，会得到错误的 [1,2,4,3]。
4. 相等值若按基本 $\geq$ 规则盖在同一摞，会以新牌先出的顺序倒置。需要稳定时改用第一个顶值 $>x$ 的摞，并在队列相等时优先原来编号较小的记录。

要做多少事 要用多少地方

每个记录先做一次摞顶二分，之后一次出队，可能还引出下一项入队。最多 p 个摞顶时，二分和堆操作各需约 $\log(p+1)$ 层，总上界 $O(n \log(p+1))$ 。 n 为记录数， p 为摞数。严格反序只有一摞，最好 $\Theta(n)$ ；随机互异值通常以及最坏为 $\Theta(n \log n)$ 。严格上升会形成 n 摞。所有摞、候选和输出通常需 $\Theta(n)$ 额外空间。基本 $\geq$ 版不稳定；上述严格大于加原编号的变体可稳定。

为什么快或慢

每摞把一些记录藏在已经更小的顶牌下面，暂时不必参与全体比较。摞少，候选就少，每次选择更省。若几乎每条记录都自己成一摞，就没有这方面的省力。

什么时候适合用

适合学习把多条有序序列合并、优先队列和输入顺序结构。摞的数量也与最长严格上升子序列有关，但求该长度和输出完整排序是两项不同任务。

最后把容易弄错的地方说清楚

不能只保存摞顶，否则下面的记录会丢失。摞顶列表有序不等于全体数据已排。空输入直接输出空列；每项必须保留摞编号，才能知道取走它以后该露出哪张牌。

参考来源 [35] [36] [37]

21 平滑排序

类型 重要变体 别名或相关名称 Dijkstra 的自适应原地堆排序

先把要用的小动作讲清楚

树可以想成有上下级的记录：一个父节点连向孩子。最大堆只要求父值不小于孩子；因此根最大，但同层或整排还未完全排好。

森林表示并排放着几棵树。这里每棵树实际上占数组里的一个连续块，根放在块的最后一格，不另造一套记录副本。

Leonardo 数定义这些块的大小： $L(0)=L(1)=1$ ， $L(k)=L(k-1)+L(k-2)+1$ ，依次为 1、1、3、5、9、15……。一个大块等于左子块、右子块，再加一格根，公式正是这样来的；两个大小为 1 的阶数仍分别记为 0 和 1。

基本思路

普通堆排序把当前最大值送到右端，逐个确定最后的位置。Smoothsort 也这样做，但它不用固定的一棵堆，而是把未完成的左边分成几棵特殊大小的堆。每棵堆根最大，再让这些根从左到右不下降，于是最右根就是整个未完成区域的最大值，可以直接留在右端。已接近这种关系的数据不需要许多交换，这就是它能利用已有顺序的关键。下面给出每一步都完整修复的清楚版本；标准优化版会省掉部分注定被后续合并替代的修复。

具体执行步骤

1. 从左到右把记录加入未完成前缀，记每块的阶数。第一条是阶 1 的单格堆。若最后两堆的阶数为 k 、 $k-1$ ，就把它与新记录合成阶 $k+1$ 的堆，新记录在新块末尾当根。否则追加单格堆：前一末堆是阶 1 时，新单格记阶 0；其余情况记阶 1。这些规则能表示任意前缀长度，不需要大小为 2 的堆。
2. 先学堆内修复 sift。阶 $k \geq 2$ 的块，由左阶 $k-1$ 、右阶 $k-2$ 、末格根组成。根编号为 t 时，右子根是 $t-1$ ，左子根是 $t-1-L(k-2)$ 。比较两个孩子，选较大者；若它比根大，就交换，并在那个子堆重复。若根已经不小于孩子，或走到单格堆，就停。每次向下走到更小的块。
3. 还要修复堆与堆之间的根顺序，这个动作叫 trinkle。当前根编号 t 、当前堆阶数 k 时，前一堆根就在 $t-L(k)$ ，不必扫描森林寻找。仅当这个前根大于当前根，而且也大于当前根的两个孩子（若有）时，交换两个根，将当前位置转到前一堆，继续向左检查。大根被移到右堆时仍不小于那里的孩子，因此右堆保持堆关系。
4. 若前根没有同时满足以上比较，就不再向左交换，在当前堆执行 sift；没有前堆时也执行 sift。这修好刚被换小的根所在的内部关系。每加入一个记录，对新出现的最右堆做完整 trinkle，便保住所有堆内关系和从左到右的根顺序。
5. 建完前缀后，最右根即全体最大，留在原格并把这一格划入完成后缀。若末堆只有一格，删除它的形状记录。若阶数 $k \geq 2$ ，拿走根后，原块自然分成阶 $k-1$ 的左堆和阶 $k-2$ 的右堆；它们的内部关系已有，只是新露出的根可能与前堆顺序不对。先对左子堆做 trinkle，再对右子堆做 trinkle。
6. 重复取出最右根、删除或拆堆、修复。每次未完成区域少一格，完成后缀多一格。交换和比较只能在未完成前缀中进行，不再动右边已确定的位置。空前缀时结束。

跟着数字一步步做

1. 输入 [3,4,1,5,2]。先加 3、4，得到阶数 [1,0] 的两个单格堆，根 $3 \leq 4$ ，无需交换。加入 1 时，末尾两阶可与它合成阶 2、大小 3 的堆 [3,4,1]。根 1 比孩子 4 小，交换，数组变成 [3,1,4,5,2]，前三格的根为 4。
2. 加入 5 为单格堆，阶数 [2,1]，根 $4 \leq 5$ 。加入 2 后合成阶 3、大小 5 的堆，孩子根是 4 和 5，根为 2。与更大的孩子 5 交换，得到 [3,1,4,2,5]。现在 5 在最右根，确认它最终排第五。
3. 拿走根 5 的形状后，留下大小 3 的 [3,1,4] 和单格 [2]。左根 4 比右根 2 大，于是 trinkle 交换这两个根，先成为 [3,1,2,4,5]；被换小的左根 2 又小于自己的孩子 3，sift 把 3 提上来，得到 [2,1,3,4,5]。右端 4 现在也可以确定。
4. 删除单格 4，再确定大小 3 堆的根 3。它拆成单格 2、1，两个根顺序相反，交换为 [1,2,3,4,5]。继续确认 2、1 即结束。过程中的 [3,1,4] 是堆，不是已排序段；不能直接把它作为最终输出。

要做多少事 要用多少地方

加入或拆堆时，修复沿树向下或沿森林根向左走，最长约 $\log n$ 层，因此无明显结构的典型及最坏总时间 $O(n \log n)$ 。已上升的输入满足所需关系，很多修复立即停止，加上形状更新的摊销成本，最好 $\Theta(n)$ 。 n 为记录数。标准实现用能表示 n 的常数个机器字和位编码保存森林形状，额外空间 $O(1)$ ；本条若显式存阶数列表，则需 $O(\log n)$ 空间。通常不稳定：非相邻交换可改变相同值的先后。

为什么快或慢

它保留已经符合堆内和根间关系的部分，不必每次无条件重新找最大值。但数据杂乱时仍要上下、左右修复，回到对数层级的成本。这里的适应性不是声称所有接近有序的衡量方式都带来线性时间。

什么时候适合用

适合进阶理解利用已有顺序的原地堆排序。需要先掌握父子关系、普通堆内下沉，再看森林根之间的 trinkle；第一次学排序可暂时跳过这一条。

最后把容易弄错的地方说清楚

完整修复版用于讲清不变关系，优化实现可能推迟一些建堆交换。Leonardo 的两个单格阶必须区分，不能随意合成大小 2。不要每次扫描所有堆根找最大，否则会失去线性最好情形。子根位置由连续布局计算，不能套普通二叉堆数组的 $2i+1$ 公式。

参考来源 [38] [39]

22 线股排序

类型 算法 别名或相关名称 抽取有序子序列再归并；不是连续自然段归并

先把要用的小动作讲清楚

子序列是按原先顺序挑出的若干记录，中间允许跳过一些。例如 [4,1,3,2,5] 中挑 4、5 可以组成子序列，它们不必相邻。

合并两个有序队列时，只需比较各自最前面的数，拿走较小的那个。因为同一队列后面的数都不更小，没必要现在逐个看它们。

基本思路

从乱序记录里挑出一条能从小到大接下去的链，把暂时接不上的数留着，下一轮再处理。挑出的链已经有序，可以和前面做好的结果合并。这就是链式排序。它找的是可以跳过记录的有序子序列；自然归并排序找的是连续相邻的有序段，两者不要混为一谈。

具体执行步骤

1. 准备一个空的有序结果。只要还有未处理记录，就取走其中最前面的一条，作为本轮链的开头。取走意味着以后不能再重复处理它。
2. 从左到右查看余下记录。如果当前值不小于链尾，把它取走并接到链尾；否则把它留在余下队列中。跳过的记录仍保持原有相对顺序。每次接入都保证链尾没有下降，所以整条链有序。
3. 将这条链与现有结果合并。各用一个位置指向尚未使用的最前记录，比较两个当前值，输出较小者，然后只推进它的指针。一边用完，把另一边剩余记录依次接上。相等时优先选择较早建立的结果，保留旧记录先于本轮新记录的顺序。
4. 把本轮剩下的队列作为下一轮输入，直到它为空。使用数组时，可以在扫描中另建一个余下队列，不要每取走一个值就移动整条数组尾巴；链表则可改动连接而不搬动后续记录。

跟着数字一步步做

1. 输入 [4,1,3,2,5]，结果为空。取 4 开头。1、3、2 都比链尾 4 小，先留下；5 能接在 4 后面。本轮得到链 [4,5] 和余下 [1,3,2]，结果先变成 [4,5]。
2. 第二轮从 1 开始。3 不小于 1，接上；2 小于当前链尾 3，留下。新链 [1,3] 与旧结果 [4,5] 合并：1 和 4 比选 1，3 和 4 比选 3，然后旧结果还有 4、5，依次接上。结果 [1,3,4,5]，余下 [2]。
3. 最后一条链只有 [2]。合并时先比较 1 和 2，取 1；再比较 3 和 2，取 2；新链耗尽，接上 3、4、5。最终 [1,2,3,4,5]。
4. 这里允许相等直接到链尾，因此重复值不会被漏掉。较早一轮有相同值时，合并先取旧结果；再配合每轮不打乱跳过项的顺序，这个版本稳定。

要做多少事 要用多少地方

每轮要扫描余下记录，还要把本轮链与越来越长的结果合并。若一轮就能选走所有记录，只扫描和输出一次，最好 $\Theta(n)$ 。反序时每轮只选走一个，余下扫描长度为 $n-1$ 、 $n-2$ ……，合并结果也反复经过旧记录，最坏 $\Theta(n^2)$ 。典型成本取决于输入能形成怎样的链，这里不假设一个通用平均界。 n 为记录数。通常需 $\Theta(n)$ 输出或余下存储；链表可重用节点。上述保持顺序且旧结果优先的版本稳定。

为什么快或慢

一条长链一次就处理很多记录，轮数少；很多短链让同一批记录反复被看、旧结果反复被合并，所以慢。它不保证每轮把问题减半。

什么时候适合用

适合讲解从已有顺序提取子序列和合并的思想，也可用于本来含有少数长链的链表数据。一般通用数组排序更多选用有明确 $O(n \log n)$ 最坏界的方法。

最后把容易弄错的地方说清楚

传统描述有时只允许严格更大的值接入，也能排序，但重复值会分散到更多轮。明确本书用不小于规则。空输入无需抽链。比较失败的记录必须留下，而不是丢弃；合并时必须推进刚输出的那一边。

参考来源 [40]

23 笛卡尔树排序

类型 算法 别名或相关名称 基于笛卡尔树的自适应堆排序；Levcopoulos-Petersson 方法

先把要用的小动作讲清楚

节点是装一条记录的小盒子，另有左、右连接指向其他盒子；没有孩子时连接为空。整棵树最上面的节点叫根。

中序访问依次走左边所有节点、当前节点、右边所有节点。这里它要保留输入顺序，不是排序输出。另一个要求是每个父值 $\leq$ 孩子，称为最小堆关系。

栈从最后放入的项开始取出。优先队列则每次取最小值。建树使用栈，输出使用优先队列，两个结构用途不同。

基本思路

能否先把输入搭成一棵树，让某个子树里最小的值总在顶部？笛卡尔树同时保持两件事：按左、自己、右访问仍能读出原输入；父节点不大于孩子，所以每棵子树的根都是那一块的最小值。树搭好后还不能按左右顺序输出，因为那样只是得到原输入。真正排序时，先输出根，再把它的孩子当作下一批候选，始终从候选中选最小。

具体执行步骤

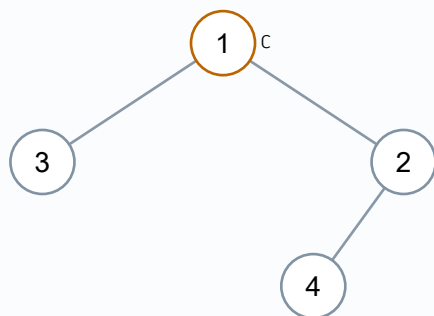
1. 为每条输入记录建立独立节点，左右连接先为空。维护一个值从栈底到顶不下降的栈，它记录当前树最右边的路径。处理新节点 x 时，先把 $last$ 设为空；只要栈顶值比 x 大，弹出栈顶，并用 $last$ 记住这次弹出的节点。最后一次弹出的节点会成为被移走整块的根。
2. 将 x 的左孩子设为 $last$ 。若栈还有顶节点，就把顶节点的右孩子改为 x ；若栈已空， x 就是新的整树根。再把 x 压入栈。注意被替换的右边整块并未丢失：它已经接到 x 的左边。新记录在输入末尾，左边放那些原来已经出现的记录，仍可保持中序访问等于原输入。
3. 为何不必逐个重建旧子树？弹出的节点之间本来就有连接，保留它们，最后弹出的节点带着其整个子树接到 x 下方即可。每节点只压栈一次、最多弹一次。读完输入后，保存的根就是笛卡尔树的根，左右连接给出了完整树。
4. 开始排序输出，优先队列只放根。队列中的候选是父节点已输出而自己未输出的节点；每个候选都代表一个尚未输出的子树，其中没有记录比它更小。候选之外的记录都被某个这样的根挡在下面，因此全体剩余最小值一定在候选中。
5. 从队列取最小候选，将其记录写入结果；再把它存在的左、右孩子加入队列。队列可以用另一个二叉最小堆数组：新项加在末尾，比父项小就交换向上；取根时以末项补根，再与较小孩子交换向下直到父值不大于孩子。此堆用于候选，并不是刚搭好的笛卡尔树。
6. 重复取最小、加入孩子，直到队列为空。每节点只输出一次。为了相同值按原顺序输出，从建树到队列都比较 (数值, 原始编号)：值相等时编号小的更小，不只在某一步临时加规则。

跟着数字一步步做

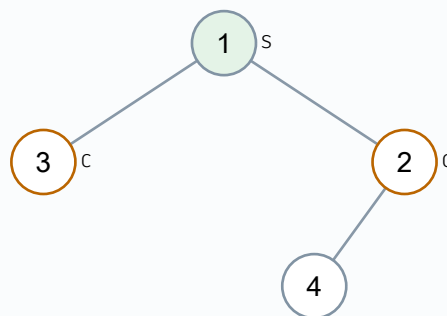
1. 输入 [3,1,4,2]。读 3：没有旧节点，3 成为根，栈 [3]。读 1：弹出 3， $last=3$ ；设 1 的左孩子为 3。栈空，根改为 1，栈 [1]。
2. 读 4：栈顶 1 不大于它，不弹；将 1 的右孩子设为 4，栈 [1,4]。读 2：弹出 4， $last=4$ ；接为 2 的左孩子。留下的栈顶是 1，将 1 的右孩子改为 2，栈 [1,2]。4 没消失，它现在在 2 下方。
3. 最终根 1，左孩子 3，右孩子 2，2 的左孩子 4。按中序访问，左边先读 3，读根 1，再在右边读 4、2，得到原输入 [3,1,4,2]；这说明中序不是排序步骤。
4. 候选开始 [1]。取 1，输出 [1]，加入 3、2。取候选中较小的 2，输出 [1,2]，加入孩子 4，候选现在 3、4。再取 3、4，最终 [1,2,3,4]。左右孩子必须一起进入候选，不能未经比较就先遍历整棵左子树。

笛卡尔树与候选前沿 同一示例 [3,1,4,2]

树构建完成
最初候选仅为根 1



输出 1 后
2 与 3 成为下一候选



要做多少事 要用多少地方

建树时每节点压栈和弹栈至多一次，合计 $\Theta(n)$ 。输出时每节点进、出候选队列各一次，每次最多走 $\log(q+1)$ 层，总上界 $O(n \log(q_{\max}+1))$ ， $q_{\max}$ 是同时出现的最多候选数， n 为记录数。单链树的候选始终只有一个，最好 $\Theta(n)$ ；一般典型和最坏 $O(n \log n)$ ，不能把线性建树误写成线性排序。节点、栈、队列和输出总额外空间 $\Theta(n)$ 。全程以数值和原编号比较可稳定；任意处理相等值则不保证稳定。

为什么快或慢

它一次建立了每个连续输入区域谁先有资格输出的关系，之后只比较已经露面的候选。若树是单链，几乎没有选择成本；若很多分支同时露面，队列仍要进行对数层比较。

什么时候适合用

适合学习输入顺序与堆关系可以同时存在，以及用树中已知的局部最小值组织排序。它也是区间最小值结构的相关基础；初学者先懂节点、栈和优先队列再读此条。

最后把容易弄错的地方说清楚

它不是二叉搜索树：左孩子不要求值更小。每个重复记录都要有独立节点。反复扫描区间找最小值再递归建树，最坏会 $\Theta(n^2)$ ，本条使用栈避免此问题。树可能很不平衡，但候选二叉堆是另一种结构，不依赖这棵树的高度。

参考来源 [41] [42]

三 通用比较排序与条件优化

一般大数组中，先讲保证或总体摊还 $O(n \log n)$ 的层；随后讲枢轴、输入结构等条件下高效但可能退化的方法。Shellsort 的界依增量而变。缓存、比较次数及大记录移动的优化单列于本组后部，度量不同，不按实际秒数强行排序。

24 自顶向下归并排序

类型 算法 别名或相关名称 递归归并排序；Merge sort 的经典实现

先把要用的小动作讲清楚

数组就是一排有编号的格子；本节说“前半、后半”，只是限定看哪些格子，并不丢掉其他数。

已经有序的两排数，各自最左边就是这一排剩下的最小数。只比较这两个头，便能决定下一项该拿谁。

“递归”是对较小的一排重复同样任务。只有一个数时不用做任何事；这就是停止规则。

基本思路

先把大问题拆成小问题：一排分成两半，每半再分两半，直到一项一排。然后反着往回走，每次把两个已排好的小排接成一个更大的有序排。接法不是直接拼起来，而是不断比较两排还没取走的第一个数。

具体执行步骤

1. 指定当前范围。若里面只有零项或一项，立即结束，这个范围已经有序。否则从中间分开，先排左半，再排右半；每半都照这几步做。
2. 两半都完成后，给它们各放一个指针。指针只是记住“下一项在哪”的编号。准备能装下整个范围的临时格子，输出位置从第一个空格开始。
3. 比较左右指针所指的数，把较小的完整记录复制到输出格，那个指针向前一格，输出位置也向前一格。遇到相等，先取左半的记录，原来的先后关系就不会颠倒。
4. 只要两边都有剩余项就继续比较。一边取完后，另一边本来已经有序，按原次序全部复制过去，不必再两两比较。
5. 把临时结果复制回原范围。回到上一层时，这一整排已经有序，可以和另一半用相同方法接起来。

跟着数字一步步做

1. 从 [4,1,3,2] 开始，分成 [4,1] 与 [3,2]；每半再分成单项 [4]、[1]、[3]、[2]。单项直接返回。
2. 接 [4] 和 [1]：比较 4 与 1，先写 1；右边空了，再写左边的 4，得到 [1,4]。接 [3] 与 [2] 同理先写 2，再写 3，得到 [2,3]。原数组现在是 [1,4 | 2,3]。
3. 接最后两排：头是 1、2，取 1，临时输出 [1]；左头变成 4，右头仍是 2。比较 4、2，取 2，输出 [1,2]。
4. 右头前进成 3，比较 4、3，取 3，输出 [1,2,3]。右边已经空了，把左边的 4 接上，成为 [1,2,3,4]，再写回原数组。

步骤状态图 C 比较 M 移动 S 已有序

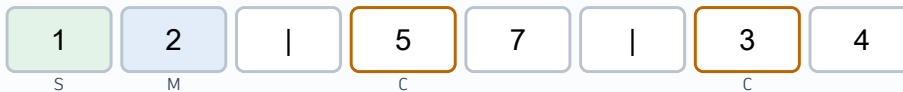
两个已排序段：左 | 右



输出 1 | 左余项 | 右余项



输出 1,2 | 左余项 | 右余项



输出 1,2,3,4 | 左余项



右段耗尽，追加 5,7



要做多少事 要用多少地方

设 n 为记录数。每次接两排，要逐项读写，约做与这一排长度成正比的工作。同一拆分层的排合计仍是 n 项；长度反复减半约有 $\log_2 n$ 层，所以基础版最好、一般、最坏都为 $\Theta(n \log n)$ 。 Θ 表示同一增长量级。临时数组要 $O(n)$ 格，等待返回的调用记录要 $O(\log n)$ ；相等时先取左边则稳定。增加“两半边界已经顺序正确就不接”的检查后，已经有序输入可接近 $\Theta(n)$ 。

为什么快或慢

两排各自有序后，不需要把左边每个数和右边所有数比较。每次只看两个头，读过的项就不回头，因此每层只是扫一遍。代价是需要临时存放和复制记录。

什么时候适合用

需要可预测的最坏速度和稳定性时很合适；稳定表示相等记录仍按输入先后出现。大记录的反复复制和临时数组需要计入实际成本。

最后把容易弄错的地方说清楚

两排内部有序不等于全数组有序， $[1,4 | 2,3]$ 就是反例。只复制记录的键而丢掉附带信息不算正确排序。真正的实现通常只分配一个大临时数组，再反复使用其中相应范围。

参考来源 [23] [43]

25 自底向上归并排序

类型 重要变体 别名或相关名称 迭代归并排序；归并排序变体

先把要用的小动作讲清楚

一项本身就有序；把长度一的两排接起来能得到长度二的有序排，再把长度二的两排接成长度四。

接两排时，两个指针分别记录两排尚未取走的头，另一个位置记录输出写到哪里。

这里用循环，意思是完成一轮后改一下排的长度，再做下一轮，不需要层层调用自己。

基本思路

不从大排向下拆，而从单项向上组装。先两项一组，再四项一组，再八项一组。每次只接已经有序的邻接两排；只要排长覆盖全数组，就完成了。

具体执行步骤

1. 令当前排长 $w=1$ 。沿数组从左到右，拿最多 w 项作左排，再拿接下来的最多 w 项作右排。末尾可能不足规定长度；没有右排就保持左排。
2. 接这两排：分别看下一项，较小的复制到临时输出，移动供给这一项的指针。相等时先取左排。某排取空后，把另一排剩余记录顺序复制过去。
3. 把接好的范围复制回去，再向右走到下一个尚未处理的两排。一次完整扫描结束后，每个长度约 $2w$ 的范围已经有序。
4. 把 w 变成 $2w$ ，重新从左到右扫描。只在 $w < n$ 时开下一轮， n 是数组项数； n 为零或一时根本不用扫描。

跟着数字一步步做

1. $[4,1,3,2,5]$ 的第一轮 $w=1$ ： $[4]$ 与 $[1]$ 比较先写 1 再写 4； $[3]$ 与 $[2]$ 先写 2 再写 3；末尾 $[5]$ 没有配对排，留下。得到 $[1,4 | 2,3 | 5]$ 。
2. 第二轮 $w=2$ ，接 $[1,4]$ 与 $[2,3]$ 。比较 $1/2$ 取 1， $4/2$ 取 2， $4/3$ 取 3，再取 4，得到 $[1,2,3,4 | 5]$ 。末尾 5 继续保留。
3. 第三轮 $w=4$ ，左排 $[1,2,3,4]$ 、右排 $[5]$ ：依次比较 $1/5$ 、 $2/5$ 、 $3/5$ 、 $4/5$ ，取左边四项，最后取 5。得到 $[1,2,3,4,5]$ 。 w 再翻倍后超过 n ，停止。

要做多少事 要用多少地方

设 n 为项数。每轮把每项读写一次，排长每轮翻倍，需要约 $\log_2 n$ 轮；基础版最好、一般、最坏 $\Theta(n \log n)$ 。 Θ 表示同一增长量级。复用临时输出需 $O(n)$ 格，循环编号只需 $O(1)$ 控制空间，无递归栈。相等时左排先输出则稳定；增加跳过已接顺序的边界检查可改善有序输入。

为什么快或慢

与自顶向下归并做的主要接排工作相同，只是通过排长控制顺序。它免去调用栈，但仍要读写临时记录，不会因此自动比所有归并实现快。

什么时候适合用

适合希望使用明确循环、避免调用自己，或者顺序处理连续数据的实现。

最后把容易弄错的地方说清楚

不能假设长度是二的整数次幂。读写范围都必须截到数组末尾；示例中的第五项正是边界情况。两排完成后才允许把它们当作下一轮的有序输入。

参考来源 [44] [45]

26 堆排序

类型 算法 别名或相关名称 二叉堆排序；堆式选择排序

先把要用的小动作讲清楚

把一排数按层看成树：第一个是根，下层两个是它的孩子，再下层是孩子的孩子。不必创建新树，位置关系就存在数组编号里。

位置从 0 开始时，位置 i 的孩子在 $2i+1$ 、 $2i+2$ ，超过当前项数就不存在。根在位置 0。

最大堆只要求父项不小于孩子。顺着父子关系往下，数不会变大，因此根最大；兄弟或相邻格子却不一定有序。

“下沉”是把过小的父项和较大的孩子交换，再在它新到达的地方重复，而不是重排整个树。

floor 表示向下取整，例如 $\text{floor}(5/2)=2$ 。查找最后父项时，半个位置不能使用，要取这个整数。

基本思路

先让数组符合最大堆的父子规则。最大的数就能立即从第一个格子取出，把它放到末尾固定。将堆缩小一项，令换到根的新数沿一条向下路径寻找合适位置，再取下一个最大值。固定的尾部从右往左增长，最终全排升序。

具体执行步骤

1. 先学一次下沉。当前堆有 m 项，某父位置 i 有至少一个孩子时，读它存在的孩子；两个都有就先比孩子，挑较大者 c 。若父值 $\geq$ 孩子 c ，停止：父值也 $\geq$ 另一个孩子。否则交换父与 c ，把 i 改成 c ，继续往下。没有孩子也停止。
2. 建堆从最后一个父位置 $\text{floor}(n/2)-1$ 开始，依次处理更小父编号，直到 0。底层没有孩子，本来符合规则；从下往上做时，孩子下面的分支已经符合规则，所以上述一次下沉足以整理当前分支。
3. 建好后，交换位置 0 与 $m-1$ 。根是剩余最大值，所以它到了全局应在的最右位置，之后不再碰它。令 m 减一，尾部不属于于堆。
4. 若 $m>1$ ，从根执行下沉。这次只有换入根的数可能破坏父子规则，其他分支内部仍然符合；用一条路径处理就够。
5. 反复取根、缩小、下沉。最后一项自然最小，结束。空输入和单项不用建堆。所有孩子编号检查都使用 m ，不要把已经固定的尾部当孩子。

跟着数字一步步做

1. $[4,1,3,2]$ ， $n=4$ ，最后父编号 $\text{floor}(4/2)-1=1$ 。位置 1 的值 1 只有孩子位置 3 的 2，交换成 $[4,2,3,1]$ 。新位置 3 没孩子，停止。
2. 处理根 4：孩子是位置 1 的 2、位置 2 的 3，先挑 3； $4 \geq 3$ ，不换。现在是堆，但 $[4,2,3,1]$ 显然不是升序。
3. 交换根 4 与末项 1，成 $[1,2,3 | 4]$ ， $m=3$ 。比较孩子 2、3，选 3； $1 < 3$ ，交换为 $[3,2,1 | 4]$ 。新位置 2 无孩子。
4. 根 3 与当前末项 1 交换，成 $[1,2 | 3,4]$ ， $m=2$ 。根唯一孩子是 2，交换为 $[2,1 | 3,4]$ 。
5. 最后交换根 2 与 1，得 $[1 | 2,3,4]$ ， $m=1$ ，停止。竖线右边每一项都是移过去时就固定的；从未丢掉一个原数。

步骤状态图 C 比较 M 移动 S 已有序

建最大堆：交换 1 与孩子 2



4 放末端；3 上升修复根



3 放末端；2 上升修复根

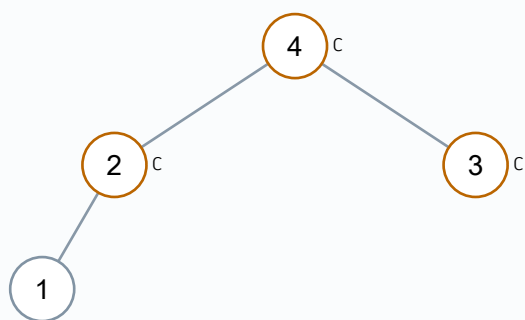


2 与 1 交换，完成

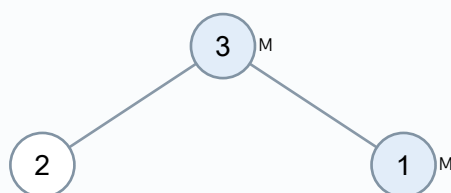


堆的树形关系 节点只是同一数组的另一种画法

建堆后比较根与孩子
数组为 [4, 2, 3, 1]



取出 4 后修复剩余堆
堆 [3, 2, 1] 输出后缀 [4]



要做多少事 要用多少地方

设 n 为项数。建堆从下到上：很多父项离叶子很近，只需一两次比较，少数靠上项才走较长路径，合计 $\Theta(n)$ ，不是每项都走到底。取最大值后下沉最多走约 $\log_2 n$ 层，取出约 n 次，所以通常、最坏 $\Theta(n \log n)$ ；不同键时最好也 $\Theta(n \log n)$ 。本下沉遇相等会停，全等可能 $\Theta(n)$ 。迭代只用少量编号和交换临时值，额外 $O(1)$ 空间。不稳定，因为根与尾的远距离交换可改变等值记录先后。

为什么快或慢

选择下一最大值时只看一条树路径，不用重新扫描全部剩余数。但父孩子格子不都相邻，跳跃访问可能比连续归并更不利于缓存。

什么时候适合用

需要最坏速度保证又不愿分配大数组时很好；也适合给快排混合算法作坏情况后备。

最后把容易弄错的地方说清楚

“自底向上建堆”别与“bottom-up heapsort”变体混淆：后者可先让大孩子逐层上移到空位，再把拿住的旧根插回路径，以省比较。多叉堆让每父有更多孩子，树更矮却要比较多孩子。

参考来源 [46] [47]

27 弱堆排序

类型 算法 别名或相关名称 Weak-heap sort ; 经典 n 个反向位实现

先把要用的小动作讲清楚

这里的树编号和普通二叉堆不同：根 0 只有孩子 1；位置 $i \geq 1$ 的两个物理孩子在 $2i$ 、 $2i+1$ 。物理父编号是把 j 除以 2 向下取整。

每位置多存一个反向位 $r[i]$ ，只有 0、1 两数值。位为 0 时， $2i$ 叫逻辑左孩子、 $2i+1$ 叫逻辑右孩子；位为 1 则两种名字交换。树格子没搬，改变的是孩子角色。根 $r[0]$ 永远为 0，根没有左孩子，不能把公式的 2×0 当孩子。

子树是从某孩子继续往下能到达的所有位置。最大弱堆要求某节点不小于其逻辑右子树全部值，却不要求大于左子树全部值。根的唯一右子树包含其余全部位置，所以根仍最大。

这里的 join 是一项特定操作：比较两个指定位置，必要时换值并翻一个位。它不是把任意两数比较后就能建立整个弱堆。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

普通堆每层通常先挑较大孩子，再与父比较。弱堆通过每位置一个位放松规则，把一些比较得到的信息存下来。建堆、取最大值时，按规定找对比较的两个位置；一次 join 只作一次键比较。省比较的代价是多了一套寻找祖先和读写位的规则。

具体执行步骤

1. 先解释特殊祖先。对位置 j ，若它是父的逻辑左孩子，向父爬一次；继续爬，直到当前位置是父的逻辑右孩子，此时那个父就是特殊祖先。判断左孩子的方法是： j 的奇偶数（除以 2 的余数）等于父位置的 r 位。匹配就把 j 改成 $\text{floor}(j/2)$ ；不匹配就返回 $\text{floor}(j/2)$ 。必须保留原 j 给后面的比较用。
2. $\text{join}(i,j)$ ：若 $a[j] > a[i]$ ，交换两个值，同时把位置 j 的位从 0 变 1 或从 1 变 0；否则什么也不动。位属于位置，不跟着值搬。翻 j 的孩子角色是这套弱堆规则成立的一部分，不能只交换值而省位。
3. 建堆：所有位设 0，从 $j=n-1$ 逐项倒着走到 1。每次找到原位置 j 的特殊祖先 i ，再 $\text{join}(i,j)$ 。由底往上已经建立的小弱堆逐步接入，最后根拥有最大值。空输入不做这些操作。
4. 排序时，把根与当前最后一项交换，让最大值固定在尾部，当前大小 m 减一。若 $m \leq 1$ ，立即停止这次重建，不准去读位置 1。
5. 若 $m > 1$ ，从位置 1 向逻辑左孩子下降。当前 j 的该孩子是 $2j+r[j]$ ；只要它小于 m 就继续到那里。没有该孩子时达到路径末端。
6. 从末端向上，每次 $\text{join}(0,j)$ ，然后 $j=\text{floor}(j/2)$ ，直到 $j=0$ 。这里向上走物理父，不是重复找特殊祖先。这条顺序把路径代表的候选值与根逐个比较，重新使根最大。再取出根、缩小、重复。

跟着数字一步步做

1. $[4,1,3,2]$ 的位全 0。先 $j=3$ ：3 是父 1 的逻辑右孩子，特殊祖先 $i=1$ 。比较 $a[1]=1$ 与 $a[3]=2$ ，换成 $[4,2,3,1]$ ，翻 $r[3]$ 为 1。
2. 再 $j=2$ ：2 为父 1 的逻辑左孩子，爬到 1；1 是根的右孩子，特殊祖先为 0。比较 3 与根 4，不换。 $j=1$ 与根比 2、4，也不换。建堆结束。
3. 交换根与尾，得 $[1,2,3 | 4]$ ， $m=3$ 。从 1 沿逻辑左孩子到 2。 $\text{join}(0,2)$ 比较 1、3，交换得 $[3,2,1 | 4]$ ，翻 $r[2]$ 。再向物理父 1， $\text{join}(0,1)$ 比较 3、2，不换；再到 0 停。
4. 取出根 3，得 $[1,2 | 3,4]$ ， $m=2$ 。从 1 没有存在的左孩子； $\text{join}(0,1)$ 交换 1、2，得 $[2,1 | 3,4]$ ，翻 $r[1]$ 。
5. 最后交换根与另一项，得 $[1,2,3,4]$ ， $m=1$ ，直接停。绝不能还从位置 1 出发，因为它已不属于当前堆。

要做多少事 要用多少地方

设 n 为记录数。建堆每个非根位置做一次键比较，非空输入恰为 $n-1$ 次，连同找祖先总工作 $\Theta(n)$ 。经典取出过程反复走约 $\log_2 n$ 的路径，总最好、一般、最坏 $\Theta(n \log n)$ 。这不意味着全部输入都需要相同交换数。额外必须有 n 个位和 $O(1)$ 控制变量；若每位单独用一个字节，实际要 n 字节。不能写成严格 $O(1)$ 辅助空间。不稳定。

为什么快或慢

每次 join 不用先比较两个孩子，可能减少昂贵键比较。但奇偶判断、找祖先、翻位、跳跃访存也花时间，因此比较少并不保证机器上最快。

什么时候适合用

比较很昂贵的专门排序、弱堆优先队列研究，以及理解如何用少量附加信息省比较。

最后把容易弄错的地方说清楚

资料可能把逻辑左右镜像命名。必须一起采用本节的孩子公式、特殊祖先条件和取出路径，不能各摘一个公式拼起来。位翻转是改子树角色，不是倒序数组。

参考来源 [48] [49] [50]

28 锦标赛排序

类型 算法 别名或相关名称 胜者树排序；淘汰赛排序

先把要用的小动作讲清楚

想成淘汰赛，比较两位选手时小的赢。比赛只记录谁赢，不会把输的原记录删除。

树的叶子是最下层没有孩子的位置，每条原记录占一个叶子。上面的比赛位置存“哪个叶子赢”，不是多出一份要输出的记录。

某叶子退出后，只有通向顶端的比赛曾依赖它；别的比赛结果仍有效。所以不用从头比较所有选手。

退出用“活跃/不活跃”标记表示，不要随便拿一个数字当永远最大；输入可能真的包含那个数字。

基本思路

先让相邻记录两两比赛，再让胜者两两比赛，直到顶端记录最小项。把这一项输出并让其叶子退出，仅重打它向上的比赛。顶端就指出下一最小项，反复输出直到没有活跃记录。

具体执行步骤

1. 选择叶子数 P ，为不小于 n 的最小二的整数次幂（1、2、4、8、16...之一）， n 是输入项数。可以用树格子编号：根为 1，孩子为 $2v$ 、 $2v+1$ ，叶子为 $P..2P-1$ 。前 n 个叶子指向原记录，剩余叶子标不活跃。空输入直接结束。
2. 定义一次比赛。两个孩子都不活跃，就让父不活跃；只有一个活跃，就让父记它的记录编号；两个都活跃就比较记录值，父记较小者的原编号。若相等，用原始输入编号较小者获胜，以保留先来先输出。
3. 从最后父位置 $P-1$ 倒着到 1 做比赛。孩子的结果先做好，父才有正确候选；根最后存着全体冠军的记录编号。
4. 读取根编号，输出那条完整原记录。把对应叶子设不活跃，不能仅清掉根；否则下层还会把同一条推上来。
5. 从这个叶子的父位置开始，重新比较它两个孩子的当前结果，再往父的父走，直到根。这些编号每次除以二向下取整。
6. 根若还有活跃编号，就输出新冠军、退出、更新路径；输出 n 条或根不活跃就结束。输出缓冲只按这些冠军逐条增加，比赛树内编号不再额外输出。

跟着数字一步步做

1. 输入 [4,1,3,2], $P=4$ 。叶子位置 4、5、6、7 分别指原记录 0、1、2、3。位置 2 比叶子 4、5 的 4、1，记原编号 1；位置 3 比 3、2，记原编号 3；根比 1、2，记编号 1。
2. 根指出值 1，输出 [1]，让叶子 5 不活跃。位置 2 只剩 4，自动记编号 0；根比较 4 与另一边胜者 2，改记编号 3。值 3 的叶子和它的比赛都没必要重做。
3. 输出 2，让叶子 7 退出。位置 3 只剩 3，根比较 4、3，冠军变 3，输出已为 [1,2]。
4. 再输出 3、让叶子 6 退出；位置 3 空了，根自动记左边剩下的 4。输出 4，所有叶子不活跃，结果 [1,2,3,4]。四条输入只各输出一次。

要做多少事 要用多少地方

设 n 为项数， $P < 2n$ ($n > 0$)。初建不到 $2P$ 个树位置，工作 $\Theta(n)$ 。每输出一项沿约 $\log_2 n$ 个父位置更新，共 n 次，基础完全胜者树最好、一般、最坏 $\Theta(n \log n)$ 。存叶子编号、比赛结果和通常的输出需要 $O(n)$ 空间；不是原地常数空间方法。按原编号处理所有等值比赛则稳定；随意选某边获胜不一定稳定。

为什么快或慢

每个新最小值只需重打一条路径。保存很多旧比较结果节省重新全扫的工作，但要付树格子、输出和跳跃读取的成本。

什么时候适合用

反复选择下一最小项，特别是把多条已排序输入流接成一条时很自然：输出某流头之后，只换该流的下一项并更新路径。

最后把容易弄错的地方说清楚

败者树是相关结构：比赛位置记录败者，换入选手一路挑战记录的对手，顶端留冠军。外排的替换选择还会把不能接当前输出的项冻结到下一排；这是生成有序排的步骤，不是这个完整排序的另一个名字。

参考来源 [51] [52]

29 内省排序

类型 算法 别名或相关名称 Introspective sort；快速排序与堆排序的混合

先把要用的小动作讲清楚

快排先按一个标准值分左右，再分别处理两边。若每次只能分出一项，几乎整排会被扫描很多遍。

预算在这里是一个整数计数器，记录同一条拆分路径还允许继续分区多少次，不是时间测量，也不是全局所有分区共用一个钱包。

插入动作是拿住下一项、把前面更大的项右移、放进空位。最大堆则是把每个父项摆得不小于孩子，让第一项一定最大。

基本思路

先走通常很快的快排路线，同时数拆分层数。层数太深说明这一条路不顺，就对那一块改用最坏工作受控的堆排序；非常小的块直接插入。它不是一种神秘的新搬动，而是按明确条件选择已有的三组动作。

具体执行步骤

1. 空输入或只有一项时直接结束，不计算预算。其余输入有 n 项时，常把预算设为 $2\text{floor}(\log_2 n)$ 。具体可从整数 n 开始，每次除以二去掉小数部分，直到只剩一，数做了几次，再乘二。 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 有四次，预算八； $10 \rightarrow 5 \rightarrow 2 \rightarrow 1$ 有三次，预算六。选固定小范围阈值。
2. 处理范围时，若它很小，做插入：从第二项起保存 x ，向左看，只要前项 $> x$ 就把前项右移，最后把 x 写入空位，直到整块有序。若不是小范围且预算为零，跳到堆方法。
3. 预算还有时，挑枢轴。常取第一、中间、最后三个值的中位数：几次比较排出这三值，取中间值，不是把所有数求平均。把它移到末端，扫描其余项；用前部边界收集 $\leq$ 枢轴的数，最后把枢轴交换到边界，留下

两边未排块。这里展开一种可用分区，具体库可用其他正确分区。

4. 预算减一，两个子块各获得这个减一后的数。分别继续同样决策；处理一个子块不会另行消耗另一个子块的余额。为省栈，可先调用较小子块，较大子块用循环继续。
5. 后备堆只处理当前未完成块。把块内编号从 0 看起， i 的孩子为 $2i+1$ 、 $2i+2$ 。从最后父项往前，比较孩子选较大的；父比它小就交换并沿新位置往下，直到父足够大或无孩子。
6. 堆根成为最大值后，交换根与本块最后活跃项，缩小活跃范围，再让换入的根沿较大孩子向下找位置。反复直到只余一项。这个块有序后就返回，绝不能重排其他已经确定的范围。

跟着数字一步步做

1. 真实小数组 $[4,1,3,2]$ 通常直接插入：拿住 1，移 4，得到 $[1,4,3,2]$ ；拿住 3，移 4，得到 $[1,3,4,2]$ ；拿住 2，移 4、3，再放到 1 后，成为 $[1,2,3,4]$ 。
2. 为看清切换，另用同样输入，但故意把预算设为一、关闭小数组捷径。选末项 2 为标准。 $4>2$ 留下； $1\leq 2$ ，与前部 4 换位，成 $[1,4,3,2]$ ； $3>2$ 留下。再交换标准与 4，成 $[1,2,3,4]$ 。
3. 枢轴 2 固定，左块 $[1]$ 已完；右块 $[3,4]$ 继承余额零，改建堆。父 3 比孩子 4 小，交换为 $[1,2,4,3]$ ，这里只改右块。
4. 把右块根 4 与它的末项 3 交换，得 $[1,2,3,4]$ ；余下一项停止。这是为展示机制而选的控制设置，不是普通库在四项上真的会走的路线。

要做多少事 要用多少地方

设 n 为项数。每层分区合计最多扫描 n 项，路径预算只有 $O(\log n)$ 层，所以分区总量 $O(n \log n)$ 。后备块互不重叠，各自堆排的工作加起来也不超过 $O(n \log n)$ 。通常 $O(n \log n)$ ，最坏 $O(n \log n)$ ；基础非自适应版最好也约 $O(n \log n)$ ，小块插入不会改变大输入的这个量级。通常实现栈 $O(\log n)$ ，交换临时空间 $O(1)$ 。不稳定。

为什么快或慢

正常数据保留快排的连续扫描；坏分区不能无限延续。额外只多预算判断，坏情况下堆虽然常数成本大些，却把总工作封住。

什么时候适合用

适合通用数组排序，尤其不能容忍特殊输入导致平方级工作的情况。

最后把容易弄错的地方说清楚

预算按当前拆分路径递减，不是按完成了多少交换递减。只有当前未完成范围切换到堆。某些实现最后用一次插入扫描收尾，也要先保证未完成小块的大小受到控制。

参考来源 [53] [54]

30 模式破坏快速排序

类型 算法 别名或相关名称 pdqsort；模式抵抗快排

先把要用的小动作讲清楚

先记住三种小动作：按枢轴扫描分左右；拿住一项向前插入；让最大值沿父子规则升到堆根、再送尾部。这里是把这些动作配上观察和切换条件。

“模式”指输入已有规律，例如有序、逆序或周期排列。这些规律可能让某种选枢轴方式反复选坏标准。打破模式只交换少数位置，不生成或删除记录。

“有限尝试”是给插入搬动规定一个小上限；超过就停这条捷径，继续正常排序。试失败之后的数据可以变了，但所有记录必须还在。

基本思路

pdqsort 一边快排，一边问两个具体问题：这次分区是否很偏？这一块是不是其实容易排？容易的用短插入结束，偏得厉害的先打乱少量位置，坏分区累计太多就堆排。还有专门等值处理，避免把重复项反复拆下去。

具体执行步骤

1. 小范围直接插入：从第二项拿住 x ，前面 $>x$ 的项逐格右移， x 填到停止处。大范围从第一、中间、末尾等小样本选中间值；更大范围可用多个三项样本的中间值再取中间值。这只挑标准，不是完成排序。
2. 一种普通分区把枢轴暂放左端 lo 并保存其值 p ， $i=lo+1$ 、 $j=hi$ 。只在 $i \leq j$ 时扫描：左侧遇值 $<p$ 就 i 加一；右侧遇值 $\geq p$ 就 j 减一；若仍 $i \leq j$ ，交换错边两项，再前进 i 、后退 j 。交叉后把枢轴与位置 j 交换，小项在它左、其余在右，继续两侧。记录的信号是内部错边交换；挑样本和放回枢轴的交换不算。作者实现另有优化的边界检查和分支少版本。
3. 记录内部错边交换是否发生；挑样本、最后放枢轴的交换不等同于这个信号。如果分区不太偏且没有内部交换，分别尝试有限插入。每次向左挪 x 时累计挪了多少格；在小上限内排完就结束这块，超过就放好拿住的记录后退出尝试，继续分区。
4. 如果某一侧少于当前范围约八分之一，就记为坏分区。交换大侧一些预先指定的远处分布位置，让后续样本不再看到同样布局；坏分区预算减一，初值与 $\log_2 n$ 成正比，花完就堆排当前块。交换不能保证立刻均衡，真正最坏保护来自预算和堆后备。
5. 还有一个有条件的等值捷径：若前一次分区已知这一块所有值 $\geq$ 左邻旧枢轴，而新枢轴恰等于旧值，就扫描把等值放前、严格更大者放后，等值区无需再排。没有那条已知下界时，不能擅自忽略较小值。
6. 堆后备把当前块按父 i 、孩子 $2i+1/2i+2$ 看待。从未父向前，挑大孩子，父较小时交换并继续下沉。根成为最大后与活跃末项交换、缩小、继续下沉，直到一项。小范围插入、等值区完成和预算规则共同保证终止。
7. 可选分支少的分区先把左侧错边位置收进小编号缓冲，右侧错边位置收进另一缓冲，再按配对交换。缓冲存的是“哪格错了”，不是额外输出记录；其容量固定。

跟着数字一步步做

1. 作者实现对 $[4,1,3,2]$ 走小范围插入：插 1 得 $[1,4,3,2]$ ，插 3 得 $[1,3,4,2]$ ，插 2 时移 4、3，得 $[1,2,3,4]$ 。这条真实小例没有调用大范围模式逻辑。
2. 单看容易分区信号：局部有序 $[1,2,3,4,5]$ 选标准 3，把它暂放左端，成为 $[3,2,1,4,5]$ 。两头扫描在中间交叉，未交换内部错边对；枢轴回边界后变回 $[1,2 | 3 | 4,5]$ 。两边插入检查都不用移较大项，可快速确认。
3. 单看坏分区信号：一块十六项若标准最小，左边零项、右边十五项，属于坏分区。预算减一，再交换右边若干样本位置；若随后仍不断出现这种分割，余额用完就只堆排余块。固定位置的具体选择是作者实现策略，不能由这个说明猜出一套必定最快的置换。

要做多少事 要用多少地方

设 n 为项数。容易且被识别的输入可只扫描和短检查，最好 $O(n)$ 。一般 $O(n \log n)$ 。每次坏分区减少预算，大幅未缩小的范围不能无限重复；堆后备使最坏 $O(n \log n)$ 。通常作者实现栈 $O(\log n)$ ，固定容量编号缓冲额外 $O(1)$ 个机器字。不稳定。阈值与固定交换位置影响常数，不改变后备的总工作保护。

为什么快或慢

不盲目把容易排的块继续拆，也不任由坏模式反复耗时。额外的试探、采样和缓冲有成本，比较本身昂贵时，减少条件分支未必更划算。

什么时候适合用

高效通用数组排序，且不要求等值记录保持先后时。适合了解实际库如何把多个小优化组织成明确控制流程。

最后把容易弄错的地方说清楚

有限插入失败后仍须把拿住的记录放回；不能直接返回并丢掉它。原作者完整实现还包含防越界哨兵、采样布局等工程细节，生产应复用其代码。

参考来源 [\[55\]](#) [\[56\]](#) [\[57\]](#)

31 伸展树排序

类型 算法 别名或相关名称 Splay sort ; 自调整树排序

先把要用的小动作讲清楚

先有查找树：比节点小的走左，大的走右，空连接才建新节点。它不是已经有序的数组，只是保存了大小关系。

一次右旋把父 p 的左孩子 x 提到上面。 p 成 x 的右孩子， x 原来的右分支接到 p 的左边；该分支的值介于 x 与 p 之间，所以放在那里合适。左旋完全反向。

当前节点 x 的上一层叫父 p ，再上一层叫祖父 g 。要看 x 和 p 分别是上一层的左孩子还是右孩子，才能选择旋转顺序。

“摊还”看整串操作总成本。有一次走很长，不代表每次都长； $O(\log n)$ 摊还不是给每个单次操作承诺的上限。

基本思路

每插入或访问一个节点，都把它通过旋转送到根，叫作 splay。刚用过的值和附近值以后可能更容易到达。全部插完后按左、当前、右读出。这是一种会自己改形状的树排序，不是每次都硬性维持树高。

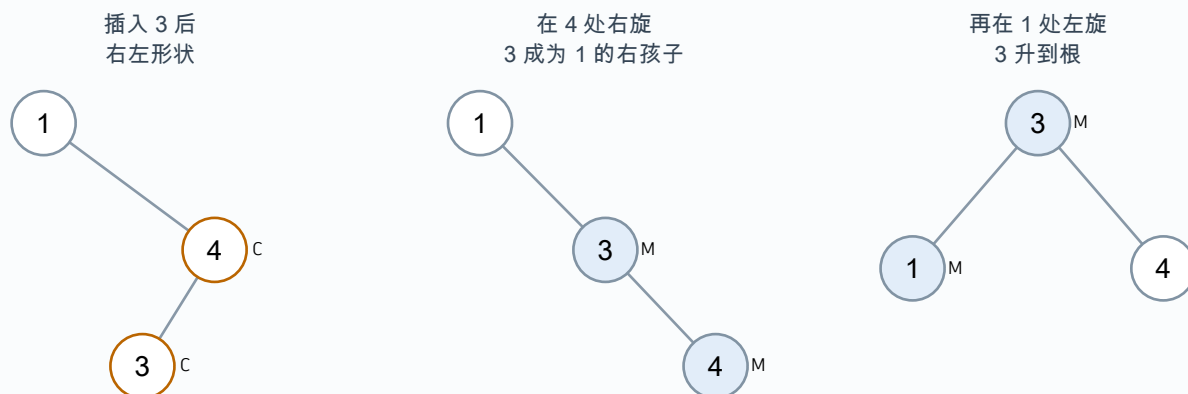
具体执行步骤

1. 插入 x 时从根逐节点比较，小则向左、大则向右，到空连接建节点。相等可以在现有节点队列末尾追加记录，并把这个现有节点当作 x 。记住沿途父连接，接下来需要向上改变它们。
2. 若 x 已是根，不转。若父 p 是根，只转一次： x 在 p 左则右旋 p ， x 在 p 右则左旋 p 。 x 直接成为根；这个情形称 zig。
3. 若 x 、 p 都是左孩子，先右旋祖父 g ，把 p 提上去，再右旋 p ，把 x 提上去；都是右孩子时先左旋 g 再左旋 p 。此同方向双旋叫 zig-zig。顺序是先父上祖父，不是随便转两次。
4. 若 x 在 p 右、 p 在 g 左，先左旋 p ，让 x 上到 p 原位置，再右旋 g ；反向情形先右旋 p 再左旋 g 。这种拐弯双旋叫 zig-zag。
5. 每次重接被转部分与更上层的连接；检查 x 现在的父、祖父，再选合适情形继续，直到 x 成根。转动时只改连接，不丢子树；转移的中间分支必须挂回去。
6. 所有记录都插完后，读出规则是：空连接返回；先完全读左分支，再输出节点队列，再读右分支。用调用栈或显式列表记住待返回的节点。相等队列保持到达次序。

跟着数字一步步做

1. 输入 $[4,1,3,2]$ 。4 是根；插 1 到它左边。父 4 是根，右旋一次：1 成根，右孩子是 4。顺序关系 $1 < 4$ 不变。
2. 插 3：从 1 往右到 4，再往左建 3。是右—左拐弯：先右旋 4，3 取代 4 成为 1 的右孩子，4 接 3 右；再左旋 1，根成 3，左右孩子为 1、4。
3. 插 2：从 3 左到 1，再右建 2。先左旋 1，2 成 3 的左孩子、1 成 2 左孩子；再右旋 3，根成 2，左孩子 1、右孩子 3，3 右仍有 4。四条记录都在。
4. 读出先走到 1 输出它，再输出根 2，然后右分支的 3、4，得到 $[1,2,3,4]$ 。
5. 另观察某个待转部分若是左链 $4 \rightarrow 3 \rightarrow 2$ ：先右旋 4，变 3 上、左 2、右 4；再右旋 3，变 2 上、右 3、3 右 4。这说明 zig-zig 的两次顺序，并不是说刚才四项例子一定经过此形状。

伸展树的两次旋转 对应插入 3 的文字步骤



要做多少事 要用多少地方

设 n 为项数。一轮插入包含路径比较和沿路径若干旋转；单次可能 $O(n)$ 。但按 splay 规则算整个序列，通常和完整排序最坏 $O(n \log n)$ ，不能把最坏单次乘 n 得出此算法一定平方。单调到达时每次新值在根旁插入并转一次，最好可 $\Theta(n)$ 。节点 $O(n)$ 空间；最终树仍可能是长链，递归读出最坏栈 $O(n)$ ，不是固定 $O(\log n)$ 。相等队列或原编号规则正确则稳定。

为什么快或慢

根始终靠近最近使用的值，后续相近键可能少比较。旋转、连接读取和分配盒子都花时间，不能只看比较量级。

什么时候适合用

适合学习利用已有顺序的树排序，或本来要使用自调整查找树的数据。

最后把容易弄错的地方说清楚

把两个同方向旋转顺序反过来，会变成别的调整规则，不能直接沿用 splay 的摊还结论。树可以很高；必须保留旋转中的中间分支，不能提升节点时顺手删除它。

参考来源 [58] [59] [60]

32 最小最大堆排序变体

类型 重要变体 别名或相关名称 最小最大堆提取排序；双端优先队列排序

先把要用的小动作讲清楚

把数组按层填进二叉树：根占位置 0，两个孩子占 1、2，下一层从左到右占 3、4、5、6。每个位置 i 的孩子下标是 $2i+1$ 、 $2i+2$ ，存在才读取。父位置是 $\text{floor}((i-1)/2)$ ， floor 就是去掉小数部分。

普通最小堆只便于取最小。这种树让层交替承担大小任务：根是最小层，下一层最大，再下一层最小。最小层节点不大于它的所有后代，最大层不小于后代。所以根最小，根的两个孩子中较大的给全树最大。

修复时为什么看孙子？同性质的下一层是隔一层的孙子：最小层应该沿最小层继续向下检查。直接孩子属于最大层，交换时还要照顾这一层的相反大小关系。

基本思路

同时在一棵树里保留最小端和最大端。要升序输出，就反复拿根的最小值，然后把树修好。它适合本来就需要“两端都能取”的结构，单纯排序未必值得这些额外检查。

具体执行步骤

1. 用数组表示完整二叉树，层从 0 开始，偶数层最小、奇数层最大。建堆时可以用层边界一次认清角色：位置 0 是第 0 层，1–2 是第 1 层，3–6 是第 2 层，7–14 是第 3 层；下一层起点是本层起点的两倍加 1。倒着扫描时，只有越过层起点才把层数减 1，不必让每个节点反复走父链。编号 $i+1$ 的二进制位数减 1 也等于层数，但需要按所用机器的位操作成本计算。

2. 从最后一个有孩子的位置 $\text{floor}(n/2)-1$ 开始，向左直到根，逐个修复，保证当前节点之下的子树已修好。
3. 在最小层，检查存在的两个孩子和最多四个孙子，找其中最小。没有比当前值小的就停；最小候选是直接孩子则交换一次并停。
4. 候选是孙子时，交换当前值与它。下移到孙子位置的旧值若比它的最大层父亲还大，再和父亲交换校正。接着从该孙子位置继续第 3 步。最大层全做对称操作：找最大、过小的下移值与最小层父亲校正。
5. 建好后，取根写输出。把数组最后一条搬到根，缩短树，用最小层规则修复根；反复取出直到树空。零项、一项时直接处理，不读取不存在的孩子。
6. 若需降序，可以改取根孩子中较大的记录并做最大层修复。本文的升序版本使用独立输出数组，不能把它说成完全零额外空间。

跟着数字一步步做

1. 输入 [4,1,6,2,5,3]。根 4 是最小层，孩子 1、6 是最大层。最后有孩子节点 6 下只有 3，6 已够大；节点 1 下是 2、5，应把最大 5 换上，得 [4,5,6,2,1,3]。
2. 修根 4 时，孩子 5、6 和孙子 2、1、3 中最小的是 1。与它交换，成 [1,5,6,2,4,3]。下移 4 的父是 5， $4 \leq 5$ ，因此不用校正；它已在叶子，停止。
3. 输出 1，用末项 3 补根并删去末格，得到 [3,5,6,2,4]。根候选最小 2 在孙子处，交换成 [2,5,6,3,4]； $3 \leq 5$ ，无需校正。
4. 输出 2，末项 4 补根，剩 [4,5,6,3]；与孙子 3 换成 [3,5,6,4]。再出 3，4 补根，孩子 5、6 都较大，不换；出 4，6 补根，与直接孩子 5 换成 [5,6]；最后出 5、6。完整结果 [1,2,3,4,5,6]。

要做多少事 要用多少地方

底向上建树为 $O(n)$ ，每次取最小最多沿约 $\log n$ 层修复，所以全排序上界 $O(n \log n)$ 。一般互异输入达到这个量级，全等输入可能少做修复。 n 是记录数。堆本身复用输入，但这里把逐个取出的记录写到新数组，需 $O(n)$ 输出空间；迭代管理仅 $O(1)$ 。通常不稳定。它比普通堆多检查孩子、孙子和跨层关系，用途是同时提供两端，而不是无条件更快。

为什么快或慢

同时获得最小和最大值很方便；单纯全排序通常不需要这份双端能力，且孩子与孙子校正比普通堆复杂。

什么时候适合用

已经需要双端优先队列、之后又要提取为有序结果的应用。

最后把容易弄错的地方说清楚

看孩子却不看孙子，会漏掉隔层的更小或更大记录。候选是直接孩子与候选是孙子，后续处理不同；这也是步骤必须分开讲的原因。

参考来源 [61]

33 块归并排序：Grailsort / WikiSort

类型 算法家族 别名或相关名称 块排序；Grailsort 与 WikiSort 是相关实现，互非别名

先把要用的小动作讲清楚

块是一串连续格子。交换两块时，第一项互换、第二项互换，直到块末；不是只交换块首。

工作区临时装数据。本算法借原数据作工作区；里面的原记录仍要归还，不能被覆盖丢掉。

标签是暂放在 A 块首的不同数，表示块原先次序。真正首值与标签交换，寄存在标签区；标签次序和真正数值有不同用途。

旋转能换邻段而保留各段内部顺序：各自反转，再整体反转。[7,8 | 5,6] 经 [8,7 | 5,6]、[8,7 | 6,5] 后，整体反转成 [5,6 | 7,8]。

$|A|$ 表示 A 中的项数； $\sqrt{\quad}$ 表示平方根，例如 16 的平方根是 4，因为 $4 \times 4 = 16$ 。

基本思路

借少量真实记录作工具，把其余数成块搬动，再做局部接排，最后归还工具记录。以下具体讲 WikiSort 的两内部缓冲机制：只给左有序排 A 的块贴标签，右排 B 保留真正块首。Grailsort 是相关实现，但调度和边界不能混用。

具体执行步骤

1. 先形成相邻有序排 A、B。短排可插入排序：拿住下一项，把前面更大的项右移，再放进空位。大排逐轮接起来。一次大接排取约 $\sqrt{|A|}$ 的块长，即块长与块数大致相等。
2. 找两组不同键：一组能装一块作工作区，另一组有每个剩余 A 块一个标签。先找选中键的位置，再从后面往前旋转，将选中键聚到保留区，保持未选记录相对次序。保留区不参加滚动。
3. 按 A 块原次序，用升序标签分别与块首交换。真正首值寄存在相应标签槽。最小剩余标签指出下一 A 块；读标签区的对应真首值才能决定它插在哪。B 块不贴标签。
4. 把滚动区最左 A 块与下一个同长 B 块逐项交换，让 B 前进、A 后移；A 块允许暂乱序。若下一 A 真首值仍大于刚越过 B 块的末值，再滚一块 B。否则扫描剩余 A 标签，找最小标签块，把它换到滚动区最左。
5. 在刚越过的 B 块内找第一个不小于 A 真首值的位置：比较中间值，若太小就找右半，否则找左半，直到得到插入点。旋转 B 的这个后缀与 A 块，让 A 放在此处，相等时 A 在前。交换标签与寄存真首值，把块首恢复。
6. 下一 A 块落定后，前一 A 和夹在两块之间的 B 项可以排定。先把前一 A 逐项交换到工作区。比较工作区未用头和 B 未用头，把较小者与下一输出格交换，前进所选头和输出位置。相等先取 A。被挤走的借用记录仍存着，不能直接覆盖。一边空后，另一边余项顺序交换到位。
7. 最后一 A 与剩余 B 同法处理。打乱的工作区键用插入排序复原；标签恢复后仍有序。用搜索和旋转把保留键送回主体，按抽取来源保持等值先后。不同键不够时，WikiSort 只保留一个标签区、加大块长以减少块数，局部用搜索加旋转；不足整块的首尾片段也另作旋转。

跟着数字一步步做

1. 从整体仍无序的 $[1,2,3,4,6,9,11,14 \mid 5,7,8,10,12,13,15,16]$ 开始。两半分别有序，但 $14 > 5$ 。借 $[1,2]$ 作工作区、 $[3,4]$ 作标签，主体 $A=[6,9][11,14]$ ， $B=[5,7][8,10][12,13][15,16]$ 。16 个数全保留。
2. 标签 3 换真首 6，标签 4 换真首 11。A 暂为 $[3,9][4,14]$ ，标签区 $[6,11]$ 。滚过 $[5,7]$ ，因 $6 \leq 7$ ，在 7 前插入第一 A 块并恢复真首，形成 $[5][6,9][7]$ 。
3. 第二 A 真首 $11 > 10$ ，越过 $[8,10]$ ；到 $[12,13]$ 时 $11 \leq 13$ ，在 12 前落定并恢复。主体布局 $[5][6,9][7,8,10][11,14][12,13,15,16]$ ，标签回到 $[3,4]$ 。
4. 交换 $[1,2]$ 与 $[6,9]$ ，工作区变 $[6,9]$ ，局部暂为 $[1,2,7,8,10]$ 。取 6 与输出格的 1 交换，局部 $[6,2,7,8,10]$ 、工作区 $[1,9]$ 。 $9 > 7$ ，交换出 7，局部 $[6,7,2,8,10]$ ； $9 > 8$ ，再交换出 8，局部 $[6,7,8,2,10]$ 。
5. $9 < 10$ ，交换工作区 9 与输出格 2，局部 $[6,7,8,9,10]$ ，工作区回 $[1,2]$ 。10 已在正确格。最后一 A 与 B 输出 11、12、13、14、15、16。主体成为 5 到 16，归还 $[1,2]$ 、 $[3,4]$ ，得到完整 1 到 16。小块用于展示；正式实现直接处理这种小数组。

要做多少事 要用多少地方

设 n 是项数。成熟实现每层工作 $O(n)$ ，约有 $\log_2 n$ 层，通常和最坏 $O(n \log n)$ 。一轮约 $\sqrt{n}$ 个 A 块，反复扫描这些标签最多约 $(\sqrt{n})^2 = n$ 次；把 $\sqrt{n}$ 个工作键插入重排也只需 $O(n)$ 。原地配置外部空间 $O(1)$ 个机器字；借来的 $O(\sqrt{n})$ 项属于输入，不是新分配。可选外部缓冲改变空间。跳过已顺序正确的接排可使最好接近 $O(n)$ 。成熟实现稳定，前提是抽取、相等规则、归还都正确。

为什么快或慢

用块移动避免让单个左项反复跨越整排。省内存的代价是抽取、标签、恢复与边界管理，常数成本高，初学实现容易错。

什么时候适合用

需要稳定又不能分配完整副本时有价值。先手演示动作，生产时复用经过测试的代码。

最后把容易弄错的地方说清楚

标签只标明原块次序，不能用它代替真首值决定插入点。少键和不整块情况是完整算法所需规则，不是可任意省去的优化。

参考来源 [62] [63] [64]

34 快速排序：Lomuto 分区

类型 算法 别名或相关名称 快排；本节采用 Lomuto 单向扫描分区

先把要用的小动作讲清楚

本节用从 0 开始的位置编号。lo、hi 分别是当前排的第一和最后位置，两端都属于这一排。

枢轴只是临时挑来作分界标准的数。这里挑最后一项；其他记录照旧保留。

分区不是完全排序，而是把“小于等于标准”的放一边，把“更大”的放另一边。交换两个格子时，两个值都保留。

a 是这排数的名字，a[i] 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

从左到右看一次，把不大于末尾枢轴的数收集到前面。最后把枢轴夹到两组之间。这一项此时已经到正确位置；再对左右两组重复，所有位置就逐渐确定。

具体执行步骤

1. 若范围少于两项，结束。保存末项的值 p，令 b=lo：b 指向前面已收集小项区之后的第一个位置；最开始小项区为空。
2. 令扫描位置 j 从 lo 走到 hi-1，暂时不扫描枢轴格。若 $a[j] \leq p$ ，交换 $a[j]$ 与 $a[b]$ ，然后 b 加一。若 $a[j] > p$ ，不交换，只让 j 继续。
3. 为什么可交换？b 到 j 之间已经看过的项都大于 p，把其中一项换到 j，并不会破坏小项区；j 自己等于 b 时自交换也无害。
4. 扫描结束，把末尾枢轴与 $a[b]$ 交换。b 前的项全 $\leq p$ ，b 后的项全 $> p$ ，所以 b 上这个枢轴无需再参与排序。
5. 对 $[lo, b-1]$ 和 $[b+1, hi]$ 分别重做这些动作。每次留下一个固定位置，范围变小，单项或空范围停止。可以用函数调用记住待做范围，也可自己用列表记录它们。

跟着数字一步步做

1. $[4, 1, 3, 2]$ 中 $p=2$ 、 $b=0$ 。j=0 看 4， $4 > 2$ ，留下；j=1 看 1， $1 \leq 2$ ，交换位置 1 与 0，成为 $[1, 4, 3, 2]$ ， $b=1$ 。
2. 看 j=2 的 3， $3 > 2$ ，留下。扫描结束，交换 $b=1$ 上的 4 与末尾 2，成为 $[1, 2, 3, 4]$ 。位置 1 的 2 固定，左组 $[1]$ 、右组 $[3, 4]$ 。
3. 左组单项停止。右组末项 $p=4$ ，扫描 3 时它 ≤ 4 ，只和自己交换；枢轴 4 最后也和自己交换，右组已排好。结果 $[1, 2, 3, 4]$ 。这个例子刚好一次分区就形成全局顺序，通常左右组仍要继续处理。

步骤状态图 C 比较 M 移动 S 已有序

末尾枢轴 2；4 大于 2，略过



1 ≤ 2：交换 1 和前部 4



3 > 2；再把枢轴换到边界



左右子区间完成



要做多少事 要用多少地方

设 n 为记录数。一轮分区扫描整个当前范围，所以成本随范围长度增长。若每次分成大小接近的两组，约有 $\log_2 n$ 层，最好 $\Theta(n \log n)$ ；随机打乱输入或随机选枢轴后期望 $\Theta(n \log n)$ 。每次只分出一项则要扫描 $n, n-1, n-2 \dots$ 项，最坏 $\Theta(n^2)$ 。本规则全相等时每项都进左组，也会退化。通常调用栈 $O(\log n)$ ，最坏 $O(n)$ ；每次只调用较小组、较大组用循环可把栈限在 $O(\log n)$ 。不稳定。

为什么快或慢

扫描动作很简单，许多小项能直接前移。但末项若总是极值，大部分范围要反复扫描；相等项多也不利。自交换和记录移动有真实成本。

什么时候适合用

适理解快排基本机制和核对分区逻辑。通用工程实现通常增加随机枢轴、三路处理或退化回退。

最后把容易弄错的地方说清楚

固定的是这次枢轴的一项，不是前面全部数。边界 b 两边再排时必须排除 b 。若改成严格小于 p ，等值分组会变，但单侧等值退化仍可能存在。

参考来源 [65] [66]

35 快速排序：Hoare 分区

类型 重要变体 别名或相关名称 Hoare 原始分区；双向扫描快排变体

先把要用的小动作讲清楚

这里位置从 0 编号。 lo 、 hi 都包括在当前范围内；靠左中点就是 $(lo+hi)$ 除以 2 后去掉小数部分。

两个指针是两个位置编号：一个向右找，一个向左找；它们不会自动移动，步骤要明确规定每次移多少。

枢轴只是一份保存的数值。原来存它的记录可以被交换到别处，但比较标准不变。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

从两头找“站错边”的数：左边找不小于枢轴的，右边找不大于枢轴的，把这一对交换。扫描碰到或越过时，便得到能分别继续排的两组。返回的位置只是分界；不能当成枢轴已固定的位置。

具体执行步骤

1. 范围少于两项就停止。取下中点 $\text{floor}((\text{lo}+\text{hi})/2)$ 的值为 p ； floor 的意思是向下取整。设 $i=\text{lo}-1$ 、 $j=\text{hi}+1$ ，先站在范围外。不要改用上中点；两项有序时可能分出没有缩小的范围。
2. 先让 i 加一，再只要 $a[i]<p$ 就继续加一；遇到 $\geq p$ 的项停下。让 j 先减一，再只要 $a[j]>p$ 就继续减一；遇到 $\leq p$ 的项停下。
3. 若 $i \geq j$ ，不交换，返回 j 。左范围 $[\text{lo}, j]$ 的项都 $\leq p$ ，右范围 $[j+1, \text{hi}]$ 的项都 $\geq p$ 。这里允许两组都有等值项。
4. 若 $i < j$ ，交换两处。较小或相等项去了左边，较大或相等项去了右边；回到扫描步骤，两指针都先移动一次，避免在等值处永远打转。
5. 分别排包含 j 的 $[\text{lo}, j]$ 与 $[j+1, \text{hi}]$ ，重复同样规则。下中点与上述扫描使两个子范围比原范围小；零项或一项停止。不要丢掉分界 j ，它没有被保证固定。

跟着数字一步步做

1. $[4, 1, 3, 2]$ ， $\text{lo}=0$ 、 $\text{hi}=3$ ，下中点索引 1 的值是 $p=1$ 。 i 从 -1 前进一步，在 4 停； j 从 4 后退，跳过 2、3，在位置 1 的 1 停。
2. 交换 4、1，成为 $[1, 4, 3, 2]$ 。下一轮 i 前移到位置 1 的 4； j 后移到位置 0 的 1。 $i \geq j$ ，返回 $j=0$ 。两组为 $[1]$ 、 $[4, 3, 2]$ 。
3. 右组边界 1..3，下中点 2 的值 $p=3$ 。左扫描停 4、右扫描停 2，交换成为 $[1, 2, 3, 4]$ 。再移动后都停在位置 2 的 3，返回分界 2，留下 $[2, 3]$ 、 $[4]$ 。
4. $[2, 3]$ 的下中点是 2；两扫描相遇在它的左位置，分成 $[2]$ 、 $[3]$ 。全体停止，结果 $[1, 2, 3, 4]$ 。本例某些分界刚好也是最终位置，但算法没有这个承诺。

步骤状态图 C 比较 M 移动 S 已有序

保存中间值 $p=1$ ；停在 4 与 1



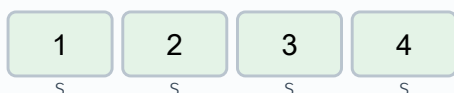
交换；之后返回分界 0



右组 $p=3$ ：交换 4 与 2



继续按分界递归后完成



要做多少事 要用多少地方

设 n 是项数。每次两头扫描总共经过当前范围的项；一项不会在同一轮被无限追看。均衡分区最好 $\Theta(n \log n)$ ，随机输入或恰当随机枢轴下期望 $\Theta(n \log n)$ ，坏分区最坏 $\Theta(n^2)$ 。全等时两指针不断对称前进，仍分成近半两组，时间 $\Theta(n \log n)$ ，没有三路快排的一轮结束优势。通常栈 $O(\log n)$ ，最坏 $O(n)$ ；较小侧调用、大侧循环可把栈限制到 $O(\log n)$ 。不稳定。

为什么快或慢

一个交换同时纠正两边的一项，通常比只从一边收集少一些交换。不过次数少不代表任何数据上都快；分区是否均衡仍然关键。

什么时候适合用

常用作高效数组快排的分区方式，前提是仔细配合中点选择、等值停止和子范围边界。

最后把容易弄错的地方说清楚

有的教材把最后放回枢轴的双向扫描也称 Hoare 风格。那个版本会固定一项，边界不同。本节使用原始返回分界方案，必须保留 j 在左子范围。

参考来源 [67] [68]

36 三路快速排序

类型 重要变体 别名或相关名称 三向快排；肥枢轴快排；分区使用荷兰国旗思路

先把要用的小动作讲清楚

把一排分成三种是按值与标准比较：小于、等于、大于。等于不是删除重复数；每条等值记录都要留下。

三个位置把数组分成四区：已知小、已知等、尚未检查、已知大。只有尚未检查区需要继续判断。

交换之后，当前位置换进来的数可能从未检查过，所以不能一律向前走。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

普通快排对许多等值项可能反复做无用分区。三路快排一次把所有等于枢轴的项集中到中间，中间不用再排，只处理较小和较大两边。

具体执行步骤

1. 选一个范围内的值保存为 p 。位置从 0 编号，设 $lt=lo$ 、 $i=lo$ 、 $gt=hi$ 。 $lo..lt-1$ 是已知小项； $lt..i-1$ 是等值项； $i..gt$ 尚未检查； $gt+1..hi$ 是已知大项。开始前三个已知区都为空。
2. 只在 $i \leq gt$ 时判断 $a[i]$ 。若 $a[i] < p$ ，交换 $a[i]$ 与 $a[lt]$ ，然后 lt 、 i 各加一。小项进了左区；若换进的是一个等值项，它已经检查过，可以同时前进。
3. 若 $a[i] = p$ ，只让 i 加一，中间等值区扩大一项，保留这条记录。
4. 若 $a[i] > p$ ，交换 $a[i]$ 与 $a[gt]$ ， gt 减一，但 i 不前进。大项已进右区；从右边换来的项还不知道大小，必须再检查一次。
5. 当 $i > gt$ ，未知区已经空了。只重排 $lo..lt-1$ 与 $gt+1..hi$ ；中间 $lt..gt$ 的每项都等于 p ，作为整体完成。子范围少于两项时结束。

跟着数字一步步做

1. $[3, 1, 3, 2, 3]$ ，取 $p=3$ ， $lt=i=0$ 、 $gt=4$ 。第一项 3 相等， i 到 1。看到 $1 < 3$ ，交换位置 1、0，变 $[1, 3, 3, 2, 3]$ ； $lt=1$ 、 $i=2$ 。
2. 位置 2 的 3 相等， i 到 3。 $2 < 3$ ，交换位置 3 与 $lt=1$ ，变 $[1, 2, 3, 3, 3]$ ； $lt=2$ 、 $i=4$ 。最后一项相等， i 到 5，未知区空了。
3. 小组 $[1, 2]$ 继续排，选 1 作标准。1 相等； $2 > 1$ ，交换自己并缩小右界，留下单项 $[2]$ 。最终 $[1, 2, 3, 3, 3]$ ，三个 3 全保留。
4. 为看清“不前进”：局部 $[2, 3, 1]$ 取 $p=2$ ，看到中间 3 就和右端 1 交换成 $[2, 1, 3]$ ；仍检查同一中间位置的新 1，把它换到左边，得到 $[1, 2, 3]$ 。若直接跳过，就漏查了 1。

步骤状态图 C 比较 M 移动 S 已有序

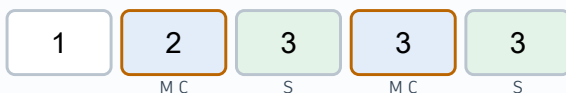
枢轴 3；检查 1



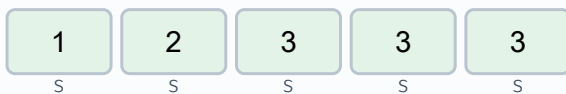
1 小于 3，交换到左侧



2 小于 3；中段三个 3 已完成



再排左侧 [1,2]，完成



要做多少事 要用多少地方

设 n 是项数。一轮中每项进入某个已知区，扫描 $\Theta(n)$ 。全部相等时一轮就完成，最好 $\Theta(n)$ 。不同键、随机枢轴或随机输入通常期望 $\Theta(n \log n)$ ；枢轴连续极偏时最坏 $\Theta(n^2)$ 。通常栈 $O(\log n)$ ，最坏 $O(n)$ ，采用较小侧调用可把栈限制 $O(\log n)$ 。交换能跨过等值记录，因此不稳定；中间值相同不等于原身份顺序不变。

为什么快或慢

大量重复值只需判断一次，不再把等值区拆下去。不同键居多时仍是快排，选坏枢轴依然有反复扫描成本。

什么时候适合用

重复键多的统计、类别、评分数据很适合；也常作字符串按单个字符分组的基础。

最后把容易弄错的地方说清楚

不要在大于分支同时增加 i 。三个已知区的边界每一步都要匹配；把等值区遗漏或输出一次，都会丢掉重复记录。

参考来源 [69] [70]

37 双枢轴快速排序

类型 重要变体 别名或相关名称 双轴快排；Yaroslavskiy 风格双枢轴分区

先把要用的小动作讲清楚

两个枢轴是两个保存的标准值 $p \leq q$ 。它们把值分成“小于 p ”“从 p 到 q ”“大于 q ”三段。

扫描区只是一段还没判断的格子；前后都可留已经分类的记录，不需要立刻让它们内部有序。

末尾换来一个新数时要重新判断，这一点和三路分区相同。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

把两个标准放在两端，扫描中间。很小的向左收集，很大的向右收集，其余留在中段。最后把两个标准放到各自边界，再分别排三段。本节展开一个简单可执行的教学分区；成熟 Yaroslavskiy 实现会用更精细的扫描和优化。

具体执行步骤

1. 少于两项停止。比较两端，若左大右小就交换，保存左值 p 、右值 q 。设 $lt=lo+1$ 、 $gt=hi-1$ 、 $i=lt$ ；两端暂不参与扫描。
2. 当 $i \leq gt$ ，若 $a[i] < p$ ，和 $a[lt]$ 交换， lt 、 i 各加一。若 $p \leq a[i] \leq q$ ，只让 i 加一，这项属中段。
3. 若 $a[i] > q$ ，和 $a[gt]$ 交换，再把 gt 减一。暂不移动 i ，回头判断换来的项；它可能小于 p 、位于中段或仍大于 q 。
4. 未知区空后， lt 减一、 gt 加一。交换左端枢轴与 $a[lt]$ ，交换右端枢轴与 $a[gt]$ 。两个枢轴夹住已分类的三段。
5. 分别处理 $lo..lt-1$ 、 $lt+1..gt-1$ 、 $gt+1..hi$ ，不再含这两项。若 $p=q$ ，中段全相等，可整段跳过。若 $p \neq q$ ，中段只是落在两值之间，仍要排。单项和空段停止。

跟着数字一步步做

1. $[4,1,5,2,3]$ 先交换端点 4、3，成为 $[3,1,5,2,4]$ ， $p=3$ 、 $q=4$ ； $lt=i=1$ 、 $gt=3$ 。 $1 < 3$ ，和自己交换， $lt=i=2$ 。
2. 看到 $5 > 4$ ，交换位置 2 与 3，成为 $[3,1,2,5,4]$ ， $gt=2$ 。 i 仍为 2，重新看到 $2 < 3$ ，和 lt 的自己交换， $lt=i=3$ 。 $i > gt$ ，未知区空。
3. lt 减到 2，左枢轴 3 与位置 2 的 2 交换，得到 $[2,1,3,5,4]$ 。 gt 加到 3，右枢轴 4 与位置 3 的 5 交换，得到 $[2,1,3,4,5]$ 。
4. 枢轴 3、4 固定；左段 $[2,1]$ 用端点比较换成 $[1,2]$ ，中段为空，右段 $[5]$ 单项。完成 $[1,2,3,4,5]$ 。

要做多少事 要用多少地方

设 n 是项数。一轮仍扫描当前范围，但一项可能先与 p 再与 q 比较，并有搬动。不同键、良好采样或随机输入通常期望 $O(n \log n)$ ，最坏 $O(n^2)$ ；把两端直接作枢轴没有普遍速度保证。本教学版若两标准相同就跳过中段，全等最好 $\Theta(n)$ 。通常栈 $O(\log n)$ ，未经控制最坏 $O(n)$ 。不稳定。各具体双枢轴实现的精确比较数不同，不能混成一个结论。

为什么快或慢

一轮可留下三块待做范围，可能减少后续层数，却增加一轮内部判断和交换。是否更快还受采样、重复值、缓存和机器影响。

什么时候适合用

适合研究多标准分区，以及了解一些语言库为什么采用双枢轴快排。

最后把容易弄错的地方说清楚

千万不要因为中段在 p 与 q 之间就认为中段已经有序。例如 $[2,7,3,8]$ 的中间两项仍可能要再排。这里的步骤不声称逐行复制某个语言运行库的实现。

参考来源 [\[71\]](#) [\[72\]](#)

38 样本排序

类型 算法家族 别名或相关名称 抽样排序；Sample sort；多路分区家族

先把要用的小动作讲清楚

桶是一组临时收集的记录；进桶不等于桶内有序。桶按数值范围排序，因此前桶所有值 $\leq$ 后桶所有值。

样本是从原输入选出少量值的副本，用来决定界限。原记录仍要正常分类，样本副本不能额外加到输出。

有多个升序界限时，可以比较中间界限、缩小候选范围，避免每次从第一个界限逐个试。

基本思路

用小样本猜出几个能把数据分得较均匀的界限，把所有记录放进相应范围桶，分别排桶，再按桶次序接起来。它用值比较找桶，不要求键是能直接当编号的整数。

具体执行步骤

1. 确定桶数 b ，抽取比 b 多一些的样本。把样本排好，例如按插入动作把下一值插进已排前部。再从排好样本的约等间隔位置取 $b-1$ 个界限；样本多一些通常更不易选偏，但没有一次必定均匀的承诺。
2. 明确每个界限的归属，例如等于界限进左边桶。对每条记录用界限比较找到唯一桶。比较中间界限后，一次可以排除一半候选桶；最终得到范围编号。
3. 最容易的教学实现给每桶一个列表，按输入先后把完整记录追加过去。也可先统计数量、计算起止位置，再通过交换放成原数组内的连续桶；后者边界管理更复杂。
4. 每桶分别排：短桶拿住下一项、右移更大前项、插入；大桶可再取样划分。若所有值或界限都相等、或者整桶还全落入一个同样大子桶，就停止这条分桶路线，检测全等或换成有最坏保证的排序。例如反复二分小排、比较两排头输出，可保证继续缩小。
5. 从最低范围桶开始，按各桶已经排好的次序输出。无需再跨桶比较，因为界限已确保前桶不会有超过后桶的值。

跟着数字一步步做

1. 输入 $[7, 2, 5, 1, 6, 3]$ ，教学界限取 2 、 6 ，范围为 $x \leq 2$ 、 $2 < x \leq 6$ 、 $x > 6$ 。可从样本 $[7, 2, 5, 1, 6]$ 排得 $[1, 2, 5, 6, 7]$ 后，取其中 2 、 6 ；这些是副本，原 2 、 6 还在输入。
2. 逐项分类： $7 > 6$ 进第三桶； $2 \leq 6$ 且 ≤ 2 进第一桶； $5 \leq 6$ 且 > 2 进第二桶； 1 进第一； 6 进第二； 3 进第二。桶为 $[2, 1]$ 、 $[5, 6, 3]$ 、 $[7]$ 。
3. 第一桶拿住 1 、移 2 ，成 $[1, 2]$ 。第二桶先 $[5, 6]$ 无需挪；拿住 3 ，依次移 6 、 5 ，再写 3 ，成 $[3, 5, 6]$ 。单项 $[7]$ 保留。
4. 顺次接桶，得 $[1, 2, 3, 5, 6, 7]$ 。没有再加一次样本 2 、 6 ，没有漏掉输入项。

要做多少事 要用多少地方

设 n 为项数、 b 为桶数。用平衡界限树找桶约每项比较 $\log_2 b$ 次，再移动一次；均衡后大桶不断缩小，良好随机采样通常给期望 $O(n \log n)$ 。这是一家族，最好取决于桶数、基础方法和数据，不能统一写一个固定最好界。天真版可能 $O(n^2)$ ，不缩小还可能不结束；检测失败并换有保证方法可给 $O(n \log n)$ 最坏。列表版额外 $O(n+b)$ 空间。输入顺序追加且桶内稳定才整体稳定。

为什么快或慢

一次分出多块，还可把不同桶分给不同工作者。但采样、界限比较、暂存都有成本；样本选偏会把大部分工作留在一个桶。

什么时候适合用

经常用于并行和分布式排序；在顺序学习中能看清分桶与桶内排序是两件事。

最后把容易弄错的地方说清楚

重复界限不能定义相互重叠的归属。全等桶要保留全部记录并直接结束。并行速度还要算工作者数量与传输，不能用这里的顺序工作量直接宣布实际秒数。

参考来源 [\[73\]](#) [\[74\]](#)

39 树排序与平衡树变体

类型 算法家族 别名或相关名称 二叉搜索树排序；BST sort；平衡树排序是其变体

先把要用的小动作讲清楚

树节点是存一条记录、以及“左边下一节点在哪里、右边下一节点在哪里”的小盒子。没有下一节点就记为空。第一盒叫根。

查找树约定：比当前节点小的放左分支，大的放右分支；这个约定对整个分支都成立，不只是紧邻孩子。

读树不能直接按创建盒子的次序。要先读左分支，再读当前盒，再读右分支；每个节点这样做，就按值从小到大输出。

旋转只改少量连接。例如右孩子提升到父上方时，旧父成它左孩子，提升者原来的左分支接到旧父右边；中间值仍在中间。

基本思路

把记录逐条安放到查找树中，每遇一个节点就判断该向左还是向右，直到找到空连接。树保存了小与大的关系，最后按“左、当前、右”读出有序结果。若树变成一根很长的链，查找会很慢；平衡版本会转动局部连接，避免长期这样。

具体执行步骤

1. 从空树开始，第一记录创建根。插下一条 x 时，从根比较。 x 小则走左连接，大则走右连接；连接为空时建新节点填上去。新节点不需要比较所有旧记录，只沿这一条路径。
2. 相等时不能覆盖旧节点。最容易稳定的做法在同键节点里追加原记录到队列，读节点时按队列到达次序全部输出；另一种是把原输入编号作为第二比较条件。
3. 读出时，若节点为空就返回。否则先完成左分支，再输出当前节点队列，再完成右分支。调用栈记住“做完左边之后还要回来读谁”。也可自己用列表记住这些节点，避免函数层层调用。
4. 普通树不改形状。AVL 版本给每节点存高度：空连接高度零，节点高度为一加左右较大高度。从新节点向根更新；某处左右高度差超过一，就检查较高一侧的孩子向哪边偏。
5. 若父和孩子向同一边偏，做一次旋转；若拐了弯，先把孩子向另一方向转，再转父。左旋具体是：父 p 的右孩子 x 上升， x 原左分支接 p 的右边， p 成 x 的左孩子，旧上层连接改指 x 。右旋对称，之后重新算受影响高度。
6. 树全部建好后再读出。旋转保持“左边较小、右边较大”的关系，所以输出规则无需改。红黑树、B 树、Treap 也可用这个建结构再顺序读取的框架，但平衡动作各不一样。

跟着数字一步步做

1. 插 $[4,1,3,2]$ ：4 是根。1<4，接到 4 左边。3<4 向左，到 1；3>1，接 1 右边。2<4 向左，2>1 向右，到 3；2<3，接 3 左边。四项都在树里，但创建顺序没有排好。
2. 从根 4 先走左节点 1。1 左边空，先输出 1；读其右节点 3，先读 3 左边的 2，输出 2；再输出 3；最后返回输出根 4。得到 $[1,2,3,4]$ 。
3. 另看平衡动作，插 $[1,2,3]$ 会成 1 的右孩子 2、2 的右孩子 3。AVL 发现根左右高度 0、2，相差二，且右孩子也向右偏。左旋 1：2 提升为根，1 接在它左边、3 留右边。
4. 新树先读 1、再 2、再 3，仍得到 $[1,2,3]$ 。旋转改变的是路径长短，没改变数值顺序。若 2 原有左分支，其值介于 1、2 之间，转到 1 的右边正好。

要做多少事 要用多少地方

设 n 为记录数、 h 为树高。一次插入比较路径上 h 个节点左右；树矮时约 $\log_2 n$ ，长链时可能 n 。普通不同键 BST 最好 $O(n \log n)$ ，随机插入期望 $O(n \log n)$ ，最坏 $O(n^2)$ 。AVL、红黑等保证整体 $O(n \log n)$ ，读出 $O(n)$ 。相等都存同节点时，全等可 $O(n)$ 。节点与队列额外 $O(n)$ ，读取等待空间 $O(h)$ ，普通树最坏 $O(n)$ ，平衡树 $O(\log n)$ 。队列或原编号规则正确则稳定。

为什么快或慢

树矮时一次比较排除一个大分支。顺序数据插入普通树反而容易长链，每次都走越来越远；平衡要多改连接，却给路径上限。

什么时候适合用

本来就需要反复按值查找、插入、顺序遍历的资料，树结构很有用。只为一次数组排序时，分配节点和间接访问可能不划算。

最后把容易弄错的地方说清楚

红黑树用颜色限制黑节点路径长度，通过换色和旋转控制高度；B 树一个节点容多个升序键，满节点分裂，中间键上移；Treap 用额外随机优先级，通过旋转让小优先级上升，平衡为概率保证。它们不是普通 BST 自动具有的性质。

参考来源 [75] [76]

40 希尔排序与增量变体

类型 算法 别名或相关名称 缩小增量排序；Shell sort

先把要用的小动作讲清楚

插入排序拿住下一项，向左找位置，把挡路的大项右移。本方法先把“一格”改成“隔 h 格”。

间隔 $h=4$ 时，位置 0、4、8...是一小排，1、5、9...是另一小排。它们交错占同一个数组，不必复制出来。

某一间隔的小排都排好，叫 h 有序；相邻项仍可能颠倒，不是全排完成。

最后 $h=1$ 时只有一小排，覆盖全数组，插入才保证全局升序。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

让远处的错位数先大步移动，再缩短步子，最后普通插入。先缓解长距离错位，可能让最后一格一格的插入省很多搬动。间隔选法决定速度，不能只给 Shellsort 一个通用复杂度。

具体执行步骤

1. 选递减正整数间隔，必须以 1 结束。Knuth 从 1 开始反复算 $3h+1$ ，得到 1、4、13、40...，取适合数组的那些倒用；大于等于数组长度的间隔没有需比较的小排。
2. 对某间隔 h ，扫描 i 从 h 到 $n-1$ 。拿住 $x=a[i]$ ，令空位 $j=i$ 。只要 $j \geq h$ 且 $a[j-h] > x$ ，就把前项搬到 j ，再让 j 减 h 。
3. 不能继续时把 x 写进 j 。它插到这一交错小排的正确位置，其他小排不受这次动作影响。按 i 顺序做完，就将所有交错小排分别插入排序。
4. 换较小 h ，重新扫描。新小排关系不同，仍要比较；旧轮的作用是减少远距离混乱。
5. $h=1$ 完成后结束。空、单项不用处理。每次拿住 x 必须写回； $j < h$ 时没有前方同排项，要停，不能读负编号。

跟着数字一步步做

1. [8,1,6,2,7,3,5,4] 用间隔 4、1。 $h=4$ ， $i=4$ 拿 7，比位置 0 的 8 小，移 8、写 7，得 [7,1,6,2,8,3,5,4]。 $i=5$ 的 3 在 1 后不移； $i=6$ 拿 5，比位置 2 的 6 小，移 6、写 5，得 [7,1,5,2,8,3,6,4]； $i=7$ 的 4 在 2 后不移。
2. 换 $h=1$ 。插 1、移 7，得 [1,7,5,2,8,3,6,4]。插 5、移 7，得 [1,5,7,2,8,3,6,4]。
3. 插 2、移 7 和 5，得 [1,2,5,7,8,3,6,4]。8 不动。插 3、移 8、7、5，得 [1,2,3,5,7,8,6,4]。
4. 插 6、移 8 和 7，得 [1,2,3,5,6,7,8,4]。插 4、移 8、7、6、5，得 [1,2,3,4,5,6,7,8]。第一轮后的 [7,1,...] 还没全局有序。

要做多少事 要用多少地方

设 n 为项数、 h 为间隔。每轮至少检查约 $n-h$ 项，还可能反复移项。Knuth $3h+1$ 最坏 $O(n^{3/2})$ ；有序输入仍扫约 $\log n$ 个间隔，最好约 $\Theta(n \log n)$ 。不假定普遍精确的随机公式。原始不断减半间隔最坏 $\Theta(n^2)$ 。Pratt 取小于 n 的全部 $2^p 3^q$ (p, q 为非负整数)，最坏 $O(n \log^2 n)$ ，轮数更多。额外 $O(1)$ 空间，不稳定。

为什么快或慢

单项先跨好几格，末尾的小数可更快到前面。但某些间隔将应比较的项一直分在不同小排，成本拖到最后才发生。

什么时候适合用

中小数组、操作要简单且不分配副本时有用。间隔选择要参考可靠结论或测量，不能认为越多越好。

最后把容易弄错的地方说清楚

Ciura 的经验间隔 1、4、10、23、57、132、301、701 有多种延伸策略，不能全用同一已证界。等值也能被间接越过： $[2a, 2b, 1, 3]$ 在 $h=2$ 后可成 $[1, 2b, 2a, 3]$ ，两个 2 的身份顺序改变。

参考来源 [77] [31] [78]

41 漏斗排序

类型 算法 别名或相关名称 Funnel sort；缓存无关排序的一种

先把要用的小动作讲清楚

高速缓存是处理器附近的小块快存储，普通内存更大却较慢。读一个数时，机器常搬来附近整块；连续读取更容易利用已经搬来的数。

两排有序数通过比较两条未用头接排：输出较小者，只前进供给它的头。一边空后，取另一边剩余。

缓冲队列暂存已排好的一批数，先进先出。连续数组环形实现用读头 head、写尾 tail、现有数 count：读后 head 加一且 count 减一，写后 tail 加一且 count 加一；到容量边界绕回 0，不能覆盖未读项。

上游小合并器生产一批，下游消费这一批。暂时队列空，不等于原输入真的结束。

公式只给批量命名： $\sqrt{k}$ 是平方根， $k^{3/2}=k \times \sqrt{k}$ 。四路时 $\sqrt{4}=2$ 、一批为 $4 \times 2=8$ ，中间队列容量为 $2 \times 8=16$ ；输入少于一批就只生成真实剩余记录。

基本思路

先排小段，再通过多层小接排器和中间队列接大段。小接排器集中反复处理局部数据，较容易留在缓存；下游需要更多时才叫它再产一批。“缓存无关”表示程序不用知道具体缓存容量，不是无视缓存。

具体执行步骤

1. 将 n 项切成约 $n^{1/3}$ 个连续段，各段约 $n^{2/3}$ 项，再分别照同样步骤排。为使教学边界具体，可在 $n \leq 2$ 时直接插入结束，其他取段数 $\text{ceil}(n^{1/3})$ ， ceil 表示向上取整；各段长度只差至多一。八项就分两条四项，四项分两条两项。插入时拿住下一项、移开更大前项、填空位。
2. 用 k 路合并器接 k 条有序输入。二路时直接比较两个未用头，输出较小者并前进。较大 k 分成约 $\sqrt{k}$ 组，每组一个 $\sqrt{k}$ 路小合并器输出到中间队列；末级 $\sqrt{k}$ 路再接这些队列。重复构造至常数路。
3. 原始构造的一次 k 路调用最多产约 k^2 项。各中间队列容量约 $2k^{3/2}$ ，上游 $\sqrt{k}$ 路一次产约 $k^{3/2}$ ，末级一次也至多消费这批量。运行末级前，检查队列；不到半满且源还有数，就让上游补一批。这种容量配合使一批消费不必逐项回头启动上游。
4. 上游本身也按同样规则调用更小上游，逐项写入队尾并更新 count。真正源结束时设结束标记；结束的源仍可能有队列余项，先读完才能退出。不能因一次队列空就断定源结束。
5. 末级比较可用队首，输出较小记录，推进它的 head、减 count。相等优先原输入较早那条流，各级必须相同。完成一批后再检查补货，直到产够请求的项数或所有输入结束。

6. 各级只转交原记录，最终输出不能写同一条两次。接出的有序排写回当前范围，再供给更大任务，直到全排完成。

跟着数字一步步做

1. 完整小例 [7,1,6,2,8,3,5,4]，八项顶层分成两条四项。前条拆 [7,1]、[6,2]，小段插入后为 [1,7]、[2,6]，比较两头接为 [1,2,6,7]。后条同样得到 [3,4,5,8]。
2. 最终比较 1/3 取 1，2/3 取 2，6/3 取 3，6/4 取 4，6/5 取 5，6/8 取 6，7/8 取 7，再取余项 8，得到 [1,2,3,4,5,6,7,8]。
3. 单看四路合并器，用同样八条记录作已排输入 [1,7]、[2,6]、[3,8]、[4,5]。第一个二路比较 1/2、7/2、7/6，生产 $Q1=[1,2,6,7]$ ；第二个二路生产 $Q2=[3,4,5,8]$ 。这是合并器示意，不是把八项的顶层拆分改成四段。
4. 下游先取 $Q1$ 的 1、2，head 从 0 到 2，剩 [6,7]；再取 $Q2$ 的 3、4、5，head 到 3，剩 [8]；再取 $Q1$ 的 6、7，最后取 8。本例全部装得下，不需要中途补货；更大输入才会消费一批、补一批。

要做多少事 要用多少地方

设 n 为项数。比较和转交仍有约 $\log n$ 层，通常、最坏 $O(n \log n)$ ；最好也只给此通用上界，基础版不承诺有序输入线性。输出常用 $O(n)$ 格， k 路全部缓冲 $O(k^2)$ 。理想模型缓存容 M 项、每次搬 B 项，采用原连续布局与批量规则且 $M=O(B^2)$ 时，缓存传输 $O(1+(n/B)(1+\log_M n))$ 。这是搬块次数，不是 CPU 比较数。各级等值先取较早流才稳定。

为什么快或慢

批量缓冲把小接排器工作集中在局部，减少数据跨较慢存储的往返。队列和层次管理有成本，小数据未必胜过普通归并。

什么时候适合用

大数据多层内存的搬运与缓存效率研究。先会二路接排和环形队列，再读多层结构。

最后把容易弄错的地方说清楚

M 、 B 是分析参数，不是程序需要的设置。缓存界还需要理想缓存和足够大缓存条件；队列和递归合并器结构按原构造连续布局，不能改成散乱节点还直接套界。实际磁盘外排还涉及设备和显式读写。

参考来源 [79] [80]

42 归并插入排序

类型 算法 别名或相关名称 Ford-Johnson 算法；不是普通归并与插入的任意混合

先把要用的小动作讲清楚

先比较两个数，就知道较小的应在较大的前面，这条事实可以留到后面用。把一对写成 (b,a) ， b 表示小的， a 表示大的；字母只是标签。例如 $(3,7)$ 说明 $3 \leq 7$ 。

把 3 插进已有序的一排、且知道它必须在搭档 7 前面时，搜索只需到 7 为止。二分插入每次看候选区间中间值，再舍弃一半不可能的空隙。找到空隙以后，数组里的后续数仍要右移。

为什么特别安排插入顺序？有 7 个数时，共有 8 个插入空隙，三次二选一比较可以区分 8 种可能；有 8 个数时有 9 个空隙，三次已不够。这个方法努力让待搜索范围停在 3、7、15……这些门槛以内。

基本思路

先花一次比较记住每对的大小关系，再排好大的那一半。每个小数只需在其搭档之前找位置。最后按特定次序插这些小数，尽量把搜索范围控制在比较次数较省的门槛内。

具体执行步骤

1. 总输入没有记录或只有一条时直接返回，不建立配对或主链。其余输入相邻两两配对，每对比较一次，记为小 b 、大 a 。只剩一个孤项时，给它编号 $b_{(m+1)}$ ， m 是对数；它也参加下面的待插入分组，只是没有搭档上限。

2. 用同一方法递归排序各大值 a 。移动一个 a 时，它配对的 b 信息也跟着走。大值排序后重新编号，使 $a_1 \leq a_2 \leq \dots$ ，同时每个 b_i 仍不大于自己的 a_i 。这个大成员子任务只有零条或一条 a 时无需再比较；这不表示整个排序结束，仍要建立主链并插入待处理 b 和孤项。
3. 建立主链 $[b_1, a_1, a_2, \dots]$ 。因为 $b_1 \leq a_1$ ，而 a_1 又是大值中最小的， b_1 可以无须新比较直接放第一。剩下 b 等待插入。
4. 把待插入编号按组倒序处理：先 b_3 、 b_2 ，再 b_5 、 b_4 ，再 b_{11} 、 b_{10} b_6 ，再 b_{21} b_{12} 。超出实际编号的项跳过。组边界从 1、3 开始，下一边界等于前一个加前一个的两倍，得 5、11、21.....。
5. 插入 b_i 时，查它搭档 a_i 当前在哪，只在 a_i 之前已有序的前缀二分查空隙。找到空隙后保留 b_i ，向右挪出空间，放入。搭档位置随插入而变，不能一直使用原来的下标。孤项搜索整链。
6. 待插入项用完后，主链有序且包含全部记录。重复值按一致规则比较；需要稳定时从头就按键和原序号比较。

跟着数字一步步做

1. 输入 $[7, 3, 6, 1, 8, 2, 5, 4]$ ，比较四对得到 $(3, 7)$ 、 $(1, 6)$ 、 $(2, 8)$ 、 $(4, 5)$ 。把大值 7、6、8、5 排成 5、6、7、8，搭档跟着成为 $(4, 5)$ 、 $(1, 6)$ 、 $(3, 7)$ 、 $(2, 8)$ 。
2. 主链 $[4, 5, 6, 7, 8]$ ，4 是 b_1 。编号 $b_2=1$ 、 $b_3=3$ 、 $b_4=2$ 。按组先插 b_3 ，搭档是 7，所以只查 $[4, 5, 6]$ 。
3. 查 3：中间 5 比 3 大，缩到 $[4]$ ；4 也比 3 大，插最左空隙。右移 4、5、6、7、8，链变 $[3, 4, 5, 6, 7, 8]$ 。
4. 插 $b_2=1$ ，搭档 6 之前是 $[3, 4, 5]$ 。先看 4，再看 3，都比 1 大，放最左，链为 $[1, 3, 4, 5, 6, 7, 8]$ 。
5. 最后 $b_4=2$ 的搭档是 8，只查它之前六项 $[1, 3, 4, 5, 6, 7]$ 。按中间值不断缩小，找到 1 与 3 之间；右移后放 2，得 $[1, 2, 3, 4, 5, 6, 7, 8]$ 。本组允许最多查七项，本例实际只查六项。

要做多少事 要用多少地方

它主要节省比较次数，尤其是 $n \log n$ 之外的较小项：比较量级 $\Theta(n \log n)$ ， n 是记录数，并非对所有 n 都最优。数组插入还要移动后面的记录，普通实现总时间最坏可达 $O(n^2)$ 。教学实现常用 $O(n)$ 搭档与链管理信息、 $O(\log n)$ 递归栈。普通版本不稳定。比较很贵而移动便宜时，省几次比较才可能值得这些管理动作。

为什么快或慢

搜索最多 $2^q - 1$ 项可用 q 次比较完成；分组顺序利用这个门槛。但成对关系与插入位置的管理成本较高。

什么时候适合用

比较特别昂贵而数据量不大的情境、最少比较次数研究。

最后把容易弄错的地方说清楚

搭档身份比某个固定下标重要。每次插入后重新知道 a_i 的当前位置，才能限制搜索。不能只排大值却丢了哪一个小值属于它。

参考来源 [81]

43 间接排序

类型 辅助技术 别名或相关名称 索引排序；指针排序；此处 tag sort 指附带索引

先把要用的小动作讲清楚

若一张记录很大，里面有照片、地址和很多字段，反复搬整条记录很贵。可以先给它贴一个小位置编号，只移动编号。编号 0 表示去原位置 0 读取记录，不表示键值是 0。

比较两个编号时，仍要去看它们指向的记录键。例如编号 0 指向 30，编号 1 指向 10；按键排序时，编号 1 应在编号 0 前。编号是一张小指路牌，记录本身可以不动。

基本思路

先排序小编号，得到一张“按什么顺序看原记录”的清单。只要按清单访问，就有一个有序视图。若真的要改变原记录排列，再按这张清单做一次循环搬运；这是一种省大记录移动的技术，仍要选底层排序方法。

具体执行步骤

1. 创建 $[0, 1, \dots, n-1]$ 编号数组， n 是记录数。每个编号唯一指向一条原记录。
2. 选择已经讲过的排序，例如归并。比较两个编号 i 、 j 时，读取原记录 i 与 j 的键再决定顺序；交换或移动的是编号，不是原记录。
3. 如果只需按序显示，依次用排好的编号读原记录，马上可以结束。原数据仍保留原位置，也能再建立另一张不同规则的清单。
4. 如果要真实重排，把排后清单理解成“目标位置 p 应放原位置 $\text{index}[p]$ 的记录”。准备已处理标记，逐个寻找没处理的位置 s ，先保存原 s 的整条记录，留下空位 $p=s$ 。
5. 读 $q=\text{index}[p]$ 。若 $q=s$ ，把暂存记录填到 p ，标记 p 已处理，然后结束这个循环；否则把 q 的记录搬到空位 p ，标记 p 已处理，让 $p=q$ ，继续读其原映射。两种分支都要标记刚填好的位置，包括关闭循环的最后一个位置。不要改掉尚需读取的映射。
6. 完成所有循环后，记录已经按清单排列。相等键如按原位置作第二比较条件，清单和最终排列都可稳定。

跟着数字一步步做

1. 原键 $[30, 10, 20]$ 放在编号 0、1、2。编号清单开始 $[0, 1, 2]$ ；按它们指向的键排序，得 $[1, 2, 0]$ 。
2. 按清单读取原位置 1、2、0，看到 10、20、30。到这里已经能按序展示，而且原记录还没动。
3. 现在要实移。目标位置 0 需要原位置 1 的 10。先保存位置 0 的 30，再把 10 从位置 1 搬到 0，空位转到 1。
4. 目标位置 1 需要原位置 2 的 20，搬过来，空位转到 2。清单说位置 2 需要原位置 0；那里原来的 30 已保存在手里，所以填入它。
5. 得到 $[10, 20, 30]$ ，循环 $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ 结束。标记三格都处理过，后面不重复搬这一圈。

要做多少事 要用多少地方

排序编号仍要识别键次序，不能因此变成通用 $O(n)$ 排序。常用底层比较排序上界 $O(n \log n)$ ，额外编号 $O(n)$ 。如果需要实移，再做 $O(n)$ 次大记录搬运，另需一条临时记录与标记；标记可在不破坏未读映射的前提下复用编号存储。省的是反复搬大记录，代价是比较时绕去读键，可能造成较零散的内存访问。最好、最坏和稳定性仍取决于底层方法。

为什么快或慢

小索引的交换比大记录移动便宜；缺点是按索引读键可能降低缓存命中。

什么时候适合用

每条记录很大、原数据需保持、或要同时建立多个排序视图的情况。

最后把容易弄错的地方说清楚

有序视图与原数组已经重排是两种结果，要先确定应用需要哪一种。 $\text{index}[p]$ 的方向必须讲清；把“目标到原位置”与“原位置到目标”混用会搬错。

参考来源 [82]

四 外存并行与受限输入的方法

本组不能与单核内存算法混排：外存按块读写轮次，CPU/GPU 按总工作与等待链，网络按比较器数与层数。先外存，后并行，再固定网络和特殊输入；这些是不同模型的分组，不是跨机器快慢名次。置换选择仅负责生成有序段。

44 外部多路归并排序

类型 算法家族 别名或相关名称 多路外排序；k 路归并外排序

先把要用的小动作讲清楚

先想象桌子只能摊开一小叠纸，整箱纸放在柜子里。内存像桌子，磁盘像柜子：桌上操作方便，来回拿柜里的纸较费时。外部排序是在这类条件下工作；“外部”说的是数据装不进内存。

文件里的一个有序段，就是一串已从小到大大写好的记录。归并时只看各段当前的第一条。为了少跑柜子，程序一次读一块记录，暂存在小内存区；这个区叫缓冲区。读完区里的记录，再拿下一块。输出也攒够一块再写。

段数多时，不必每次重新比较所有段头。最小堆把段头组织成一棵父不大于孩子的树。每个堆项同时记住“这个数来自哪一段”。取走根后，补进同一段的新段头，再按较小孩子向下调整；这样下一次根仍最小。

数组位置从 0 编号，位置 i 的孩子在 $2i+1$ 、 $2i+2$ ，只有下标小于当前堆项数才存在；除根外，父位置是 $\text{floor}((i-1)/2)$ ， floor 表示去掉小数。向上调整就是反复与父比较并交换，向下调整就是反复找存在的较小孩子、当前比它大才交换。

基本思路

一次排不了整箱，就先把能拿上桌的小批次分别排好，再合并这些小批次。每次输出各段头里最小的那张，输出后只推进它来自的那一段。数据一直往前读，不为找一个数反复翻整箱。

具体执行步骤

1. 先决定一次能处理多少记录。用 M 表示可用内存容量，用 B 表示一次磁盘读写包含的记录数。合并 F 段时，至少要给 F 个输入缓冲区和一个输出缓冲区留地方，还要留堆和管理信息；这些合计必须装入 M 。 F 至少为 2。
2. 按原文件顺序读入一批，用一个内存排序方法排好，写成有序段。重复到读完原文件。此时只是产生了多个有序段，整文件还没有全局有序。
3. 选不超过 F 个段，先读各段的一块。把每段头和它的段编号放进最小堆。建堆可以逐项加入：新项放末尾，若比父小就向上交换，直到关系正确。
4. 取堆根，把该记录放进输出缓冲区。只从它来源的段取下一条，替换堆根，再按较小孩子向下交换到关系正确。那段已经用尽时，用堆的最后一项补到根，堆项数减一，再按较小孩子向下调整；缩小后若只剩零项或一项，直接停止调整，不读不存在的孩子。
5. 输入缓冲区空了，才从同一段补读一块；输出区满了，才写一块。全部输入段耗尽时，把尚未满的最后输出块也写出。
6. 如果仍有多段，把较长的新段再次分批合并。每轮段数下降，最终只剩一个含全部记录的有序段。不要漏掉任何未满的一批。

跟着数字一步步做

1. 为了看清分段，先每批取三个数。原文件 $[7,2,5,8,1,4]$ 被分为 $[7,2,5]$ 和 $[8,1,4]$ ，分别排成 $[2,5,7]$ 与 $[1,4,8]$ 。真实批量还要按缓冲区预算决定。
2. 两个段头是 2、1。1 最小，输出区变 $[1]$ ，第二段向前一格，候选变 2、4。2 最小，输出 $[1,2]$ ，第一段向前，候选变 5、4。
3. 输出 4，候选变 5、8；输出 5，候选变 7、8；输出 7 后第一段空了。只剩第二段的 8，把它接上。最终 $[1,2,4,5,7,8]$ 。
4. 每次被拿走的都是各段剩余记录中最小的那个，因此不可能在后面再遇到一个比它小的数。这正是输出一条就能确定一条的理由。

要做多少事 要用多少地方

文件每经过一轮合并，就要完整读一遍、写一遍。每轮大约是 n/B 块读写， n 是总记录数。路数越大，段数下降越快，轮数通常越少。充分利用 M 的标准外存模型， $n > M$ 时读写界写作 $O((n/B) \log_{(M/B)}(n/B))$ ；它数的是块传输。生成段和维护段头的常规比较工作上界是 $O(n \log n)$ 。内存预算 $O(M)$ ，临时磁盘还要容纳新段。稳定性需要初始段排序和所有合并都保留相等记录的原先先后。

为什么快或慢

顺序读写和较大的合并路数减少整文件经过磁盘的次数。路数过大导致缓冲区太小或随机读写，反而变慢。

什么时候适合用

日志、数据库和超大文件；按磁盘吞吐与内存限制评估，不能直接和完全在内存中的快排比较。

最后把容易弄错的地方说清楚

空段不用放进堆。普通按原文件连续分批且按相邻批次合并时，相等先取较早批次即可保留先后；任意重排批次时，带上原序号更可靠。文件大小、可打开的流数量、最后半满的块，都要明确处理。

参考来源 [83] [84]

45 平衡多路外部归并

类型 重要变体 别名或相关名称 平衡 k 路归并；外部归并的轮次调度变体

先把要用的小动作讲清楚

上一种方法讲了怎样合并段。这里另一个问题是：旧段放在哪些文件里，新段写到哪里？把文件分成输入组和输出组，可以避免把尚未读完的旧记录覆盖掉。

一个文件可以连续装好几个段。必须另记每段的长度或结束标记；比如 $[1,7]$ 后接 $[2,8]$ ，整个文件 $[1,7,2,8]$ 并没有升序，不能误当一个四项段。

基本思路

把旧段尽量均匀放在一组文件中，逐批合并；新段轮流写到另一组文件。做完一轮后，两组交换职责。所谓平衡，说的是下一轮各输入文件有足够段参加合并，不是新的比较规则。

具体执行步骤

1. 选择可同时合并的段数 F ， F 至少为 2。准备 F 个输入流和 F 个输出流，记录各流的每个段边界，并分配能装入内存的输入和输出缓冲。
2. 从每个尚有段的输入流各取下一段。用比较段头、取较小头、只推进来源段的办法，把这一批合成一个长段； F 较大时用最小堆挑头。
3. 把第一批长段写到第一个输出流，第二批写到第二个，轮流写。每个长段都留下自己的边界，不和随后写入的段混成一个。
4. 旧输入全部消费完时，结束一轮。关闭或倒回读位置，让原输出组成为下一轮输入，原输入组成为可以重用的输出。
5. 继续直到全体只剩一个段。某批不足 F 段也照常合并；只有一段时直接作为最终结果，别无谓重复。

跟着数字一步步做

1. 用两路。文件 A 有 $[1,7]$ 、 $[2,8]$ 两段；B 有 $[3,6]$ 、 $[4,5]$ 两段。还准备空文件 C 和 D。
2. 第一批拿 A 的 $[1,7]$ 与 B 的 $[3,6]$ 。比头 1 和 3，输出 1；再比 7 和 3，输出 3；再比 7 和 6，输出 6；接上 7，得到 $[1,3,6,7]$ ，写到 C。
3. 第二批拿 $[2,8]$ 和 $[4,5]$ ，同样取头得到 $[2,4,5,8]$ ，这次写到 D。四个短段变成两个长段，且一边一个。
4. 下一轮读 C 和 D。依次取 1、2、3、4、5、6、7、8，形成一个段。初始四段，第一轮两段，第二轮一段。

要做多少事 要用多少地方

如果开始有 R 个段，一轮约把段数除以 F ，所以需要约 $\text{ceil}(\log_F R)$ 轮； ceil 表示不足一轮也算一整轮。每轮读写全部 n 条记录，约 $\Theta(n/B)$ 块传输， B 是每块记录数。常规总比较工作上界 $O(n \log n)$ 。缓冲和段头管理需按内存容量计算，量级 $O(FB)$ 加少量管理信息；临时磁盘 $O(n)$ ，常用 $2F$ 个工作文件。输入已排序也不自动省轮数，除非先识别了更长的初始段。

为什么快或慢

段数每轮大幅下降，而且不必每轮先重分配段。代价是要有两套流或文件；实际磁盘也可能不适合太多并发流。

什么时候适合用

随机访问文件系统、可提供足够工作文件的外排序；与多相归并比较时要固定可用流数量。

最后把容易弄错的地方说清楚

段数量平衡不意味着段字节数一样大。要稳定，初始段排序与平局选择必须都保留原先次序；如流分配改变了原段顺序，保留原序号并用它处理平局。

参考来源 [83]

46 多相归并排序

类型 重要变体 别名或相关名称 多相外部归并

先把要用的小动作讲清楚

一条流可以理解成只向前读或写的一排记录，磁带尤其像这样。输入流有尚未读完的段，输出流必须有安全的空地方。三条流也能归并：两条读、一条写；关键是段数要安排好，才能不断轮换。

下面给出可以照着实施的三流版本。若真实段数是 R ，从段数对 $(1,1)$ 开始，反复把 (a,b) 改成 $(a+b,a)$ ，直到 $a+b$ 至少为 R 。例如 $(1,1)$ 、 $(2,1)$ 、 $(3,2)$ 、 $(5,3)$ 。把 a 段放 A ， b 段放 B ， C 空。多出来的位置填空段：它只有一个段结束标记，不包含任何真实记录。这些数只安排段，不比较键。

基本思路

把“哪条读、哪条写”随着段的消耗不断轮换。有一条输入读完，就把它变成空输出，让刚产生的新段参加下一相。这种做法叫多相归并；相就是在同一套角色下连续合并的一段过程。

具体执行步骤

1. 先把原文件分批排成有序段。按上面的段数对，把段分配到 A 、 B ；不足处补空段。记录每段边界，并准备两输入、一输出的块缓冲。
2. 从两条输入各取一个段，合并到输出。两段头中小的先出，只推进来源段；空段贡献零条记录，另一段直接接上。
3. 继续一对一合并，直到一条输入所有段都用完。这时结束当前相，不把输出复制回原输入位置。
4. 耗尽的输入现在当输出；刚才的输出和仍有剩余段的旧输入当新输入。让它们各从下一未消费段开始读。
5. 不断换角色。最终只剩一个真实段，而且它含所有记录时结束。更多流要用对应的广义段数计划，不能直接照搬三流的数对。

跟着数字一步步做

1. A 有两段 $[2,7]$ 、 $[1,8]$ ， B 有一段 $[3,6]$ ， C 空。这个 $(2,1,0)$ 配置正好符合三流计划。
2. 第一相合并 $[2,7]$ 与 $[3,6]$ ：取 2，再取 3，再取 6，最后取 7，写 $C=[2,3,6,7]$ 。 A 还有 $[1,8]$ ； B 没有段了。
3. 第二相让 B 写。比较 A 的头 1 与 C 的头 2，先写 1；接着写 2、3、6、7，最后写 A 的 8。 B 得到 $[1,2,3,6,7,8]$ 。
4. 这时 A 与 C 都空， B 只有一个含全部六条记录的段，任务结束。每个原数只因归并移动，空段没有凭空制造数字。

要做多少事 要用多少地方

这种方法想省的是受限流设备上的搬文件与重新分配。每相读写多少，由哪些段相遇和段有多长决定，不能只凭“多相”断定比平衡归并快。常规比较工作上界 $O(n \log n)$ ， n 是总记录数。F 路输入通常需要 $F+1$ 条流，输入输出缓冲约 $O(FB)$ ， B 是每块记录数，另有段信息；临时磁盘 $O(n)$ 。本例 $F=2$ 。段数计划安排好时相数有规律，但不等长段与空段位置仍影响实际传输。

为什么快或慢

在流数量严格受限时，轮换文件角色减少专门的重分配。现代磁盘上的高路数普通归并未必需要这种复杂调度。

什么时候适合用

历史磁带外排序、少量顺序设备；学习外排序调度时有用。

最后把容易弄错的地方说清楚

不同相可能把原来不相邻的段放在一起。需要稳定时，给每条记录保留原位置；数字相等先出原位置较早的，并让初始段也使用这个规则。仅固定优先 A 或 B，不能保证原先次序。

参考来源 [85] [86]

47 置换选择生成有序段

类型 辅助阶段 别名或相关名称 替换选择；外排序的段生成阶段

先把要用的小动作讲清楚

有序段不一定只能和内存一样长。假设手里只能留三张纸，但拿走一张就能再读进一张。如果新读的数不小于刚输出的数，它仍可加入当前上升的段；如果更小，放在本段末尾会破坏顺序，就先留到下一段。

“冻结”没有把记录删掉，只是暂时不允许它参与本段的最小值选择。当前候选和冻结候选共享同一份 m 个记录的预算。可以给记录加当前/下一段标记，或把两区分开放在这份数组里。当前区用最小堆组织。

数组位置从 0 编号，位置 i 的孩子在 $2i+1$ 、 $2i+2$ ，只有下标小于当前堆项数才存在；除根外，父位置是 $\text{floor}((i-1)/2)$ ， floor 表示去掉小数。向上调整就是反复与父比较并交换，向下调整就是反复找存在的较小孩子、当前比它大才交换。

基本思路

持续输出当前允许的最小值，再用新读的记录补它的空位。符合当前顺序的记录继续参加，不符合的冻结。这样一边读、一边输出，一个段可能长于手里同时保存的记录数；所有段之后仍须归并。

具体执行步骤

1. 读入最前面至多 m 条，建当前最小堆。此段刚开始还没有上一输出，所以任何键都允许。冻结区开始为空。
2. 拿走堆根作为本段下一输出，记住它的键 last 。堆的末项补根，把当前堆项数减一，再向存在的较小孩子交换到父不大于孩子，保持下一次根最小。缩小后若只剩零项或一项，不再读孩子；冻结区的记录不计入这个当前堆项数。
3. 若源文件还有记录，读一条填进刚空出来的容量。键不小于 last ，就插入当前堆并向上调整；键小于 last ，就放冻结区。本段以后不会再考虑它。
4. 当前堆空时，给本段写结束标记。把冻结记录全改为当前记录，建新堆，清空冻结区的角色，从新段开始再取最小值。
5. 文件已读完时，不再补新记录，但仍把当前段排空，随后把冻结段也排空。最后合并这些段才得到全文件的顺序。

跟着数字一步步做

1. $m=3$ ，输入 $[4,1,3,2,6,0,5]$ 。初始候选是 4、1、3，根是 1。这里的候选列表写的是集合成员，不表示堆数组一定按大小排列。
2. 出 1，读 2： $2 \geq 1$ ，可以参加本段，当前有 2、3、4。出 2，读 6： $6 \geq 2$ ，当前有 3、4、6。

3. 出 3，读 0：0<3，冻结；当前仅 4、6，冻结有 0，两区合计仍三条。出 4，读 5：5≥4，当前有 5、6，冻结仍有 0。
4. 源文件结束，依次出 5、6。第一段是 [1,2,3,4,5,6]，现在当前堆空。把 0 解冻，新段 [0] 也写完。
5. 两段还不能直接拼接，因为 0 比第一段里的数小。把它们归并，才得到 [0,1,2,3,4,5,6]。

要做多少事 要用多少地方

n 是总记录数， m 是能同时保留的记录数， B 是一次磁盘块的记录容量。每条记录读一次、写入一个段一次；当前堆最多 m 条，每次进出沿约 $\log_2(m+1)$ 层调整，这个写法也包括 $m=1$ 的固定成本。段生成上界 $O(n \log(m+1))$ ，内存 $O(m)$ ，磁盘顺序读写 $\Theta(n/B)$ 块；最后不足一块仍算一次，严格可写 $\Theta(\text{ceil}(n/B))$ ， $n=0$ 时无需读写。最终归并的工作另算。若键独立随机且来自连续分布，常用分析给平均段长约 $2m$ ；这不是对每个输入的保证。已排序输入可以形成一整段，倒序常只有约 m 长。原序号可处理相等记录先后。

为什么快或慢

更长的初始段意味着后续可能少一轮整文件归并；堆维护比一次性内部排序更零散。

什么时候适合用

内存紧张、文件读写代价高的外排序。

最后把容易弄错的地方说清楚

相等的新数可以留在当前段，应使用“不小于”，不是“严格大于”。冻结区仍占容量。绝不能让当前堆和冻结区各自都增长到 m 条。

参考来源 [83]

48 外部分布排序

类型 算法家族 别名或相关名称 外部桶排序；外部范围分割排序

先把要用的小动作讲清楚

想把一箱数字分装成几箱：小于 4 的进第一箱，4 到 6 的进第二箱，7 及以上进第三箱。前一箱的每个数都比后一箱小，但箱内可能乱。这种按范围分装叫分布；在外存版本中，各箱是磁盘文件。

边界可以来自已知范围，也可以先抽少量样本：把样本排好，选若干间隔较开的值作分界。样本只是帮助估计各箱大小，不能保证所有箱均衡。每桶小缓冲区攒满才写磁盘。

基本思路

先让每个数去正确范围的文件，再分别整理这些文件。一个桶装得下就搬到内存排；装不下就再细分。全部桶内部有序后，按范围接起来就可以，因为跨桶大小已经由范围保证。

具体执行步骤

1. 选不重叠且覆盖全部键的范围，明确边界上的数属于哪桶。例如 $x<4$ 、 $4\leq x<7$ 、 $x\geq 7$ 。给各桶准备写缓冲，并确保缓冲总量装得下。
2. 完整读一遍输入，每条只放入一个对应桶。桶缓冲满时顺序写该桶文件，结束时也写最后半满部分；记录每桶总大小。
3. 桶能装入内存时，用一个完整的内部排序方法排好，再写出。不能只因它已分到一个桶，就称它有序。
4. 过大的桶再细分。必须检查有进展：若全部键相等，直接保留它们；若采样边界仍把所有记录送到一个桶，可用已找到的最小键建立等值桶，余下较大键另放，保证下一任务变小。
5. 等所有桶都处理完，从最小范围开始读其已排序记录，再接下一个范围。无需重新把不同范围的头互相比，因为范围顺序已经正确。

跟着数字一步步做

1. 输入 [7,2,5,8,1,4]，用 $x < 4$ 、 $4 \leq x < 7$ 、 $x \geq 7$ 。先读 7 进第三桶；2 进第一；5 进第二；8 进第三；1 进第一；4 进第二。
2. 桶得到 [2,1]、[5,4]、[7,8]。前两桶显然内部还没有排好。分别比较并交换，得到 [1,2]、[4,5]、[7,8]。
3. 先写 1、2，再写 4、5，最后写 7、8，输出 [1,2,4,5,7,8]。第一桶任一值都小于 4，第二桶任一值都小于 7，所以这次接起来不会产生跨桶倒序。
4. 若一桶里是 [5,5,5,5]，继续用相同边界拆它不会缩小。全等检测让它直接结束，也保留四条不同记录。

要做多少事 要用多少地方

一层分桶把全部记录读写一遍，约 $\Theta(n/B)$ 块传输， n 是记录数， B 是块容量。 F 个桶的缓冲量约 $O(FB)$ ，还要留管理空间。若桶划分足够均衡，所需层数少，可达到与标准外部归并同阶的读写界；按固定位分桶的版本则按位数算轮数。偏斜桶要反复读写很多次，严重时很慢。没有进展检查，朴素版本甚至可能永远重复同一桶。临时磁盘每层约 $O(n)$ ，及时回收旧层可降低峰值。稳定性需要保序分桶和稳定桶内排序。

为什么快或慢

平衡桶可以分别处理，适合并行。错误的边界和热门重复键会集中到同一桶，抵消好处。

什么时候适合用

外部整数或范围键排序、大数据分区；必须评估分布和桶大小。

最后把容易弄错的地方说清楚

相同键必须遵守同一个边界规则，不可在两桶随意拆开后乱接。抽样估计和实际桶大小是两件事；记录桶大小，再选择下一步。

参考来源 [83] [84]

49 并行归并排序

类型 重要变体 别名或相关名称 并行合并排序

先把要用的小动作讲清楚

并行就是几个工作者同时做互不妨碍的事。两个人分别整理桌子左右两堆纸，可以同时开始；若两人同时往同一个空位写数，记录就可能被覆盖。因此必须先划清各自负责的位置。

归并时，选定一个数 x 后，可以数出两排里总共有多少数应该在 x 前面。这个总数决定 x 的输出位置。只要两排已排序，就能用二分查找快速划出这些较小数；左边合并和右边合并会写到不同位置。

相等数的先后也影响切点。 x 来自原左排时，右排里等于 x 的记录应留在它后面； x 来自原右排时，原左排里等于 x 的记录应留在它前面。这样才能保持稳定。

基本思路

先让工作者同时排好不同小段，再把“合并两排”也拆给不同工作者。关键不是只多叫几个人，而是每一步明确谁写哪几个格子、谁必须等谁做完。

具体执行步骤

1. 把待排区间分成两半，交给两个排序任务。每个任务继续拆自己的区间；足够小时在一个工作者上用已讲过的稳定插入排序：拿住下一条，把前面严格比它大的记录逐条右移，再放进空位，相等记录不越过。等两半真的排好，再合并。稳定叶排序和下面的归并平局规则要同时满足，才能保持原先相等记录的先后。
2. 合并时，一排为空就复制另一排到指定输出区。一共只有少量记录时，直接比较段头做普通归并，避免创建过多小任务。
3. 否则在较长排中选中间记录 x 。用二分查找切另一排： x 来自左排时，找首个不小于 x 的位置；来自右排时，找首个大于 x 的位置。查找每次看中间值，排除不可能含边界的半边。

4. 把两排切点之前的记录数相加，就知道有多少记录应排在 x 前。将 x 写入这第一个尚未属于左子任务的位置。 x 已经确定，不再参加子任务。
5. 切点前的两小排交给左合并任务，切点后的两小排交给右合并任务。给它们不同输出范围，并行完成。
6. 两个子任务都结束后，整个输出区才算完成。父任务等待后再交给上层使用，不能提前读取尚未写好的格子。

跟着数字一步步做

1. 输入 $[7, 2, 5, 8, 1, 4]$ 。两个排序任务分别得到左排 $[2, 5, 7]$ 、右排 $[1, 4, 8]$ 。内部也可以继续拆小任务，本例直接展示排好之后。
2. 选左排中间的 5。右排中间是 4， $4 < 5$ ，所以切点在它右边；再看 8， $8 \geq 5$ ，所以切在 8 之前，右排分成 $[1, 4]$ 和 $[8]$ 。
3. 左排在 5 之前有一个 2，右排在切点前有两个数 1、4。共三条应在 5 前，因此 5 放第 4 格。
4. 左任务只写第 1 至 3 格，归并 $[2]$ 与 $[1, 4]$ ：比 2 与 1，取 1；比 2 与 4，取 2；接 4，得 $[1, 2, 4]$ 。
5. 右任务只写第 5 至 6 格，归并 $[7]$ 与 $[8]$ 得 $[7, 8]$ 。它不用等左任务每次比较，因为位置不重叠。等两边结束，整排是 $[1, 2, 4, 5, 7, 8]$ 。

要做多少事 要用多少地方

n 是待排记录总数。几个工作者合起来做的全部动作叫工作量 W ；其中必须按先后完成、无法同时做的最长一条链叫跨度 S 。这种并行排序的工作上界 $O(n \log n)$ 。本条二分递归合并的跨度 $O(\log^2 n)$ ，接上约 $\log n$ 层排序任务，得到 $O(\log^3 n)$ 排序跨度。若最后仍一个人合并整排，跨度会到 $\Theta(n)$ ，其他人只能等。常用 $O(n)$ 输出数组，并另算任务栈。 p 个工作者的时间至少受 W/p 和 S 两者限制，还有调度与传输开销。所述平局规则可保持稳定。

为什么快或慢

足够大的数组可以分摊比较和移动；任务创建、同步和内存带宽会限制收益。

什么时候适合用

多核上的大数组、需要稳定性的排序。

最后把容易弄错的地方说清楚

每个任务要同时知道输入区间和输出区间。只写“左右同时归并”而没有位置分工，不能保证安全。零项、一项和已经空掉的分支应立即停止。

参考来源 [87]

50 并行快速排序

类型 重要变体 别名或相关名称 并行快排

先把要用的小动作讲清楚

枢轴只是选来作参照的一个键。例如选 5，可以把每个数分到“小于 5、等于 5、大于 5”三组。小组里面还可能乱，分组只保证跨组方向正确。

几个人一起分组时，先数各人要写多少，再分空位。假如第一人有一个小数，第二人有 2 个小数，第二人就从小数区第 2 格开始写。把前面人数逐个加起来得到偏移，这个动作叫前缀和。它只是在算每人从哪格开始。

基本思路

同时整理枢轴两边，也让分组阶段分工。每人先判断自己的记录属于哪组，再按预先分配的空位写出。三组准备好后，小组和大组可以独立继续排序，等值组已经不需要按键大小再排。

具体执行步骤

1. 选一个输入键作枢轴，把连续输入小片分给工作者。各人扫描自己的片，只统计三类数量；这次统计不丢掉原记录。
2. 相加得到三组总长度，分配连续的三个输出区域。对每一类，再按原输入片先后加各人的数量，给每人一个独占起点。
3. 各人重读自己的片，按原片内顺序把记录写到本类下一个空位。写一条后只推进自己的该类位置；不和别人争同一个格子。
4. 等待分组写完。把小于区、大于区分别交给排序任务；它们再次分组，直到区间只剩零项、一项，或小到改用完整串行排序。
5. 保持等于区，直接接在小于区之后和大于区之前。所有子任务完成后，整个父区间有序。若分组长期极偏斜，可用递归深度限额改用有保障的排序，避免一直重复大任务。

跟着数字一步步做

1. 输入 [7,2,5,8,1,4]，选 5。第一人读 [7,2,5]，数出小 1、等 1、大 1；第二人读 [8,1,4]，数出小 2、等 0、大 1。
2. 合起来小数区长 3，等值区长 1，大数区长 2。小数区第 1 格给第一人的 2，第 2、3 格给第二人的 1、4；等值格写 5。
3. 大数区第一人的 7 先写，第二人的 8 后写。得到 [2,1,4] | [5] | [7,8]。位置全分好了，没有一条被覆盖。
4. 小任务再选参照分组，排成 [1,2,4]；大任务排成 [7,8]。接在固定中间的 5 两侧，得 [1,2,4,5,7,8]。分组完成那一刻，[2,1,4] 还没有内部有序。

要做多少事 要用多少地方

每层扫描、统计、分位置和写出大约处理所有记录一遍。枢轴把任务分得较均衡时，层数约 $\log n$ ，总工作期望 $O(n \log n)$ 。这里“期望”需要良好随机枢轴假设。极差枢轴若总只去掉一条，可能累积 $O(n^2)$ 工作。并行扫描和前缀和的跨度 $O(\log n)$ ，平衡排序跨度 $O(\log^2 n)$ ；如果分组仍串行，跨度可到 $\Theta(n)$ 。本缓冲做法要 $O(n)$ 额外记录数组。按原片顺序分偏移、片内保序写入、叶子排序稳定时，可保持稳定；原地交换版本通常不稳定。

为什么快或慢

好的枢轴提供大小相近的任务；极不平衡分区会让一个核心承担几乎全部工作。

什么时候适合用

大规模内存数据；应与同硬件、同线程数和同分区策略的算法比较。

最后把容易弄错的地方说清楚

工作者多并不会自动修复坏枢轴。等值组能免去大量重复键的再次分组。必须等待分组写完，才让子任务读它们的输入。

参考来源 [87]

51 并行采样排序与 IPS4o

类型 算法家族 别名或相关名称 原地并行超标量采样排序

先把要用的小动作讲清楚

一把分成两边是快排；用多个分界值分成许多桶，是采样排序的基本想法。分界值可以先从一小份样本里挑：排好样本，再选间隔较开的几个键。挑得好，各桶大小近似；抽样不保证每次都如此。

IPS4o 是这个家族里的一个具体实现。为了少搬零碎记录，它先把同桶记录攒成整块。整块就是固定 b 条的一小排，块内还没有排序；只是知道全块属于哪桶。一个工作者为每桶留一个小缓冲，满了才搬。

在原数组中搬块时，不能直接覆盖未保存的块。先拿住一个块，留下空位；把应到该空位的块搬来，继续沿空位走，最后把拿住的块放回。这个循环搬运和暂存一张纸的道理相同。

基本思路

让很多工作者按多个范围分组，再分别排各桶。普通实现可写到另一份数组；IPS4o 尽量把满块在原数组里重新安排，用小缓冲和块的去向信息保护尚未处理的记录。

具体执行步骤

1. 抽样、排序样本、选分割键。把分割键组织成判断树：先比中间分界，知道属于哪半边，再比该半边中间分界，直到找到一个桶。对等于分界的键使用固定规则或独立等值桶。
2. 各工作者读自己的输入片，把每条放本桶小缓冲。满缓冲写成同桶整块，只写到已读过、不会毁掉未读记录的位置；末尾不足一块的记录暂留缓冲。块属于哪个桶，可以拿它的第一条记录再次通过分界判断得到，因为块内所有记录属于同桶；无需给输出记录添一个永久桶号。
3. 加各工作者桶计数，算各桶的目标起止。和前缀和一样，第一桶从头开始，第二桶从第一桶总数之后开始，依次算下去。
4. 根据重新判断得到的块桶号，把满块安排到其目标桶区。目标位置已有别的块时，交换到暂存块或以空位接续循环；只有保存好旧块，才能写新块。并行搬运还要给每块位置分配唯一工作者。
5. 清理桶边界上未满块和各工作者剩余缓冲，按逐条分类把这些记录填进对应桶还空的目标位置。实际 IPS4o 为边界碎片保留专门信息，不能跳过这一步。
6. 桶都准备好后，分配子排序任务。每桶重复采样分组或用小数组排序。桶范围有序且每桶内部有序，整数组才完成。

跟着数字一步步做

1. 用 $[7, 2, 5, 8, 1, 4]$ ，分界为 3、6。左工作者局部三桶为 $[2]$ 、 $[5]$ 、 $[7]$ ；右工作者为 $[1]$ 、 $[4]$ 、 $[8]$ 。
2. 总计每桶两条，所以目标区是第 1–2 格、第 3–4 格、第 5–6 格。按类整理得到 $[2, 1] \mid [5, 4] \mid [7, 8]$ ，分别排后成为 $[1, 2] \mid [4, 5] \mid [7, 8]$ 。
3. 另看一个满块搬运状态：块大小为 2，假设分类后已有 $[8, 7] \mid [2, 1] \mid [5, 4]$ ，三块依次属于大、小、中桶，目标却应是小、中、大。
4. 先拿住 $[8, 7]$ ，第一块位置空。把 $[2, 1]$ 搬到第一块位置，第二块空；把 $[5, 4]$ 搬到第二块位置，第三块空；把拿住的 $[8, 7]$ 放到第三块。得到 $[2, 1] \mid [5, 4] \mid [8, 7]$ 。
5. 这个满块例没有边界碎片，专门展示如何不丢记录地搬块。真实数组通常还有部分块，必须按第 5 步填齐，再排桶内。

要做多少事 要用多少地方

需要做的是采样、判断桶、搬块、清理边界和继续排桶。作者在随机采样与实现假设下证明以高概率 $O(n \log n)$ 总工作； n 是记录数，“以高概率”不是每个随机结果都保证。 p 个工作者、 k 个桶、每缓冲 b 条时，小缓冲量级约 $O(pk b)$ ，还要块去向等管理信息。所谓原地表示尽量避免第二份完整记录数组，不表示零额外内存。速度取决于桶均衡、线程数和内存带宽，通常不稳定。

为什么快或慢

批量搬块减少零碎移动，较少难预测的分支有利于现代处理器；小数组可能不够抵消采样和调度成本。

什么时候适合用

多核大数组，尤其无需稳定排序时；性能来自算法和工程实现共同作用。

最后把容易弄错的地方说清楚

上面的步骤讲清分类、整块循环和边界清理三个必要机制。真正并行实现还必须用正确的同步规则分配块位置；任意两个工作者同时覆盖同一块会出错。普通 samplesort 与 IPS4o 是家族和具体实现的关系，不是别名。

参考来源 [74] [88]

52 GPU 基数排序

类型 重要变体 别名或相关名称 图形处理器基数排序

先把要用的小动作讲清楚

GPU 可以看作擅长让许多小工作者一起处理规则相似的动作。它并非让每一步自动变快：先把数据送到它能读取的地方、启动任务，结束后必要时再取回结果，都要花时间。

按个位分组时，21 和 11 都进“个位 1”的组。稳定分组要求先出现的 21 仍在 11 前。这很重要，因为下一轮十位分组要保留上一轮已经获得的个位先后。

多人写桶时，要把“桶从哪儿开始”“这人之前的人已经写几条”“这条在本人的同桶记录中排第几”三个数加起来。相加后的格子唯一，才能避免两个人同时写一格。

基本思路

每一轮都让许多工作者读当前数位、统计数量、分好写入位置，再把记录搬到对应桶区。从低位到高位重复，且每一轮保持相同数位记录的原先先后。最终各数位的顺序连起来，就是全数大小顺序。

具体执行步骤

1. 先选键编码和每轮处理的位。编码的大小顺序必须和要排序的键一致；下面用十进制展示，实际 GPU 常用几位二进制。负数和浮点数不能未经处理就把原比特当无符号正数。
2. 把连续输入片分给线程块。每块数当前各数位出现几次，得到一张计数表；这也叫直方图，含义只是每类的数量。
3. 合计各块计数，从小数位到大数位累计，得到各桶起点。再按输入块先后累计本桶数量，给每块分一个不重叠的桶内区间。
4. 块内按原记录顺序计算同数位记录的第几个，再写到桶起点加块偏移加局部名次的位置。所有记录唯一写出，相关姓名等字段跟键一起走。
5. 这一轮完成后，新输出作为下一轮输入，继续更高位。全部位处理完才结束；需要 CPU 使用结果时再送回。

跟着数字一步步做

1. 输入 [21,12,11,22]。第一块读 [21,12]，第二块读 [11,22]。看个位：两块各有一个 1、一个 2。
2. 总共有两个个位 1，因此桶 1 从第 1 格开始，桶 2 从第 3 格开始。第一块在两个桶里各先占一格，第二块各在它之后。
3. 21 写第 1 格，12 写第 3 格；11 写第 2 格，22 写第 4 格。结果 [21,11,12,22]。个位相同的 21、11 没有颠倒，12、22 也没颠倒。
4. 再看十位：11、12 属于 1 桶，21、22 属于 2 桶。仍按本轮输入顺序写出，得到 [11,12,21,22]。在十位相等的每组里，先前的个位顺序保留下来，因而整数顺序正确。

步骤状态图 C 比较 M 移动 S 已有序

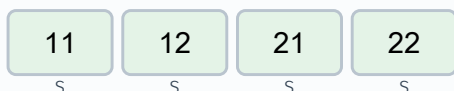
原输入



稳定个位分配



稳定十位分配



要做多少事 要用多少地方

每轮要读记录、做计数、累计偏移、再写记录。 d 轮、每轮 k 桶、 h 个线程块时，总工作上界 $O(d(n+h\cdot k))$ ； n 是记录数。只有固定配置中 $h\cdot k$ 可被 $O(n)$ 吸收时，才常简写为 $O(d(n+k))$ 。常见额外空间 $O(n+h\cdot k)$ ，包含另一份记录数组和块计数。每轮稳定的 LSD 版本可稳定。时间还受带宽、启动和 CPU/GPU 传输限制，小例通常不值得启动 GPU。只有键宽固定时才能把 d 当常数。

为什么快或慢

GPU 能批量做计数、扫描与写入；数据若本来在显存中，收益通常更容易体现。很小的输入或往返传输可能拖慢整体。

什么时候适合用

GPU 中的大批整数或可编码键；必须固定键宽和包含传输的计时范围。

最后把容易弄错的地方说清楚

没有局部名次或块偏移，两个同数位记录会争同一位置。浮点特殊值、负号和需要的次序规则必须先选清楚。本文解释稳定数位扫描机制，完整设备接口与资源选择见官方实现。

参考来源 [89] [90]

53 双调排序网络

类型 算法 别名或相关名称 Bitonic sort；双调排序；有时泛称 Batcher sort

先把要用的小动作讲清楚

比较器是一个很小的动作：接收两个位置，比较那里的数，把较小的放指定的小端，较大的放另一端。数字相等可以不动。网络把这些比较器的位置事先画好，输入数值不改变下一步去哪儿。

若一层的比较器不共用位置，就能同时执行。例如比较第 1、2 项和第 3、4 项互不妨碍。下一层可能用上一层交换后的结果，所以层与层之间要按顺序。

双调序列可以先一路升、再一路降，例如 $[1, 3, 4, 2]$ 。也允许把这个形状首尾循环移动。它比任意乱序更有规律：把相隔半段的数两两比较，可以把较小半边与较大半边分开，再继续处理两半。下面先从长度为 2、4 的具体做法看。

基本思路

先制造一个“先上升后下降”的形状，再用固定位置的比较把它变成完全上升。升序与降序只是同一个比较动作选择哪端留小值。反复处理更小的半段，最后每个位置正确。

具体执行步骤

1. 把长度补到 2、4、8、16……这类反复翻倍的数。补的不是实际记录，而是比全部真实键更大的特殊占位项，最后删除它们。单项直接结束。
2. 对当前区间，前半用本算法排成升序，后半排成降序。两半都完成后，把它们连起来就是待处理的双调形状。降序任务内部也按同样拆法，最后选择降序合并。
3. 要升序合并长度 n 的双调区间，令距离为 $n/2$ 。第 1 项与第 $1+n/2$ 项比，第 2 项与第 $2+n/2$ 项比，依次完成半数对。较小值放前半，较大值放后半。
4. 再对前半、后半分别做升序双调合并。合并只是继续这个“隔半段比较、再缩半”的动作，不重新制造升降序。长度为 2 时一次比较就结束，长度为 1 时不做事。
5. 降序合并完全对应，把较大值留前半。固定网络中可同时执行不重叠的比较；每层结束，再用新值进入下一层。

跟着数字一步步做

1. 输入 [3,1,4,2]。把前两项升序： $3>1$ ，交换成 [1,3]。把后两项降序： $4\geq 2$ ，保持 [4,2]。于是得到 [1,3,4,2]，先升再降。
2. 升序合并四项时距离是 2。比第 1 的 1 与第 3 的 4，小值 1 留前；比第 2 的 3 与第 4 的 2，交换。整排变 [1,2,4,3]。
3. 前半 [1,2] 与后半 [4,3] 各做长度 2 的升序合并：前半不动，后半交换，得 [1,2,3,4]。
4. 共做六次比较。哪怕某次不用交换，比较器仍经过，因为路线只由长度和方向确定，和数值是否恰好有序无关。

步骤状态图 C 比较 M 移动 S 已有序

构造双调序列



距离 2 比较后



两半内部比较后



要做多少事 要用多少地方

固定路线要走同样多次，所以已排序与很乱的输入都做同样比较工作。 $n=2^q$ 表示长度由 1 翻倍 q 次得到：共有 $q(q+1)/2$ 层，每层 $n/2$ 次比较，总工作 $\Theta(n \log^2 n)$ ，并行层数 $\Theta(\log^2 n)$ 。串行迭代执行可以直接交换原数组并用 $O(1)$ 管理空间；递归写法另需 $O(\log n)$ 栈，真实硬件还要连线和比较器。通常不稳定，远距离交换可能让两个相等记录越过彼此。

为什么快或慢

固定比较路线适合并行和硬件；单核上通常比优秀 $O(n \log n)$ 方法做更多比较。

什么时候适合用

GPU 小块、硬件电路、固定长度安全计算；不要把并行层数当串行工作量。

最后把容易弄错的地方说清楚

只有已经形成双调形状，隔半段合并的保证才成立。不能把这个合并动作直接施加于任意乱数组就宣布完成。Batcher 也设计了其他网络，因此这个别名不唯一。

参考来源 [91]

54 奇偶归并排序网络

类型 算法 别名或相关名称 Batcher 奇偶归并；不同于奇偶换位排序

先把要用的小动作讲清楚

这里“奇数、偶数”说的是第几个位置。数值 4 可以在第 1 个位置，不能把它当作“偶数位置的记录”。每个半段已经有序时，它的第 1、3、5……项也有序，第 2、4、6……项也有序。

因此可以把两半的第一类位置合并成一条，第二类位置合并成另一条，再交错放回。这两条各自升序，但相互交错时中间仍可能出现倒序；补做特定相邻比较就把这些缝接好。

基本思路

先排两个半段，再让交替位置分别合并，最后处理两条交错结果之间的缝。比较位置全都由长度预先决定，属于排序网络。它与反复扫相邻奇偶对的奇偶换位排序是不同方法。

具体执行步骤

1. 长度用 2 的幂，非这种长度补大占位项，最后删掉。递归把前半和后半都升序排序；单项不做事。
2. 现在合并两段。若总共只有两个数，比较一次就结束。否则挑两段各自的第 1、3、5……位置，按本合并规则递归合并这两条较短的有序序列。
3. 同样挑两段各自的第 2、4、6……位置，再做一次合并。这里挑位置时只是说明哪些位置参与；实现可用起点和步幅记录路线，不必真的复制两条数组。
4. 把第一路结果依次留在整个区间第 1、3、5……格，第二路留在第 2、4、6……格。再比较整个区间的第 2 与 3、第 4 与 5，依次到倒数第二与倒数第一，左大右小时交换。
5. 回到上层继续同样的合并。每次参与的序列变短，最终全部由两项比较组成，所以可以预先列出完整比较器路线。

跟着数字一步步做

1. 输入 [3,1,4,2]。先排两个半段：3 和 1 换，4 和 2 换，得到 [1,3] | [2,4]。
2. 第一路是两半的第一个数 1、2，已经方向正确；把 1 留整个第 1 格、2 留第 3 格。第二路是 3、4，比较后把 3 留第 2 格、4 留第 4 格。整体是 [1,3,2,4]。
3. 交错时中间 3、2 仍倒着。最后比较第 2、3 格，交换，得 [1,2,3,4]。
4. 完整四项路线从 1 编号为 (1,2)、(3,4)、(1,3)、(2,4)、(2,3)，共五次比较。前两对可一起做，中间两对可一起做，最后一对单独做。

要做多少事 要用多少地方

每次排序都按同一固定路线，总比较工作 $\Theta(n \log^2 n)$ ，并行层数 $\Theta(\log^2 n)$ ， n 为补齐后的长度。可直接在数组位置交换，不另存记录副本；用递归产生路线需要 $O(\log n)$ 栈，若预存所有比较器则需 $\Theta(n \log^2 n)$ 路线空间。通常不稳定。与双调网络相比，某些规模能少一些比较器，但实际速度还依硬件连接和执行方式。

为什么快或慢

与同规模双调网络相比常可节省比较器；线路、同步和访问模式决定硬件上的实际表现。

什么时候适合用

排序电路、固定大小并行块和需要固定访问模式的场景。

最后把容易弄错的地方说清楚

补位项必须与真实记录区分，排完只删补位项，不能删最大真实数。没有共用位置的同层比较才能同时做。这个四项结构也会在小网络设计条目里出现，那里明确不把它重复算作新算法。

参考来源 [92] [91]

55 固定小规模排序网络

类型 辅助技术 别名或相关名称 小数组比较交换网络；本条采用四项五比较器网络

先把要用的小动作讲清楚

固定小网络可以想成四条固定的轨道，每条轨道此刻放一个数。比较器连两个位置，小值去左边、大值去右边。它不根据输入选新路线，四项输入永远走相同五次比较。

两条连接线若不共用位置，就能同时做。例如第 1、2 位与第 3、4 位分别比较。这里“一层”指可以一起发生的一批比较，不是五次比较都能同时发生。

基本思路

四项数据可以用一条特别短的固定路线：先排两对，再让两端拿到最小和最大，最后排中间两个。它是小网络的设计例子；与四项奇偶归并网络同一个结构，不另算一种全新大数组算法。

具体执行步骤

1. 第一层比较第 1、2 位，左大则换；同时比较第 3、4 位。现在两对各自有序。
2. 第二层比较第 1、3 位：那里是两对较小值，小的留第 1 位。比较第 2、4 位：那里是两对较大值，大的留第 4 位。两次互不共用位置，可以同时做。
3. 此时第 1 已是全体最小，第 4 已是全体最大。中间两个可能相反，所以第三层只比较第 2、3 位，左大则换。
4. 完成这五次比较就结束，不根据“上一层没换”提前跳过后续。长度不是四项时，换成经过验证的对应网络，不能一直重复这五条线。

跟着数字一步步做

1. 输入 [4,2,3,1]。第一层 4、2 换，3、1 换，成 [2,4,1,3]。
2. 第二层第 1、3 位的 2、1 换；第 2、4 位的 4、3 换，成 [1,3,2,4]。现在两端 1、4 已在最终位置。
3. 第三层交换中间 3、2，得到 [1,2,3,4]。每个数字都经过已有位置的交换，没有创建或删除记录。
4. 可以完整检查只含 0、1 的四项输入，共 $2 \times 2 \times 2 \times 2 = 16$ 个。若它们都排对，这个比较网络能排任意全序键；这叫零一原理。直观理由是：假设数值结果有一对倒序，就在两值之间画一道门槛，大值记 1、小值记 0，会让同一比较路线也留下 1 在 0 前，和二值测试通过矛盾。

步骤状态图 C 比较 M 移动 S 已有序

输入



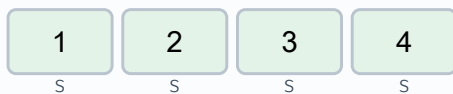
第一层两两排序



第二层两端确定



第三层排中间



要做多少事 要用多少地方

固定四项永远五次比较、三层，记录交换使用 $O(1)$ 临时空间。四项是固定规模，不能据此推导一般 n 的增长界。其他小网络要分别算比较器数与依赖层数。没有递归和动态枢轴选择，适合大算法的微型基本情况。它通常不稳定：相等值不直接互换，也可能经不同轨道绕过彼此。

为什么快或慢

没有递归、枢轴或动态路线选择；对小块可减少管理成本。规模变大时网络比较器数量与生成难度上升。

什么时候适合用

大排序的微型基础情况、向量指令、硬件小块排序。

最后把容易弄错的地方说清楚

例子 $[2a, 3, 2b, 1]$ 会先变 $[2a, 3, 1, 2b]$ ，再变 $[1, 2b, 2a, 3]$ ，中间两个等值不交换，结果仍把 $2b$ 放到了 $2a$ 前。这说明稳定性不能只看“相等时不换”。

参考来源 [91]

56 剪切排序

类型 算法 别名或相关名称 蛇形网格排序

先把要用的小动作讲清楚

把四个数放成两行两列，而不是一长排。每一行、每一列都有自己要整理的相邻关系。蛇形读取指第一行从左到右，第二行从右到左，第三行又从左到右，像一条在行间折回的路线。

若第二行在画面里是 $[4, 3]$ ，沿蛇形从右往左读到的是 3、4。因此偶数行要在画面里降序，才能在读取方向上排序。只排好每行还不够，因为上一行末尾可能比下一行开头大；列排序帮助把小数向上、大数向下。

基本思路

轮流整理行与列，让无序区域逐渐缩小。每次行排序按蛇形方向，列排序从上往下。完成足够轮次后，沿蛇形读，才得到从小到大的输出。

具体执行步骤

1. 先限定正方形网格：每边 s 个位置，共 $n=s^2$ 条记录。把输入按行放入网格，保留每个位置身份。

2. 把第 1、3、5.....行从左到右升序排好，把第 2、4、6.....行从左到右降序排好。用明确的相邻比较轮次：第一相位同时比较第(1,2)、(3,4).....个位置，下一相位同时比较第(2,3)、(4,5).....个位置，再交替下去，共做 s 个相位。升序时左大才换，降序时左小才换；同相位的对子不共用位置，才能同时操作。各行也能同时做。
3. 每列都从上到下升序排好：把行步骤的 s 个相位改成列中从上到下编号的位置，上大下小才换。每列内部同相位不重叠的对子同时做，各列之间也能同时做。列排序可能打乱上一轮某行的蛇形先后，因此继续下一轮行排序。
4. 保守地执行 $\text{ceil}(\log_2 s) + 1$ 个行列轮次，再做一次蛇形方向的行排序。这里 ceil 是向上取整，例如 $s=3$ 时 $\log_2 3$ 介于 1 和 2，取 2。
5. 最后沿蛇形输出，不要所有行都从左向右读。非方形版本需要另外的轮次与边界规则，本条只使用正方形。

跟着数字一步步做

1. 起始网格上行 [4,1]，下行 [3,2]。第一行要升序，交换 4、1 成 [1,4]；第二行要降序，3、2 已满足。
2. 现在两列分别 [1,3] 和 [4,2]。左列不动，右列交换，网格变上行 [1,2]、下行 [3,4]。
3. 再整理行。上行已升序，下行应降序，所以把 3、4 换成 [4,3]。网格现在 [1,2] / [4,3]。
4. 两列 [1,4]、[2,3] 已从上往下升序，后续保守轮次保持这个网格。蛇形读取先 1、2，再从下行右边读 3、4，得到 [1,2,3,4]。
5. 若直接所有行左到右读，会得到 [1,2,4,3]。数据没有坏，是读取路线错了。

要做多少事 要用多少地方

在 $s \times s$ 最近邻处理器网格上，一行或一列用奇偶换位排序约需 $O(s)$ 并行时间， $O(\log s)$ 个轮次共 $O(s \log s) = O(\sqrt{n} \log n)$ 时间。所有工作者加起来做的比较工作为 $O(n^{3/2} \log n)$ 。这些条件说的是网格硬件；普通单核执行全部行列工作，不能把并行时间当成自己的时间。可以直接交换网格记录，通常不稳定。更换行列排序方法，成本也要改算。

为什么快或慢

利用行列并行和局部通信；方形网格结构与同步开销决定价值。在普通单核数组上会有大量重复工作。

什么时候适合用

网格处理器研究、矩阵式并行排序教学。

最后把容易弄错的地方说清楚

每次都必须把整行或整列排完，不是每行只比较一对就算一轮。轮次公式、方形前提、最后行排序和蛇形输出，都是完整流程的组成部分。

参考来源 [93] [94]

57 位移有界数据的堆排序

类型 算法 别名或相关名称 k 位移排序；近有序最小堆方法

先把要用的小动作讲清楚

先解释前提。若最终顺序是 [1,2,3,4]，输入 [2,1,4,3] 中每张纸只离最终位置一格，这叫位移最多 1。它不等于“看起来有点乱”，而是每条记录都遵守一个已知距离上限。

若上限为 k ，最终第一个数不可能藏在原来第 $k+2$ 格或更后，因为那样它向前走得超过 k 格。因此只需在前 $k+1$ 项中找第一个输出。每出一项，再读一项，下一次同样有足够候选。最小堆帮我们从这大批候选里快速选最小。

基本思路

已知乱序距离很短时，只维护一个小候选堆，而不是排整批。每次取候选最小值、读进下一条补位。前提把可能的下一个答案限制在一个小窗口里。

具体执行步骤

1. 先从数据来源确认每条记录位移最多 k 的保证。不能只为了加速随意选一个小 k ；若无此保证，使用一般排序。
2. 读最前面至多 $k+1$ 条，逐项插入最小堆：新项放树末端，比父小就往上交换。记录数不足 $k+1$ 时全读进来。
3. 取堆根写到下一个输出位置。删除根、用堆末项补根，再与较小孩子向下比较交换，直到堆关系正确。
4. 若仍有未读输入，读一条插入堆，再取下一个最小值。每次只读一条，因为刚输出也只释放了一个候选位置。
5. 输入耗尽后继续取根直到堆空。如要稳定，用原位置作为相等键的第二比较条件。

跟着数字一步步做

1. 输入 $[2,1,4,3,6,5]$ ，每项只错开一格，因此 $k=1$ ，候选最多两条。先读 2、1，堆根是 1，输出 $[1]$ 。
2. 读 4，候选 2、4，出 2，输出 $[1,2]$ 。读 3，候选 3、4，出 3，输出 $[1,2,3]$ 。
3. 读 6，候选 4、6，出 4。读 5，候选 5、6，出 5。输入读完，最后取 6，得到 $[1,2,3,4,5,6]$ 。
4. 如果对 $[3,2,1]$ 误用 $k=1$ ，最开始只看 3、2，会错误先出 2，而真正最小的 1 还没进来。这个反例说明前提关系到正确性，不只是速度。

要做多少事 要用多少地方

候选最多 $k+1$ 条，每次堆更新只沿约 $\log(k+2)$ 层走，处理 n 条的上界 $O(n \log(k+2))$ ，额外候选空间 $O(k+1)$ 。写 $k+2$ 是为了覆盖 $k=0$ ：已经完全有序时仍需读写 n 条，是 $O(n)$ ，不可能零工作。 k 接近 n 就失去小窗口优势。违反位移保证可能直接输出错序。默认堆取出不稳定，加原序号可稳定。

为什么快或慢

堆比 n 小得多，每次更新只沿很短的路径调整； k 接近 n 时优势消失。

什么时候适合用

时间戳有有限乱序窗口、传输造成有限错位且边界已确认的数据。

最后把容易弄错的地方说清楚

若先完整排序才能确认 k ，就已经花掉一般排序的成本。实际保证通常来自传输或时间戳最多乱序若干位置的已知规则。

参考来源 [46]

五 工作较多或操作受限的方法

先看梳与 Circle 等步幅或整遍方法，其总体界需按版本判断；然后是通常平方级的比较/移动层，最后是高次与超多项式递归。Cycle 可少写但比较仍多；Pancake 按整段翻转计费。平方层内以原理依赖安排，不能宣称每项必比后一项快。

58 梳排序

类型 算法 别名或相关名称 梳子排序；CombSort11 是间隔规则变体

先把要用的小动作讲清楚

间隔是两个位置之间的距离。间隔 3 表示比较第一格和第四格，再比较第二格和第五格。

向下取整就是丢掉小数部分，例如 $4 \div 1.3 \approx 3.08$ 变成 3。间隔不能缩成 0，否则两格就没有区别。

基本思路

先比较远处的数，让放错很远的数一次跨好几格；再把比较距离慢慢缩短。最后距离变成 1，用相邻交换清理小范围错误。它像先做粗调整，再做细调整，但每个大间隔只扫一遍，并不保证当时已经排好。

具体执行步骤

1. 先记间隔为数组长度。每遍开始除以约 1.3，向下取整；算出 0 时改为 1。本条固定用这一常见规则。
2. 从第一格起，比较它与右边相距该间隔的那一格，左数大就交换。向右挪一格，继续；右侧配对超出数组时结束本遍。
3. 下一遍再缩小间隔。大间隔没有交换也继续，因为靠得更近的错误还没检查。
4. 间隔达到 1 后不再缩小，反复完整检查邻居。一整遍没有交换才停止；这时全部相邻关系都正确。

跟着数字一步步做

1. 输入 [4,1,3,2]。长度 4 除以 1.3 后取整，间隔为 3。只能比较第一、第四格的 4、2，交换为 [2,1,3,4]。2 一次跨了三格。
2. $3 \div 1.3$ 取整为 2。比较第一、第三格的 2、3，再比较第二、第四格的 1、4，两对都不交换。数列仍是 [2,1,3,4]，显然尚未有序。
3. $2 \div 1.3$ 取整为 1。先交换 2、1，成为 [1,2,3,4]；再检查 2、3 和 3、4，不动。
4. 间隔继续保持 1。再检查一整遍没有交换，结束。这里大间隔的安静一遍没有误导停止判断。

要做多少事 要用多少地方

即使输入有序，这个固定缩距规则也要做许多间隔遍； n 为个数，约 $\log n$ 遍、每遍约 n 次检查，最好 $\Theta(n \log n)$ 。 $\log$ 表示规模反复按固定比例缩小所需的层数。最坏仍可能由相邻收尾花 $O(n^2)$ 时间。额外空间 $O(1)$ ，通常不稳定；没有一个适用于所有间隔规则的精确平均公式。

为什么快或慢

长距离交换省去了一部分逐格移动，末尾却仍可能像冒泡那样反复扫描。它与希尔不同：希尔在每个间隔把相应数列充分整理，这里通常只扫一遍。

什么时候适合用

适合学习长距离比较与缩距，也可做简单实验。需要严格运行时间保证时，应选择保证更明确的算法。

最后把容易弄错的地方说清楚

例如 [2a,2b,1]，远距离交换可把 2a 放到 2b 后面，所以相等记录不一定稳定。CombSort11 常把间隔 9 或 10 改成 11，仍属于同一方法。其他初始间隔或缩距规则要另注条件。

参考来源 [95]

59 圆圈排序

类型 算法 别名或相关名称 Circlesort；不是 circular sort 的所有同名方案

先把要用的小动作讲清楚

两头向中间走：先看第一、最后一格，再看第二、倒数第二格。每次把较小值留在左边。

一个完整回合包含整串扫描和所有较小区间的扫描。局部不交换，不代表整个回合没有交换。

递归处理半段，是先暂停当前任务，完整处理左半，再处理右半，再回到当前任务结束；一格无须比较。

基本思路

先比较两端对应的数，一对一对向中间走，让小数向左、大数向右；再用同样规则处理两个半段。这样一个完整回合仍可能没排完，所以从整串再来一回，直到所有大小区间都没有发生交换。名字描绘了比较的配对方式，不要求真的把数组摆成圆。

具体执行步骤

1. 对当前区间，左指针放第一格，右指针放最后一格。左值大于右值就交换；否则保留。两指针各向中间移动一格，继续，直到不能组成不同的两格。
2. 奇数项会剩下一个中间项。本条的具体版本再把它与紧挨着的右边一项比较；中间值较大就交换。区间只有一项时直接结束，不做这一步。
3. 把当前区间分成相邻的两半，奇数项时中间项归左半。先对左半完整执行上述方法，再对右半执行，逐层缩小到一项。
4. 记下这一整回合是否有任何交换。只有从整串开始、连所有小任务都没有交换，才停止；只要有一次交换就再从整串开始。

跟着数字一步步做

1. 输入 [4,1,3,2]。先比较外端 4 与 2，交换得 [2,1,3,4]。两指针向内，比较 1 与 3，顺序已正确，不交换。
2. 暂停整串任务，处理左半 [2,1]：外端 2>1，交换，于是整体为 [1,2,3,4]。左半的单项任务结束。再处理右半 [3,4]，不交换。第一个完整回合结束。
3. 这一回合曾经交换，不能只靠眼看结果就按规则宣布结束。再做完整一回，所有外端、内端和半段比较都不交换，才停。
4. 奇数例子 [1,3,2]：外端 1、2 正确，但中间 3 还没得到合适的位置。加做中间与右邻居的比较，3>2，交换成 [1,2,3]。
5. 这时分成左半 [1,2] 与右半 [3]，左半无需交换，右半直接返回。再一个完整无交换回合，证明完成。漏掉本版本的奇数中心比较，会漏过关键关系。

要做多少事 要用多少地方

每个完整回合不是只扫一遍：整段比一批，两个半段又比一批，再拆小段。约有 $\log n$ 层， $\log n$ 在这里表示反复减半的层数；每层总工作约 n ，因此每回合 $\Theta(n \log n)$ 。若需要 p 回合，总时间 $\Theta(p n \log n)$ ， Θ 是同一增长数量级。有序输入一回合即可，最好 $\Theta(n \log n)$ 。调用层数额外空间 $O(\log n)$ 。作者报告常见 $O(n \log^2 n)$ 表现，不能当所有版本的严格最坏保证。

为什么快或慢

远距离交换能让数一下移动很远，小区间再修补局部。但对称扫描不保证左右两半已经完全分开，局部修补以后可能还需要再做整串扫描。

什么时候适合用

适合学习比较配对、递归分段和“整个回合无变化”的停止条件。实际工作通常更愿意选有明确保证的成熟方法。

最后把容易弄错的地方说清楚

一般不稳定。本条使用中心归左半、额外比较右邻居的具体规则；还有中心重叠等版本。为何无交换能够证明有序？二项区间检查内部相邻关系，偶数区间的内端检查两半边界，奇数区间的额外比较检查中间与右半边界，所有相邻关系都被覆盖。一个非常松的上界是 $O(n^3 \log n)$ ：至多 $O(n^2)$ 个有交换回合，因为交换倒置的两项会减少至少一个逆序关系，每回合又花 $O(n \log n)$ 。这只是保守上界，不是速度预测。

参考来源 [96]

60 循环排序

类型 算法 别名或相关名称 最少写入的比较式循环排序；另有已知目标位置的线性变体

先把要用的小动作讲清楚

“写入”是改变某一格存的值。交换通常写两格；把手里暂存的值放进一格，只写那一格。

“目标位置”就是这个值在有序结果中的位置。例如三个值比它小，它就应在第四格。

图中的 □ 是逻辑空位：我们已经拿起这格的数，稍后要填回。实际内存可以保留旧副本，不必真的写一个空符号。

基本思路

循环排序想少写格子。先把一个数拿到手里，数清有多少数应在它前面，直接送到目标格。目标格原来的数不能丢，把它接到手里，再送往它的目标。这样一串交接最终回到起点，填上最初留下的空位，像沿一圈位置传东西，所以叫循环。

具体执行步骤

1. 选一个起点，暂存其中的值。先检查起点后面的所有数，计数其中严格比手里值小的数。已处理的左侧不需要再看；目标就是从起点往右数这么多格。
2. 第一次算出目标就是起点，说明没有后面的数比它小，当前值已经应在这里，不启动交接，换下一个起点。
3. 若目标格已经是同值，不能把同值挤出来，否则没有实际推进，可能反复拿到一样的数。继续向右找，跳过所有同值格，找到需要改变的格。
4. 写入前先保存目标格原值，然后把手里值写进去，再让保存的原值成为手里新值。每次只写目标格一处，起点的逻辑空位一直保留。
5. 对新值重新数起点后缀里的较小值，确定新目标；仍跳过同值，然后放入、接出。已经放好的目标格保留正确值。目标回到起点时填回，这一圈结束。再从下一个起点重复，直到所有格都处理过。

跟着数字一步步做

1. 输入 [3,1,4,2]。拿起第一格的 3。后面 1、2 两个值比它小，所以目标是第三格。放入 3、接出 4，画成 [□,1,3,2]，手里是 4。
2. 后缀里的 1、3、2 都比 4 小，目标第四格。放 4、接 2：[□,1,3,4]，手里是 2。
3. 后面只有 1 比 2 小，目标第二格。放 2、接 1：[□,2,3,4]，手里是 1。没有后面值比 1 小，目标回到第一格；填入后 [1,2,3,4]。这四格各写一次。
4. 再检查第二格的 2：后面 3、4 没有更小值，所以不启动新一圈。第三格也一样；最后一格无需再处理。
5. 重复值例 [2,1,2,1]：拿起第一格 2，有两个 1 比它小，算出的第三格已是 2。跳过第三格，送到第四格，接出 1，成为 [□,1,2,2]。手里 1 的目标是第一格，填回得 [1,1,2,2]。这样只写原本错误的第一、第四格。

步骤状态图 C 比较 M 移动 S 已有序

拿起 3，数出目标



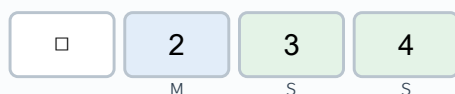
写 3，手中是 4



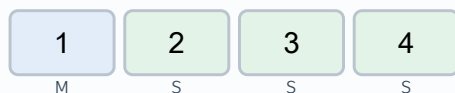
写 4，手中是 2



写 2，手中是 1



填回 1，循环闭合



要做多少事 要用多少地方

少写入需要付出反复数位置的代价。 n 是元素个数，基本比较版即使已经有序也要逐个数后缀，最好、通常、最坏 $\Theta(n^2)$ ，即二次规模比较。每个要改变的值位置尽量只写一次，写入总量 $O(n)$ ；手里只暂存一个值和少量位置，额外空间 $O(1)$ 。通常不稳定。若某种特殊键本身直接给出目标，不必反复比较，特定循环方法可以线性；那需要额外条件。

为什么快或慢

它优化的是写入，不是所有操作。数目标时会重复读、比较许多元素，但直接送到目标避免了往返交换和长串移位。读写成本差别很大时才可能合算。

什么时候适合用

适合学习排列交接，或评估写入昂贵的存储。先分别估计读、比较、写的成本，再决定是否使用。

最后把容易弄错的地方说清楚

相等值跳过规则保证值排序能推进，却不保护同值记录的身份先后，因此不自动稳定。Haddon 的线性版本利用特定键范围或已知目标；不要因为看到“循环排序”这个名字，就把它线性时间套到此处的一般比较版。

参考来源 [97] [98]

61 秩排序（枚举排名）

类型 算法 别名或相关名称 排名排序；枚举排序（不要与枚举全排列混淆）

先把要用的小动作讲清楚

一条记录可以有数值和身份，例如 3a、3b 的数值都为 3，字母只用来追踪它们。

秩从 0 数，表示该记录前面应有几条记录。秩 0 放第一格，秩 1 放第二格，不是把第 0 格放进结果。

基本思路

让每条记录自己计算应排第几：数一数其他记录中有多少条应放在它前面。这个数量就是它的秩。全部秩算出来后，把记录直接写到对应的结果格。两个相等值也必须要有不同位置，所以相等时按原先的位置决定谁在前。

具体执行步骤

1. 保留输入，不立即覆盖它；另外准备与输入同长度的结果数组。需要重复比较原记录，提前覆盖会丢掉还没使用的信息。
2. 取一条记录，计数先设为 0。与输入的其他记录比较；对方值更小，计数加 1。
3. 遇到相等值，检查原来谁更靠左。对方原来更靠左，也加 1；对方在后面则不加。同一条记录不算在自己前面。
4. 计数完毕，把该记录写到从第一格往后走“计数”格的位置。对所有记录重复。由于每条记录拥有不同秩，结果不会抢占同一格；填完再读取或复制回原数组。

跟着数字一步步做

1. 输入 [3a,1,3b,2]，3a 原来在第一格，3b 在第三格。
2. 计算 3a：1 比它小，计数变 1；3b 同值但在后，不加；2 比它小，计数变 2。所以 3a 写入结果第三格，结果暂为 [□,□,3a,□]。
3. 计算 1：其他值都更大，计数 0，填第一格：[1,□,3a,□]。
4. 计算 3b：同值 3a 原来在前，计一次；1 更小，再计一次；2 更小，再计一次，总计 3，填第四格：[1,□,3a,3b]。
5. 计算 2：只有 1 应在前，计数 1，填第二格：[1,2,3a,3b]。如果相等值不按原位置分先后，两个 3 都会算出秩 2，写同一格并覆盖一条记录。

要做多少事 要用多少地方

每条记录都要看全部其他记录， n 条就形成约 n^2 次比较；最好、通常、最坏 $\Theta(n^2)$ 。结果数组需要随 n 增长的空间 $\Theta(n)$ ，这里 n 是记录数。每条只向结果写一次，总写入 $O(n)$ 。原位置破同值平局让排序稳定。并行可以同时算不同记录的秩，但比较总工作仍是二次，还要计求和和通信。

为什么快或慢

目标独立计算，容易分给多个处理器。代价是同一批记录被一遍又一遍比较，没有像归并那样重复利用整段已有顺序。

什么时候适合用

适合同时需要排名的任务、硬件比较和并行教学；普通单处理器排序通常有更省比较的方法。

最后把容易弄错的地方说清楚

enumeration 在这里指逐个列举比较对象，不是枚举所有排列。它也不是计数排序：计数排序按有限键值范围统计次数，这里逐条比较记录。稳定性的依据是原位置规则，不是“都写进另一个数组”。

参考来源 [99]

62 选择排序

类型 算法 别名或相关名称 直接选择排序

先把要用的小动作讲清楚

把数写成一排，位置从左到右数。比较就是判断两个数谁小，比较本身不改变这排数。

交换两格时，先把第一格的数暂存，第二格的数填入第一格，再把暂存数填入第二格。这样两个数都不会丢。

基本思路

先把最小的数找出来，放在第一格；再从剩下的数里找最小的，放在第二格。一直这样做。每次只确定一格，但那格一旦确定，以后就不用再碰了。关键在于：必须看完所有候选，才能知道手里记着的最小值确实是最小。

具体执行步骤

1. 先看第一格，记下它的值和位置，把它暂时当作最小值。这只是当前最好的候选，还不是结论。
2. 从第二格开始往右看。每看见一个更小的数，就改记这个新数的位置；比较到最后一格才结束寻找。寻找期间不移动任何数。
3. 把最后记住的最小值与第一格交换。如果它原来就在第一格，什么也不搬。现在第一格比所有剩下的数都小，因此已经是最终位置。
4. 第二轮把第二格当作目标，只在第二格及其右边寻找。第三轮从第三格开始，依次继续。左边已经放好的数不再参与。只剩最后一格时，其他格都正确，它也自然正确。

跟着数字一步步做

1. 从 [4,2,5,1,3] 开始。暂记 4；看到 2 改记 2；看到 5 不改；看到 1 改记 1；看到 3 不改。现在才确定 1 是全体最小值。
2. 交换第一格的 4 与第四格的 1，得到 [1,2,5,4,3]。左边的 1 永久放好，4 仍留在待处理区里。
3. 第二轮看 [2,5,4,3]：没有比 2 更小的，保留原样 [1,2,5,4,3]。前两格已经确定。
4. 第三轮从 5 开始，看见 4 改记 4，看见 3 再改记 3。交换第三格的 5 和第五格的 3，成为 [1,2,3,4,5]。
5. 第四轮还要比较 4 和 5；没有交换。最后一格留下 5。基本方法不知道自己提前排好了，所以仍完成这些规定的查找。

要做多少事 要用多少地方

它花时间的主要是找数：第一轮比较 $n-1$ 次，下一轮 $n-2$ 次，直到 1 次。这里 n 是元素个数，总比较为 $n(n-1)/2$ ，所以最好、通常、最坏都随 n^2 增长，写作 $\Theta(n^2)$ 。至多交换 $n-1$ 次，只要一个暂存格和几个位置，额外空间 $O(1)$ 。通常不稳定：相等记录可能被远距离交换改变先后。

为什么快或慢

交换少，但查找重复很多。数组已经有序，也仍要逐轮看完剩余部分。如果写入很昂贵、比较很便宜，少交换可能有价值；它并不会因此成为一般数据上的快速算法。

什么时候适合用

适合很小的数组、学习如何确定最终位置，或专门重视少写入的简单场景。大数组通常用比较总量更少的排序。

最后把容易弄错的地方说清楚

例如 [2a,2b,1]，a、b 是两个同值记录的名字。把 1 与 2a 交换会得到 [1,2b,2a]，先后改变。要稳定，可以拿出第一个最小值，把它前面的那些记录逐格右移，再插入；但移动次数可能达到二次规模。

参考来源 [100] [31]

63 双向选择排序

类型 重要变体 别名或相关名称 双选择排序；最小最大选择排序

先把要用的小动作讲清楚

左右边界是两个位置标记，夹在它们中间的数还没放好。边界外的数已经最终确定。

记住最小、最大的位置，就是记住两格的地址；交换后值可能换地址，所以地址也要更新。

基本思路

一轮找两个目标：最小值放左端，最大值放右端。然后把两端留在原地，只处理更小的中间区域。它就是选择排序的双端版本。少了一些轮次，但每轮还是必须把中间区域看一遍，才能知道两个极值是谁。

具体执行步骤

1. 先把最左和最右格标出来。扫描这整个区域，分别记住目前最小值和最大值的位置。每个新数都可能更新这两个记录。
2. 扫描结束后，先把最小值与左端交换。左端现在放的是该区域最小值，可以固定。
3. 再把最大值与右端交换。特别检查：如果最大值原先在左端，第一场交换已经把它搬到了原最小值的位置，必须使用这个新位置。
4. 左右边界各往中间移一格，再找新的最小和最大。只剩一格时，它夹在已放好的两边之间，不需要移动。

跟着数字一步步做

1. 输入 [5,2,3,1,4]。完整扫描发现最小是第四格的 1，最大是第一格的 5。
2. 把 1 与左端 5 交换，成为 [1,2,3,5,4]。此刻最大值 5 在第四格；继续用“第一格”去找它，会错误地拿到 1。
3. 把现在第四格的 5 与右端 4 交换，得到 [1,2,3,4,5]。两端 1、5 都可以固定。
4. 下一轮只看 [2,3,4]。最小 2 和最大 4 已在这段的两端，所以不交换。把两边缩进去，只剩中间的 3，完成。

要做多少事 要用多少地方

每轮扫描排区域的全部数，虽然区域每次缩短两个，总扫描量仍随 n^2 增长； n 是元素个数。普通版本最好、通常、最坏 $O(n^2)$ 。每轮最多两次交换，总交换 $O(n)$ ，额外只记几处位置，空间 $O(1)$ 。通常不稳定。成对寻找极值可以少做一些比较，但增长级别不变。

为什么快或慢

它减少轮数，不会消除反复搜索。不能只看轮数减半，就断言它快两倍。实际差异还取决于比较和交换分别有多贵。

什么时候适合用

适合学习一遍扫描同时得到两个结果，或处理很小的数组。它不是沿相邻格来回交换的鸡尾酒排序。

最后把容易弄错的地方说清楚

全相等时不必做交换，但基本版仍要扫描。相等记录若要保留原先顺序，需要额外稳定规则；普通远距离交换没有这个保证。最大值位置被第一次交换改变，是这类实现最容易漏掉的情况。

参考来源 [100]

64 侏儒排序

类型 算法 别名或相关名称 花园侏儒排序；最初也称 stupid sort，名称有歧义

先把要用的小动作讲清楚

用一个位置标记指着当前格。标记向左或向右走不会移动数，只改变下一次检查的位置。

当前数往左移动，可能撞上另一个更大数，所以交换后还要向左退一步继续检查。

基本思路

看当前数和它的前一个邻居。两个数已经按序，就往右走；左边更大，就交换并退一步。这样一个较小数会连续穿过前面较大的数，直到它找到正确位置，然后再往右继续。这是用相邻交换来完成插入。

具体执行步骤

1. 开始指向第二格。第一格暂时只是一段长度 1 的有序部分，不需要内部比较。
2. 若前一格小于等于当前格，两格关系正确，标记向右走一格。越过最后一格，就说明处理结束。
3. 若前一格大于当前格，交换它们，然后标记向左退一格。刚被放到左边的较小数现在要与更前面的数比较。
4. 如果退到第一格，没有左邻居，就往右走一格。不断重复上述规则；相等时只前进、不交换。

跟着数字一步步做

1. 输入 [4,1,3,2]，标记在第二格 1。4>1，交换：[1,4,3,2]；退到第一格后再向右。1≤4，标记走到第三格。
2. 4>3，交换：[1,3,4,2]，退到第二格。检查 1≤3，通过；再检查 3≤4，通过，标记到第四格。
3. 4>2，交换：[1,3,2,4]，退到第三格。3>2，再交换：[1,2,3,4]，退到第二格。
4. 检查 1≤2，通过。接着 2≤3、3≤4 都通过，标记依次右移，最终越过末端。2 沿两次交换插入正确位置，其他数仍全部保留。

要做多少事 要用多少地方

有序时标记只向右走一遍，最好 $\Theta(n)$ ， n 为个数。杂乱时很多数需要反复向左穿过邻居，通常、最坏 $\Theta(n^2)$ 。额外只存一个位置和交换临时值，空间 $O(1)$ 。只交换严格逆序邻居，所以稳定。

为什么快或慢

比固定重复全遍扫描更局部：哪里出错就往哪里退。但每走过一个较大邻居仍要一次交换，远距离错误仍昂贵。

什么时候适合用

适合极小或很有序的数组，也适合学习“出错就退一步”的过程。数组插入排序可把一串值直接移位，常有更低操作开销。

最后把容易弄错的地方说清楚

优化版记住此前最远处理到的位置。较小数插入完，可以直接回到那里，不再逐格穿过已经知道有序的部分。基本版和优化版都叫 gnome sort；stupid sort 有时也指 Bogosort，不能仅凭这个别名判断过程。

参考来源 [101]

65 冒泡排序

类型 算法 别名或相关名称 下沉排序；exchange sort 在部分资料中也指冒泡

先把要用的小动作讲清楚

相邻就是两格紧挨着，没有别的格夹在中间。一遍扫描指从左走到右，把这段内每一对相邻格都检查一次。

交换只会让两个数各走一格。相等数保留原先先后；这样可以让同值记录保持稳定。

基本思路

从左向右比较邻居，左边更大就互换。一个很大的数换到右边后，下一步又与新的右邻居比较，可以一路走到最右。完整走一遍后，当前最大的数必然停在右端。把它留下，再对前面重复。

具体执行步骤

1. 从待排区最左的一对开始。左数大于右数就交换；否则原样保留。比较的目的是让这一对较小的在左、较大的在右。
2. 向右前进一步，比较下一对。注意上一对的右格也是下一对的左格，所以刚被推右的大数会继续参加。
3. 一直走到本轮右边界。一路携带到这里的值是区域最大值，因而最后一格已经最终放好。下一轮不再碰它。
4. 记录这一整遍有没有交换。若完全没有，所有相邻关系已经正确，可以停止。若有，回到左端处理缩短后的区域；只有一格时自然结束。

跟着数字一步步做

1. 输入 [4,1,3,2]。先比较 4、1，4 更大，交换成 [1,4,3,2]。
2. 下一对是现在的 4、3，交换：[1,3,4,2]。再比较 4、2，交换：[1,3,2,4]。这轮的 4 走了三格，已经定位。
3. 第二轮只看前三格。1≤3，不动；3>2，交换：[1,2,3,4]。这轮结束，3 也位于剩余区域的末端。
4. 第三轮看剩下的 1、2，不交换。整遍无交换，停止。某一对不交换只证明那一对正确，不能代替完整一遍。

步骤状态图 C 比较 M 移动 S 已有序

比较 4 与 1



4 向右一格



继续把 4 推右



第一遍：4 定位



第二遍交换 3、2



要做多少事 要用多少地方

已经有序时，一遍约 n 次比较就结束，因此最好 $O(n)$ ， n 是元素个数。普通杂乱输入和最坏情况可能需要约 n 遍，每遍又看约 n 格，总时间 $O(n^2)$ 。额外空间 $O(1)$ 。严格大于才交换的相邻版本稳定。若旧版强制做完所有遍，最好也为二次时间。

为什么快或慢

大数向右很快，小数从最右向左却慢：在左到右的一遍中，它通常只能左移一格。 $[2,3,\dots,n,1]$ 因此仍需许多遍，不能只说“看起来快有序了”。

什么时候适合用

适合学习扫描、相邻交换和停止判断。对于实际小数组，插入排序往往更省操作。

最后把容易弄错的地方说清楚

可以记住最后一次发生交换的位置，下次不用越过它，因为更右侧已没有需要纠正的相邻关系。部分文献把它叫 exchange sort，但这个名字也用于别的过程；本书的直接交换排序另有明确步骤。

参考来源 [102]

66 鸡尾酒排序

类型 重要变体 别名或相关名称 双向冒泡排序；摇摆排序；shaker sort

先把要用的小动作讲清楚

一次从左到右，一次从右到左，这叫两个方向的扫描。无论走哪个方向，比较后总是把较小值放左、较大值放右。

两端边界之间是待排区；边界缩短只是以后少检查一些格，不会删除任何元素。

基本思路

鸡尾酒排序让冒泡往返走。向右的一趟把最大值带到右端，向左的一趟把最小值带到左端。这样右边的小数也有机会在一趟中连走几格，避免一直等下一轮才能再往左走。

具体执行步骤

1. 从待排区左端向右比较所有相邻对，左数大于右数才交换。结束后区域最大值已到右端，把右边界缩短一格。
2. 现在反向走。从缩短区域的右边开始，比较一对后向左挪一个位置。仍然是左数较大才交换；被换到左边的小数会立即进入下一对。
3. 反向走到左端时，区域最小值已经被带回来。把左边界右移一格。然后在更小的中间区域继续往返。
4. 任何一个完整方向都没有交换，就说明该区域全部相邻关系已正确，可以结束。若边界之间只剩一格，也结束。

跟着数字一步步做

1. 输入 $[2,3,4,1]$ 。向右先检查 2、3，再检查 3、4，都不换；最后 $4>1$ ，交换为 $[2,3,1,4]$ 。4 固定在右端。
2. 向左只处理前三格。从 3、1 开始，交换为 $[2,1,3,4]$ 。1 被带到左边，下一对立即会再次看到它。
3. 比较 2、1，交换为 $[1,2,3,4]$ 。1 在同一趟里左移两格，现在左端固定。
4. 中间只剩 $[2,3]$ 。下一次向右比较没有交换，确认结束。反向扫描不是要求降序，它改变的是检查方向。

步骤状态图 C 比较 M 移动 S 已有序

向右：比较 4、1



4 在右端定位



向左：1 越过 3



1 在左端定位



要做多少事 要用多少地方

已排序时一趟就能确认，最好 $\Theta(n)$ ， n 为个数。通常和最坏仍可能反复做许多相邻交换，总时间 $\Theta(n^2)$ 。额外只需几个位置和交换暂存值，空间 $O(1)$ 。相等邻居不交换，所以稳定。

为什么快或慢

往返能改善某些位置模式，但一个数穿过很多其他数仍要做许多单格交换。它解决了单向冒泡的一些拖延，并没有消除二次工作量。

什么时候适合用

适合观察方向如何影响移动，或处理极小数组。一般软件排序仍优先考虑插入或更高效的方法。

最后把容易弄错的地方说清楚

鸡尾酒、shaker、双向冒泡是同一方法的别名。可以分别记录两个方向最后一次交换的位置，进一步缩小区域；必须在每次改变方向时正确更新边界。

参考来源 [103]

67 奇偶换位排序

类型 算法 别名或相关名称 砖排序；奇偶排序（区别于奇偶归并）

先把要用的小动作讲清楚

同时比较的两对不能共用一格，否则同一个数可能被两次操作争用。这里把邻居分成不重叠的配对。

位置从 1 开始数：“第一类”比较 1-2、3-4；“第二类”比较 2-3、4-5。奇偶名称指位置，不指数值奇偶。

基本思路

先让第一格和第二格排好、第三格和第四格排好；然后换一组邻居，第二格和第三格排好、第四格和第五格排好。交替检查两组邻接关系，较小的数逐步走左，较大的逐步走右。同一组内的各对互不干扰，所以可以并行执行。

具体执行步骤

1. 执行第一类配对，从第一格起每两格一组。每组左数较大就交换，否则保留。末端若落单，本阶段不动它。
2. 执行第二类配对，从第二格起每两格一组。它补上刚才没有检查的相邻边界。

3. 把这两个阶段作为一整轮。记住其中有没有任何交换；有交换就再执行一整轮。
4. 整轮都没有交换时，两类配对合起来已检查全部相邻关系，因此可以停止。只做一个阶段没有交换，还不足以确定有序。

跟着数字一步步做

1. 输入 [4,1,3,2]。第一阶段检查 4、1 和 3、2，分别交换，成为 [1,4,2,3]。
2. 第二阶段只检查中间 4、2，交换为 [1,2,4,3]。4 被向右再推一步。
3. 下一轮第一阶段检查 1、2（不换）和 4、3（交换），得到 [1,2,3,4]；第二阶段检查 2、3，不换。
4. 这轮仍发生过交换，所以再做一整轮：1、2 不换，3、4 不换，随后 2、3 也不换，才停止。对照 [2,1,4,3]：中间 1、4 已正确，但两侧仍错，说明一个安静阶段不能证明完成。

要做多少事 要用多少地方

在单处理器上逐对做，有提前停止的最好 $\Theta(n)$ ，通常、最坏 $\Theta(n^2)$ ， n 是个数；空间 $O(1)$ ，相等不交换时稳定。固定执行 n 个交替阶段的网络，总比较工作 $\Theta(n^2)$ 。若每阶段所有配对同时做，阶段数即深度为 $\Theta(n)$ ；这是并行时间，不是顺序总工作。

为什么快或慢

它容易并行，因为一阶段的配对互不重叠。代价是每次一个数最多走一格，需要许多阶段穿过长距离。

什么时候适合用

适合并行处理器链、简单比较网络 and 教学。顺序运行通常有更省比较的方法。

最后把容易弄错的地方说清楚

brick sort 是别名。Odd-even transposition 与 Batcher odd-even mergesort 不同：前者只轮换邻居，后者按另一套归并网络连接比较器，不能因为名字相近就合并。

参考来源 [104] [99]

68 直接交换排序

类型 算法 别名或相关名称 两两交换排序；exchange/interchange 名称在文献中有歧义

先把要用的小动作讲清楚

固定一格表示暂时一直使用这个位置作左侧比较对象；它的值在交换后会变化。

两两比较不要求相邻。例如第一格可以直接与第五格比较和交换，中间的格子不移动。

基本思路

固定当前左侧格，遇到一个更小的值就立即换进来，再拿这个新的当前值继续往右比较。扫到最后时，当前格留下的是整个区域最小值。下一轮固定下一格。与选择排序相比，它不等查找结束才交换，而是每遇到更小候选都先搬一次。

具体执行步骤

1. 从第一格开始，把它定为当前目标。第二格作为第一个比较对象。
2. 若目标值大于后面的值，立即交换这两格。交换后目标里的新值更小，继续用这个新值比较；否则不搬。
3. 比较对象每次右移一格，直到与全部后续格比较过。此时每个后续值都不小于当前目标值，所以目标已经最终确定。
4. 把目标右移一格，从它下一格开始扫描。已经确定的左边各格不再动，处理完倒数第二格后结束。

跟着数字一步步做

1. 输入 [4,1,3,2]。固定第一格，比较 4、1，交换成 [1,4,3,2]。现在目标值是 1，再与 3、2 比，都不换。第一格确定。

2. 固定第二格，值为 4。与第三格 3 比较，交换为 [1,3,4,2]。第二格暂存 3，但还有第四格没检查，所以还不能固定。
3. 拿现在的 3 与第四格 2 比较，再交换为 [1,2,4,3]。第二轮结束，第二格才确定为 2。
4. 固定第三格，比较 4、3，交换得 [1,2,3,4]。第二轮做了两次交换；普通选择排序先找出 2，再只交换一次，这就是过程上的差别。

要做多少事 要用多少地方

每个目标都与后续所有格比较，总比较 $n(n-1)/2$ ； n 是个数，所以最好、通常、最坏 $\Theta(n^2)$ 。最好输入不需交换，最坏交换也可能达到二次数量。暂存一个值和两个位置足够，额外空间 $O(1)$ 。远距离交换通常不稳定。

为什么快或慢

寻找最小值仍要完整看后缀，并且一格可能被反复写入更小候选。已排序输入能省交换，省不了这些固定比较。

什么时候适合用

适合学习“立即搬候选”与“先记位置再搬”的差别，也可帮助读懂旧程序中的短区间排序。

最后把容易弄错的地方说清楚

例如 [2a,2b,1]，第一格与第三格交换后 [1,2b,2a]，同值顺序被改变。exchange 和 interchange 在不同来源里也可能指冒泡或整个交换类。本书只把此处明确的非相邻双循环过程称为直接交换排序。

参考来源 [105]

69 基于旋转的原地归并排序家族

类型 算法家族 别名或相关名称 原地归并；本节采用逐项插入式旋转归并

先把要用的小动作讲清楚

把两段互相挪位、但保留各段内部先后，叫作旋转。[4,7 | 2] 旋转后是 [2 | 4,7]。本节只旋转右边的一个数。

“原地”在这里表示不再分配能装下整排数的副本；仍可临时拿住一个数。

左排、右排必须分别已经有序。两个当前位置分别记住尚未确认的左项和右项。

a 是这排数的名字， $a[i]$ 就是编号 i 的格子里那项；方括号是在指定位置，不是做乘法。

基本思路

两排各自有序时，如果右边的头更小，就把它插到左边当前项之前。为免覆盖数据，先把右项拿在手里，再从右往左逐格把挡路的数挪一格。重复后两排连成排。这是容易实现的旋转归并，不是高级研究版的速度保证。

具体执行步骤

1. 先用相邻两项形成有序小排，再按长度二、四、八等接更大的邻排。一次接排中，记 i 为左排下一项位置、 j 为右排开始位置， end 为两排之后的位置。编号从 0 开始， end 本身不属于范围。
2. 只在 $i < j$ 且 $j < end$ 时继续。如果 $a[i] \leq a[j]$ ，左项不可能大于右排任何剩余项，确认它在本范围中的顺序，令 i 加一。
3. 若 $a[j] < a[i]$ ，先保存 $x = a[j]$ 。从 $j-1$ 开始往回到 i ，把每项移到右边一格；反着移是为了读到尚未被覆盖的旧值。再把 x 写入 i 。
4. 插入后，确认区多了一项， i 加一；左排尚未处理的项整体右移，右排新头也右移， j 加一。不要只移动 i 而忘记改 j 。
5. 一排已经全部确认时结束。剩余那排内部有序，继续保持即可。扩大排长并重复，直至整个数组有序。

跟着数字一步步做

1. [4,1,3,2] 先接第一对：保存 1，把 4 从位置 0 移到 1，再写 1 到 0。第二对保存 2、移 3、写 2。得到 [1,4 | 2,3]。

2. 最后接排， $i=0$ 、 $j=2$ 、 $end=4$ 。 $1 \leq 2$ ，保留 1， $i=1$ 。现在 $4 > 2$ ，拿住位置 2 的 2，把位置 1 的 4 右移，再把 2 放在位置 1。成为 $[1,2,4,3]$ ， $i=2$ 、 $j=3$ 。
3. 比较 4 与 3，拿住 3，右移 4，再把 3 写在位置 2。成为 $[1,2,3,4]$ ， $i=j=3$ ，左排已处理完，结束。每一步拿住的值都要写回来。

要做多少事 要用多少地方

设 n 为项数。顺序合适时，每层只逐项比较，基础最好 $O(n \log n)$ ；若接排前检查左末项 $\leq$ 右首项而跳过整个接排，有序输入可达 $O(n)$ 。某个右项可能让很多左项右移；很多次这样的插入累计起来，通常与最坏可能达到 $O(n^2)$ 。迭代排长控制只需 $O(1)$ 额外空间，递归排长控制另需 $O(\log n)$ 栈。相等时保留左项，不让等值互相越过，因此稳定。

为什么快或慢

省下大辅助数组，却可能把同一左项挪很多遍。用二分查找更快找到插入位置，也不能免掉腾空位置时的搬动。

什么时候适合用

很小的范围、内存极少而必须稳定时有用；若数据很大，应了解块归并等更复杂方法。

最后把容易弄错的地方说清楚

不能给这个逐项挪动版本套用高级原地归并的 $O(n \log n)$ 最坏界。一般两块旋转还可先各自反转，再反转整体；三次反转之后，两块换位而内部顺序恢复。

参考来源 [\[106\]](#) [\[107\]](#)

70 煎饼排序

类型 算法 别名或相关名称 前缀翻转排序

先把要用的小动作讲清楚

前缀是从第一格开始的一整段，例如 $[3,1,4,2]$ 的前三格前缀是 $[3,1,4]$ 。

翻转一段就是把整个顺序反过来。实现时先交换两端，再交换向内一格的一对，直到中间；不是仅交换最外一对。

基本思路

只允许翻转从最左开始的一段，像从上面翻一叠煎饼。要把当前最大值放到待排区末端，先翻到它为止，把它带到最前；再翻整个待排前缀，让它到末端。这样每轮固定一个大值，然后只处理更短的左侧前缀。

具体执行步骤

1. 在当前前缀里逐格找最大值：先记第一格，遇到更大值就更新位置，看完才确定。若最大值已经在前缀末端，这轮不用翻。
2. 否则，翻转第一格到最大值所在格的整段。最大值原来在这段末端，翻后就在第一格。如果它本来就在第一格，这一步省略。
3. 翻转整个待排前缀。第一格的最大值会跑到这一段末端，因此到达最终位置。
4. 把前缀长度减一。右边已固定的大值不再参与翻转。只剩第一格时，其他位置都正确，结束。

跟着数字一步步做

1. 输入 $[3,1,4,2]$ 。找出最大 4 在第三格。翻前三格 $[3,1,4]$ ：交换两端 3、4，中间 1 不动，整列成为 $[4,1,3,2]$ 。
2. 翻四格：先交换外侧 4、2，暂为 $[2,1,3,4]$ ；再交换内侧 1、3，得到 $[2,3,1,4]$ 。4 最终固定。
3. 在前三格找最大 3，它在第二格。翻前两格，得到 $[3,2,1,4]$ ；再翻前三格，把 3、1 交换，得到 $[1,2,3,4]$ 。3 固定。
4. 剩余前两格最大值 2 已在第二格，不用翻。只剩 1，完成。每一场翻转都局限在当前前缀，没有碰已经固定的 4。

步骤状态图 C 比较 M 移动 S 已有序

最大值 4 在第三格



翻前三格：4 到前端



翻四格：4 定位



翻前两格：3 到前端



翻前三格：3 定位



要做多少事 要用多少地方

基本版本每轮都重新找最大，扫描长度 $n, n-1, \dots$ ，因此最好、通常、最坏 $\Theta(n^2)$ ， n 是个数。每轮最多两次翻转，总次数 $O(n)$ ；但长度 m 的翻转约需 $m/2$ 次交换，不能把一次长翻当成实际电脑上的一步。反转可原地交换，额外空间 $O(1)$ ，通常不稳定。

为什么快或慢

一次操作能改变很大范围，却也必须真的搬动这个范围的值。它研究的是操作受到“只能翻前缀”限制时如何完成排序。

什么时候适合用

适合煎饼问题、前缀翻转的排列问题和教学。普通数组没有这个限制时，更高效方法通常更合适。

最后把容易弄错的地方说清楚

本方法保证排好，不保证翻转次数最少。相等记录在段内反转时可能倒序，所以不稳定。烧焦煎饼还给每个元素加上朝向，翻转时必须同时变号，下一条变体再解释。

参考来源 [108]

71 烧焦煎饼排序

类型 重要变体 别名或相关名称 带方向前缀翻转排序

先把要用的小动作讲清楚

这里 3、2、1 是煎饼大小，正负号表示朝向。+3 是朝向正确的三号煎饼，-3 是翻面的三号；不是普通数轴上负三小于正一。

翻开头 k 个时，先把这 k 项前后顺序反转，并把它们的正负号全部反过来。只翻一项也有效：位置不变，但朝向改变。

基本思路

与普通煎饼排序一样，只能翻开头的一整叠；还要求最后每块都朝向正确。每轮把未完成部分中最大的煎饼送到该部分末尾，并同时保证它变成正号。随后缩短未完成部分，不再碰已完成的大煎饼。

具体执行步骤

1. 只看未完成的开头部分，按大小而非正负找最大煎饼。若它已经在这部分末尾且是正号，就把这一格标为完成。
2. 否则先让它到第一格：它在第 k 格时，翻开头 k 项。先反转这些项的顺序，再反转每项符号。若它本来就在第一格，可以省略这个搬到开头的翻转。
3. 接下来要把整段开头翻一次，将最大煎饼送到末尾。那次还会反转符号，所以它在第一格时必须先是负号。若现在是正号，就单独翻第一项，变成负号。
4. 翻整个未完成开头：最大煎饼落到末尾，负号变正号，于是大小与方向都正确。未完成长度减一，继续。最后剩一项时仍检查方向：负号就翻一下，正号不用动。

跟着数字一步步做

1. 输入 $[-3, +1, -2]$ ，目标 $[+1, +2, +3]$ 。最大大小 3 已在开头且是负号，可以直接翻三项。
2. 先反转次序：原来 -2 到第一格，+1 留中间，-3 到第三格；再反转三个符号，得到 $[+2, -1, +3]$ 。第三格 +3 完成，不再包含进后面的翻转。
3. 剩下 $[+2, -1]$ ，最大大小 2 已在第一格，却是正号。若直接翻两项，它会以负号到末尾，所以先翻一项，得到 $[-2, -1, +3]$ 。
4. 再翻两项：交换前两项次序，并把它们符号反转，得到 $[+1, +2, +3]$ 。第二格完成。最后第一格 +1 方向也对，结束。
5. 如果最后剩的是 $[-1, +2, +3]$ ，还必须翻第一项，成为 $[+1, +2, +3]$ 。只有大小顺序正确不算完成。

要做多少事 要用多少地方

基本版本每轮都扫描未完成段找最大大小，长度依次 $n, n-1, \dots$ ，所以最好、通常、最坏都做 $\Theta(n^2)$ 工作。 n 是煎饼个数， Θ 是同一增长数量级。每轮至多三次翻转，总翻转次数 $O(n)$ ，但翻长段需要逐项交换、改符号，实际总元素工作仍可达 $\Theta(n^2)$ 。固定几个临时变量即可，额外空间 $O(1)$ 。这里 O 是上界；翻转次数少不等于每次翻转免费。

为什么快或慢

普通煎饼排序只管位置，这里一次翻转同时改变方向，所以要提前安排最大项在开头的符号。单项翻转就是修正方向的小动作。

什么时候适合用

适理解带方向的排列、片段反转以及煎饼问题。普通数字升序没有正负朝向要求。

最后把容易弄错的地方说清楚

这是一个简单可行的贪心方法，不保证翻转次数最少。相同大小记录若带身份，翻转可能改变它们先后，通常不稳定；若还要求特定等值次序，要另定规则。比较使用绝对大小，完成检查还要检查正号。

参考来源 [108] [109]

72 Stooge 排序

类型 趣味方法 别名或相关名称 三段重叠递归排序（描述性译名）

先把要用的小动作讲清楚

这里的递归不是新操作：把同一套指令用于一小段，等它做完再回到原来暂停的位置。必须记住正在处理哪段、接下来轮到哪一步。

长度 1 已有序；长度 2 只需比较并必要时交换。这是缩小任务后真正执行到底的小动作。

两个区间重叠，表示一些格会参加两个任务。第二个任务的交换可能影响第一个任务刚排好的位置。

基本思路

先比较这一段的两端，把较小的放左。随后做三件事：排前面的三分之二、排后面的三分之二、再排前面的三分之二。每个“排”又使用相同规则，把任务继续缩小，最终变成两格比较。它靠重叠任务把值逐步搬到正确两侧，但重复次数非常多。

具体执行步骤

1. 接到一段任务时，先检查长度。空段或单格返回，不移动。长度至少 2 就比较首尾，前者大才交换。若只有 2 格，现在已经完成。
2. 长度超过 2 时，把长度除以 3 向下取整得到 t 。例如 6 得 2，4 得 1。前任务包含除末尾 t 格外的全部格；后任务包含除开头 t 格外的全部格。
3. 先完成前任务。它里面若仍超过 2 格，也先比较两端，再做自己的前、后、前三个更小任务。等所有这些任务做完，才回到外层进行下一步。
4. 再完成后任务，同样继续缩小到两格比较。它会把较大的数推向整体右侧，但也可能让一个较小值落回中间重叠区。
5. 最后再完成前任务，把重叠区的小值向左整理。三个任务都返回后，这一整段才完成。输入一开始有序也照样走这些任务。

跟着数字一步步做

1. 先看三格 [2,3,1]。比较两端 2、1，交换为 [1,3,2]。 $t=1$ ，前任务是前两格，后任务是后两格。
2. 前两格 1、3 比较不换；后两格 3、2 比较交换，成为 [1,2,3]；再比较前两格 1、2 不换。三个小任务做完，大任务结束。
3. 为了看到第三次为什么需要，取 [1,5,6,7,3,2]。首尾 $1 \leq 2$ 不换， $t=2$ 。先排前四格 [1,5,6,7]；它已按序，但仍按上述小任务比较，数列不变。
4. 再排后四格 [6,7,3,2]。首尾 6、2 交换，局部变 [2,7,3,6]；前三级任务排成 [2,3,7,6]；后三级任务变 [2,3,6,7]；再次前三级不变。整列此时是 [1,5,2,3,6,7]。
5. 此时右边已经整理好，前面却有 5、2 逆序。最后前四格任务把 [1,5,2,3] 整理成 [1,2,3,5]，最终 [1,2,3,5,6,7]。若省掉第三个任务，这个例子就不有序。

要做多少事 要用多少地方

每个长任务制造三个仍约有原长度三分之二的任务，里面又各制造三个，数量迅速增多。 n 为个数，最好、通常、最坏均为 $\Theta(n^{\log_3 2})$ ，约 $\Theta(n^{0.631})$ ：比二次增长还快。这个指数来自“长度缩到 2/3、任务数乘 3”的反复关系。每次只交换两格，但需记录约 $\log n$ 层暂停任务，额外空间 $O(\log n)$ 。通常不稳定。

为什么快或慢

问题不在两格比较难，而在重叠区域被反复处理。普通高效分治把工作拆得更小、重复更少；这里三个任务每个都还很大，产生大量相同检查。

什么时候适合用

适合学习递归的真实执行顺序、重叠任务和增长速度。作为反面教材即可，不用拿大输入等待它运行。

最后把容易弄错的地方说清楚

有些来源采用对称的后—前—后顺序，本条统一采用前—后—前。远距离首尾交换可能让相等记录换序，所以不稳定。它确实能把数排好；效率差与正确性差是两件分别检查的事。

参考来源 [110]

73 慢排序

类型 趣味方法 别名或相关名称 Slowsort；勿与 Bogosort 的模糊别名混用

先把要用的小动作讲清楚

左右两半排好后，每半最后一格是这一半最大值。全段最大值一定是这两个之一，因为其他数都不超过它们。

递归调用表示暂停当前任务，完整执行同一方法处理更短区间，再返回继续。只有空段或单格任务可以直接结束。

“去掉最后一格”只是把它划出后续任务，不是删除它；这个数仍在数组里，已经最终定位。

基本思路

慢排序先花力气排好两半，用两个末端值找到全段最大值，再把最大值留在最右。接着，它对其余几乎整段重新使用慢排序。这个最后动作浪费了刚才排好两半的大量成果，却确实会继续固定下一个最大值，直到全部完成。

具体执行步骤

1. 任务只有一格或没有元素时，直接返回。否则从中间分成左半和右半；奇数长度时左半多一格。
2. 先完整执行左半任务，再完整执行右半任务。每个较短任务仍用本方法；最终两格任务变成分别看单格，再比较它们并必要时交换。
3. 两半都完成后，比较左半最后一格与右半最后一格。如果左半更大，交换两者；否则保留。右半最后一格也是全段最后一格，此时一定持有全段最大值。
4. 把全段最后一格留在原地，给前面剩下的整个区间安排新任务。它跨越原来的两半，不是只看一小块；其内部还可能有逆序。
5. 这个新任务也先处理自己的两半、固定自己的最大值、再处理去掉末端的区间。所有较短任务返回后，外层任务才完成。即使某段碰巧有序，也仍按这套调用顺序执行。

跟着数字一步步做

1. 输入 [4,1,3,2]，分成 [4,1] 与 [3,2]。先处理左半：两个单格直接返回，比较 4、1，交换，整列成为 [1,4,3,2]；再处理它的第一格 1，返回。
2. 处理右半：单格 3、2 各返回，比较并交换，成为 [1,4,2,3]；它的首格 2 返回。两半现在分别是 [1,4]、[2,3]。
3. 两个半段最大值是 4、3。交换它们，成为 [1,3,2,4]。4 最终定位，但前面的 3、2 仍逆序，所以排好两半并不等于整列有序。
4. 处理前三格 [1,3,2]：先处理左半 [1,3]，比较 1≤3 不换，按规则再处理单格 1；右半单格 2 直接返回。
5. 再比较左半末值 3 与段末值 2，交换成 [1,2,3,4]。前三格的 3 已定位，但仍按规则处理前两格 [1,2]：两个单格返回，比较不换，再处理单格 1。

6. 所有暂停的任务都已结束，回到最外层，得到 [1,2,3,4]。这些“返回”不改变数；它们表示小任务完成，执行次序回到原来暂停的位置。

要做多少事 要用多少地方

基本过程不因为数据已经有序而少调用任务，所以最好、通常、最坏调用数量同类。 n 是长度， $T(n)$ 表示处理长度 n 的工作： $T(n)=T(\lfloor n/2 \rfloor)+T(\lceil n/2 \rceil)+T(n-1)+O(1)$ 。前两项是排两半，第三项是重排几乎整段，最后是一次比较等固定工作。总增长 $n^{\Theta(\log n)}$ ，比任何固定幂 n^2 、 n^3 等最终都快增长，但比通常 c^n 慢。普通实现最多留住约 n 层暂停任务，空间 $O(n)$ ，通常不稳定。

为什么快或慢

只为了固定一个最大值，它先排两半，然后又处理长度仅少一格的区域。这种反复付费正是它故意慢的原因，不能把“分成两半”就自动当成高效。

什么时候适合用

适合把递归执行展开、比较不同递推结构的成本，作为效率反例；不用于真实排序。它没有随机猜测，步骤确定。

最后把容易弄错的地方说清楚

$\log n$ 可以理解为反复将长度减半所需的次数； $n^{\Theta(\log n)}$ 的指数本身随规模增长，不是一个固定次数。本条给增长类别与递推式，不伪造一个精确常数指数。普通调用栈含 $n-1$ 的长链；若实现专门消除尾调用，空间行为可变化。Slowsort 不应与随机洗牌的 Bogosort 混称。

参考来源 [111]

六 特殊模型与趣味方法

珠子、面条与计时器依赖额外模型，速度不能按普通比较排序衡量。后面的排列枚举、随机打乱和递归重试在常规机器上通常非常慢。趣味名称不改变保留元素和正确输出的要求；明确区分有限穷举与无有限最坏界的随机方法。

74 珠排序（重力排序）

类型 物理模型 别名或相关名称 算盘排序；Bead sort

先把要用的小动作讲清楚

这里一个数表示珠子的数量，3 就放三颗珠子。只能直接处理零或正整数。

一行横着摆，一列竖着摆。珠子只能在自己的竖列内下落，不能跑到隔壁列。空格没有珠子，计数为零。

基本思路

把每个数画成一行从左边开始摆的珠子。随后让各列的珠子沉到底部。上面的行得到较少珠子，下面的行得到较多珠子。最后逐行数珠子，就读出了从小到大的数。这里并没有让某个完整的原始记录一路移动：珠子重新组成了数。

具体执行步骤

1. 找出最大的数，记为 M ，准备 M 列；有几个输入数就准备几行。数值 x 对应的那一行，从第一格开始放 x 颗珠子。
2. 处理第一列：数这列共有几颗珠子。清空这一列，再从最底格往上放同样数量的珠子。这样模拟了下落，珠子总数没有变。
3. 第二列、第三列也这样处理，直到所有列都处理完。每列都独立下落。
4. 从顶行开始数：这一行有几颗珠子，就写下几。依次读到底行，得到升序结果。

跟着数字一步步做

1. 输入 $[3, 1, 4, 2]$ 。最大值是 4，所以每行四格。用 1 表示珠子、0 表示空格，初始四行依次是 1110、1000、1111、1100。
2. 第一列有四颗，四行全填；第二列有三颗，最上格空着，下面三格填珠子。此时前两列从上到下是 10、11、11、11。
3. 第三列有两颗，只填最下面两格；第四列有一颗，只填最底格。最终四行是 1000、1100、1110、1111。
4. 逐行数得 $[1, 2, 3, 4]$ 。为什么不会出现上行比下行珠子多？每列的珠子都挤在底部：上行某格有珠子，同列更低的格就一定也有。
5. 输入若含 0，就有一整行空格。它下落后仍对应一个空行，读为 0。

步骤状态图 C 比较 M 移动 S 已有序

输入：四个非负整数



每格是一行：1 珠、0 空



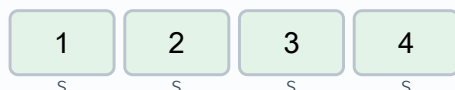
按列数珠子



各列下落后的四行



数每行珠子：升序输出



要做多少事 要用多少地方

计算机若真的准备整个格子表，每格都检查或填一次，工作量同时取决于数的个数和最大数值。 n 表示输入个数， M 表示最大值：时间 $\Theta(nM+n)$ ，格子空间 $\Theta(nM)$ ，另有 $O(n)$ 结果空间。 Θ 表示增长量同一数量级， O 表示上界。固定 n 、 M 后，这种实现的最好、通常、最坏情况没有数量级差别。若逐颗操作，还要计入所有数之和所代表的珠子总量。

为什么快或慢

让很多珠子一起落，可以在物理上同时做许多动作。但很大的数需要很宽的架子、很多珠子，或很多模拟格子。把这些资源藏起来，才会显得时间与数字大小无关。

什么时候适合用

适合很小的非负整数和理解物理计算。普通程序处理整数，计数排序或基数排序通常更合适。

最后把容易弄错的地方说清楚

负数、小数要另外编码。相同数值的原始记录身份没有自然保留下来，所以不能直接说这种方法稳定。原论文讨论不同的物理和数字实现，不能把其中某个模型的时间结论套到全部版本。图中 1110 是一整行四格，不是数字一千一百一十。

参考来源 [112] [113]

75 意大利面排序

类型 物理模型 别名或相关名称 面条排序；SAG (Spaghetti Analog Gadget)

先把要用的小动作讲清楚

把长度当作数：三厘米的杆代表 3，一厘米的杆代表 1。这里先假设长度为正且能准确制作、区分。

要比较杆的长短，底端必须落在同一个平面。把短杆举高，会误以为它更长。

基本思路

把每个数做成一根对应长度的杆，像一把干意大利面。底端齐平时，最高的顶端属于最长的杆。拿走最长的，再拿剩下最长的，就会按从大到小取出。把这串结果倒过来，才是从小到大。

具体执行步骤

1. 选定长度单位，准备每个数对应的杆。若需要知道原记录是谁，给杆加标签。
2. 把所有杆竖起来，底端都放在同一个平面上。检查没有杆悬空，也没有歪斜导致高度失真。
3. 从上方找到最高的顶端，取出对应杆，记下它的长度。现在这个数是剩余输入的最大值。
4. 继续对剩余杆做同样的事，直到取完。得到的是降序：把结果整串反转，或每次从结果的最后一个空位置往前填，形成升序。

跟着数字一步步做

1. 输入 [3,1,4,2]，做出长度为 3、1、4、2 单位的四根杆。原来谁排在前面不影响长度。
2. 底端对齐后，4 的杆最高。取出它，暂记 [4]；剩余长度是 3、1、2。
3. 接着 3 最高，取出后暂记 [4,3]。再取 2，得到 [4,3,2]。最后取 1，得到 [4,3,2,1]。
4. 反转时交换最外面的 4 和 1，得到 [1,3,2,4]；再交换中间的 3 和 2，得到 [1,2,3,4]。
5. 为什么每次取出的都可靠？因为剩余杆的底端仍然齐平，顶端最高就等于长度最长。若底端不齐，这个推理就不成立。

要做多少事 要用多少地方

理想模型若规定做杆、对齐、一次拿到最高杆都算一个动作，那么准备和读取各做约 n 次，工作量 $\Theta(n)$ ，保存 n 根杆需要 $\Theta(n)$ 资源。 n 是数的个数， Θ 是同一增长数量级。真实成本还取决于总长度、材料、精度和操作。计算机若没有一次拿到最大值的物理能力，而是每轮逐个比较寻找最大值，就会做 $n+(n-1)+\dots$ 次检查，时间 $\Theta(n^2)$ 。

为什么快或慢

快来自特殊器材把全部长度同时展示成高度。器材替我们做了大范围比较；准备器材和读出结果仍然需要工作。相近长度尤其难分辨。

什么时候适合用

适合讲解模拟计算，以及为什么效率必须说明允许什么操作。不是普通程序里免费调用的线性排序。

最后把容易弄错的地方说清楚

等长杆先取谁没有保证。若要求稳定，必须带原顺序标签并规定等高时的取出顺序。负数要加偏移，小数要求更高精度；这些都增加前提。不能把“对齐后高度已经有序”当作整项工作零成本完成。

参考来源 [114]

76 睡眠排序

类型 趣味方法 别名或相关名称 延时输出排序 (描述性名称)

先把要用的小动作讲清楚

一个任务可以先等待一段时间，之后再执行。等待结束通常只表示它可以运行，不保证它立刻抢到处理器或输出设备。

多个任务同时存在叫并发。并发不等于所有任务真的同一时刻开始、同一时刻按计划写出结果。

基本思路

给每个数开一个等待任务：小数值等得短，大数值等得长。希望它们按醒来的先后写出数，就成为升序。这依赖现实中没有普遍保证的时间条件，所以它是一种演示技巧，不能保证一般输入都正确排序。

具体执行步骤

1. 先限定非负数，选时间比例，例如每 1 个数值单位等 1 秒。若希望最小值先写出，所有等待必须以同一个起点计时。
2. 为每个输入安排任务。数值 x 的计划输出时刻是共同起点加 x 倍时间比例。不要误把依次创建任务的各自起点当成共同起点。
3. 收集实际写出的值，等所有任务完成。实际顺序取决于任务何时获得运行和输出机会。
4. 若计划时间与实际写出时间的次序不同，就得到错误结果。普通等待调用本身不提供排序正确性；演示必须明确这些理想前提。

跟着数字一步步做

1. 用 [3,1,2]，假设共同在零秒开始、每单位等一秒、没有任何延误。1 在一秒写出，2 在两秒写出，3 在三秒写出，结果 [1,2,3]。
2. 这期间没有比较两个数，也没有交换数组位置。数值被改成了计划等待时间。
3. 再看 [1,2]。任务 1 在一秒准备好，却因为处理器忙一直等到 2.5 秒才写出；任务 2 在两秒顺利写出。实际收集到 [2,1]，排序失败。
4. 还有一种错误：先创建 3 的任务，过很久才创建 1 的任务。即使各自都精确等待，较小数也可能更晚写出。共同起点可以修复这一点，却不能修复后续调度延误。

要做多少事 要用多少地方

理想条件下，创建和收集各处理 n 个任务，工作量 $\Theta(n)$ ；总等待约为时间比例乘最大值 M 。 n 是输入个数， M 是最大数值， Θ 表示同一增长数量级。通常要 $O(n)$ 任务资源， O 表示上界。只有启动动作少不代表整项工作快：若最大值随数字位数 b 达到约 2^b ，等待也可这么长。真实系统一般没有保证正确次序的调度上界，不能给它一个按 n 排列的可靠最好、通常、最坏排序时间。

为什么快或慢

它把数字大小换成真实时间，没有消除这份成本。把等待比例缩短，虽然快一些，但更容易被时间分辨率和调度误差打乱。

什么时候适合用

适合学习计时、并发和“准备运行”与“实际运行”的区别。需要结果一定正确时，不应选它。

最后把容易弄错的地方说清楚

等值记录的先后也没保证，因此不稳定。负数加偏移、小数扩大比例都不能解决任务延误。若使用能可靠按时刻取出事件的队列，队列本身可能已经需要堆或别的排序结构，其成本也应计入。

参考来源 [115]

77 穷举全排列排序

类型 趣味方法 别名或相关名称 确定性排列排序；deterministic Bogosort 在部分资料中的称呼

先把要用的小动作讲清楚

一个排列，就是同一批记录的一种摆法，不添加也不丢掉记录。例如 [3,1,2] 和 [1,3,2] 是两种摆法。

可以先选第一格放谁，再从剩余记录里选第二格。一次选到底得到一个候选；不成功就退回最近的选择，换一个还没试过的选项。

基本思路

按一个确定顺序，把可能的摆法逐一试出来。每得到一串完整候选，就检查是否从小到大；找到一串就停。它保证最后能找到答案，但前面失败的摆法几乎没给下一次提供帮助。

具体执行步骤

1. 准备当前候选和“哪些记录已经用过”的记号。从输入顺序开始，依次尝试第一格的各个可用记录。
2. 选好第一格后，把它标为已用。第二格只尝试尚未用过的记录。这样继续，直到每一格都有记录。
3. 从第一对相邻数开始检查候选。若每个数都不大于右边的数，返回这个候选。若发现前数更大，候选失败。
4. 失败后退回最近还有其他选项的位置。撤销后面记录的已用记号，换一个未试的选择，再把后面的格子逐格填上。一个位置的选项都试完就再向前退。这个规则保证没有遗漏摆法。

跟着数字一步步做

1. 输入 [3,1,2]，选项按原记录顺序尝试。第一格先放 3，第二格放 1，剩余 2 只能放第三格：候选 [3,1,2]，因为 $3 > 1$ 失败。
2. 退回第二格，把 1 放回可用记录，改选 2。第三格填剩余 1：候选 [3,2,1]，也失败。以 3 开头的选择已试完。
3. 退回第一格，改放 1。剩余记录按顺序是 3、2，所以先得到 [1,3,2]，因为 $3 > 2$ 失败。
4. 再退回第二格，改放 2，第三格放 3，得到 [1,2,3]。两对相邻数都通过，停止。以 2 开头的两个候选不必再试。

要做多少事 要用多少地方

候选增长非常快： n 个不同记录有 $n!$ 种摆法， $!$ 表示 $n \times (n-1) \times \dots \times 1$ 。这里 n 是记录个数。若每个候选都完整检查 $n-1$ 对，则第一候选已正确的最好时间 $\Theta(n)$ ，最后才找到的最坏时间 $\Theta(n \cdot n!)$ ， Θ 表示同一增长数量级。若正确候选的位置大致均匀，通常也需要阶乘量级试验。一次只保留当前候选和选择状态，额外空间 $O(n)$ ，不用存所有候选。发现错误就提前停止检查，会改变单次检查成本，但候选数仍巨大。

为什么快或慢

多一个不同记录，就把摆法数再乘一个越来越大的数。保证能结束，和能在合理时间内结束，是两件事。

什么时候适合用

适合用极少记录学习穷举和逐格选择，也可作为小输入的答案核对方法。

最后把容易弄错的地方说清楚

等值记录若各自有身份，默认可能改变顺序。按“数值、原下标”共同比较可规定唯一稳定目标。这里的排列枚举与排名排序不是同一方法：后者通过两两比较计算位置，不把所有摆法逐一尝试。

参考来源 [116]

78 Bogo 排序

类型 趣味方法 别名或相关名称 愚蠢排序；猴子排序；随机排列排序

先把要用的小动作讲清楚

先检查相邻数：左边不大于右边就是这一对顺序正确；所有相邻对都正确，整串才正确。

洗牌是重新打乱整批记录。正确的随机洗牌不丢记录，并让每种排列有同样机会；不是随便交换一次就等价。

基本思路

检查当前摆法。若已经有序就结束，否则整串随机洗牌，再检查。每次失败后都丢掉已有顺序信息，希望某次碰巧摆对。答案正确来自最后一次完整检查；难题是等待它出现可能要很久。

具体执行步骤

1. 从左到右检查相邻对。如果每对都满足左值不大于右值，就停止并返回当前数组。
2. 发现错误后做均匀洗牌。一个办法是从最后一格开始，在第一格到当前格之间等机会选一格，和当前格交换；再把当前格向左缩一格，重复到第二格。
3. 现在重新检查整串。洗牌之后，原来正确的前缀或后缀都不能继续当成已完成。
4. 只要检查失败，就再洗牌。演示可以设尝试次数上限，但达到上限的含义是尚未完成，不能把最后一串当作排序答案。

跟着数字一步步做

1. 从 [3,1,2] 开始，第一对 3、1 不对，准备洗牌。假设最后一格 2 抽中与第一格 3 交换，得到 [2,1,3]；再在前两格抽中交换，得到 [1,2,3]。这是一条成功的洗牌路径。
2. 也完全可能第一轮得到 [2,3,1]，检查因 $3>1$ 失败；下一轮又回到 [3,1,2]。前一次看起来的进步没有保证保留。
3. 只有得到 [1,2,3]，并检查 $1\leq 2$ 、 $2\leq 3$ 后，才停止。上述结果是假设的随机结果，不能照着预测下一轮。
4. 三个不同数有六种排列，均匀洗牌每次成功机会为六分之一。平均六轮不表示第六轮一定结束；第六十轮仍失败也可能发生。

要做多少事 要用多少地方

已经有序时只检查一遍，最好时间 $\Theta(n)$ ， n 为记录个数， Θ 表示同一增长数量级。不同值共有 $n!$ 种排列，其中一种有序；! 是从 n 连乘到 1。独立均匀洗牌平均需要 $n!$ 轮，每轮洗牌和完整检查都花约 n 个动作，期望时间 $\Theta(n \cdot n!)$ 。原地洗牌只用固定几个临时变量，额外空间 $O(1)$ 。随机失败可以不断发生，没有有限的最坏终止时间。若有重复数，数值排列数减少；例如 [1,1,2] 只有三种数值摆法，不是六种。

为什么快或慢

它完全不利用比较告诉我们的方向。巨大候选空间里反复抽签，即使最终结束的概率是 1，也没有某个有限时刻必定完成的承诺。

什么时候适合用

适合极小例子，用来区分平均等待、概率和最坏情况。不是实用的大数据排序。

最后把容易弄错的地方说清楚

通常不稳定，因为洗牌可能改变等值记录身份的先后。若重复值的数量分别为 c_1 、 c_2 、……，不同数值摆法数 $D=n!/(c_1!c_2! \dots)$ ，成功概率是 $1/D$ 。随机数或洗牌不符合前提会改变结论。只随机交换两个位置的是 Bozosort；stupid sort 也可能指侏儒排序，别名有歧义。

参考来源 [117] [118]

79 Bozo 排序

类型 趣味方法 别名或相关名称 随机两格交换排序

先把要用的小动作讲清楚

交换两格，要先暂存其中一个记录，再把另一格搬来，最后把暂存记录放进另一格。这样两个记录都还在。

随机选位置，是不根据数值大小决定选谁。交换后仍要检查全部相邻对，不能只看刚交换的两个数。

基本思路

每次随机选两个位置交换，然后检查整串是否已经有序。与整串洗牌不同，它一次只改两个位置。但交换不看数值方向，有时改好，有时把刚做好的关系改坏。

具体执行步骤

1. 先检查全部相邻对。有序就停；否则准备一次随机交换。
2. 这里约定从全部位置里随机选两个不同位置。把这两格的完整记录交换。另一种版本允许同一格选两次，那一轮相当于不动。
3. 重新从头检查相邻对。只有全都通过才成功；有任何一对左边更大就继续。
4. 再次随机选两格交换。任何位置以后仍可被选中，不能把碰巧正确的开头标为永久完成。

跟着数字一步步做

1. 从 [3,1,2] 开始， $3 > 1$ ，所以失败。假设随机选第一、第二格。暂存 3，把 1 搬到第一格，把 3 放到第二格，得到 [1,3,2]。
2. 检查：1 ≤ 3 通过， $3 > 2$ 失败。假设这次选第二、第三格，交换得到 [1,2,3]。全部相邻对通过，于是结束。
3. 但另一条路径会在 [1,3,2] 后又选第一、第二格，回到 [3,1,2]。第一格曾经正确，不代表以后被保护。
4. 也可能一直交换同一对位置，来回往返。每轮都遵守规则，却没有保证在有限轮数内到达目标。

要做多少事 要用多少地方

已排序时只扫描一遍，最好时间 $\Theta(n)$ ， n 是记录数， Θ 表示同一增长数量级。一次交换只要固定几个动作，额外空间 $O(1)$ ，即固定数量临时变量。困难是需要几轮：标准随机交换模型的等待可达阶乘量级；准确期望依赖选位置规则、起始排列与检查方式，不能给所有版本套同一个公式。每轮若完整检查就再做 $\Theta(n)$ 工作；发现第一处错误即停可少一些。没有有限的最坏终止时间，通常也不稳定。

为什么快或慢

交换本身便宜，却不一定朝答案走。改好一对关系时也可能破坏其他关系，下一轮又可能撤销刚才的交换。

什么时候适合用

适合用三四个数比较“一次洗全串”和“一次随机交换”的差别。不是实用排序。

最后把容易弄错的地方说清楚

若总选两个不同位置，每次交换会改变排列的奇偶性；这不妨碍最终到达目标，只影响可能到达的步数。允许同一位置被选两次会多一些空转。理想随机选择可以使最终到达概率为 1，仍不能给出固定完成期限。

参考来源 [119] [118]

80 Bogo-Bogo 排序

类型 趣味方法 别名或相关名称 递归检查型 Bogobogosort；存在不统一变体

先把要用的小动作讲清楚

副本是另一份独立记录。改副本，不会自动改原数组；下面会同时跟踪两份，不能混为一谈。

前缀是开头的一段。前缀若已经升序，它的最后一个数就是这段的最大数。再看全串最后一个数是否不小于它，就能知道整份副本是否有序。

递归是让同一方法处理更短的输入。较大任务先停在“等待小任务完成”处，小任务做完再回来继续；长度零或一时直接返回，避免一直拆下去。

基本思路

这是一种故意把检查做得很费劲的方法。普通随机排序直接检查相邻对；这里先复制原数组，再用同一个慢方法排副本的前面部分，想办法得到一份完全有序的副本，然后比较副本和原数组是否一模一样。若不同，明明已有正确副本，也不拿它当答案，而是重新洗原数组再试。

具体执行步骤

1. 空串或一项直接完成。其余情况，要检查原数组 A，就先做独立副本 C。A 在这次检查过程中不动。
2. 让同一方法排序 C 的前 $n-1$ 项， n 是当前项数。这个小任务也要复制、检查、必要时洗牌；它没完成，当前任务就等着。
3. 小任务完成后，C 的前面部分已经有序。看 C 最后一项是否不小于前缀最后一项。若更小，就把整个 C 随机洗牌，再重排它的前 $n-1$ 项，再比这两个末尾值，直到 C 完全有序。
4. 现在逐格比较有序副本 C 与检查前的 A。每格数值都相同，说明 A 本来就有序，本次检查通过；只要一格不同就失败。
5. 失败后丢掉这次检查结果，对原数组 A 洗牌。重新从复制步骤开始。千万不要把有序 C 直接复制回 A；那会变成另一种、较合理的方法。

跟着数字一步步做

1. 外层原数组 $A=[2,1,3]$ ，先复制 $C=[2,1,3]$ 。要排 C 的前两项 $[2,1]$ ，于是暂停三项任务，进入两项任务。
2. 两项任务的原串 $P=[2,1]$ ，做副本 $Q=[2,1]$ 。 Q 的前一项 $[2]$ 无须动；最后的 1 小于 2 ，所以 Q 还没排好。假设洗 Q 得 $[1,2]$ ，再处理前一项 $[1]$ ，比较 $2 \geq 1$ ， Q 有序。
3. $Q=[1,2]$ 与 $P=[2,1]$ 不同，两项检查失败。假设洗原串 P 得 $[1,2]$ ，下一次复制 $Q=[1,2]$ ，前一项直接返回，末尾比较通过， Q 与 P 相同。两项任务这才返回 $[1,2]$ 。
4. 回到三项任务，现在 $C=[1,2,3]$ 。 $3 \geq 2$ ，所以 C 有序；但 C 不等于 $A=[2,1,3]$ 。外层检查失败， A 仍是原来的摆法。
5. 假设洗 A 得 $[1,3,2]$ 。副本前两项 $[1,3]$ 本来有序，其两项检查可直接通过；但副本末尾 $2 < 3$ ，所以要洗的是副本 C 。假设洗 C 得 $[2,1,3]$ ，它的前两项像上面那样最终排成 $[1,2]$ ， C 又得到 $[1,2,3]$ ，仍与 $A=[1,3,2]$ 不同。
6. 外层再失败，再洗 A 。假设这回 $A=[1,2,3]$ 。副本的前两项检查通过，末尾 $3 \geq 2$ ，副本与原串逐格相同，外层终于通过。这些洗牌结果只是选来说明过程的可能路径，不保证实际轮数。

要做多少事 要用多少地方

即使输入本来有序，检查也要做长度 n 、 $n-1$ ……的副本，并逐层比较，所以这个版本最好也花 $\Theta(n^2)$ 复制和检查工作； n 是项数， Θ 表示同一增长数量级。较大任务等待时仍保留自己的副本，基本实现同时保存的格子数为 $n+(n-1)+\dots$ ，额外空间 $O(n^2)$ ， O 表示上界。随机重试还可能无限继续，没有有限的最坏终止时间。不同 Bogobogosort 定义没有统一公认的精确期望公式，不能把作者的估测写成定理。

为什么快或慢

它把本可一遍完成的检查，换成套着套的排序，并且在检查失败后扔掉已排好的副本。内层等待与外层等待一起放大浪费。

什么时候适合用

只适合极小例子，帮助看懂递归等待、副本与原数据的区别，以及为什么检查方法本身也有成本。

最后把容易弄错的地方说清楚

这里用“大于或等于”检查末项，以支持重复的最大值；严格大于的原始写法对这些重复值有问题。数值相同的记录身份仍可能被洗乱，通常不稳定。还有逐步增长前缀等不同版本，它们不应继承本条的准确成本结论。

参考来源 [\[120\]](#) [\[121\]](#)

实现检查与学习练习

1. 只检查是否升序不够。比较输入和输出的多重集合：每个值出现次数必须一致。对记录要比较身份，不能只比较键。
2. 稳定性测试使用 $[2a, 1, 2b, 1c]$ ，应得到 $[1, 1c, 2a, 2b]$ ，其中首个 1 是原来的那条记录。默认不稳定的算法可以通过加原位置次键改造，但需重新计算额外资源。
3. 边界样例至少包括空数组、单项、已排序、逆序、全部相等、很多重复、负数、极小/极大键、非二次幂长度。专用算法还要检查键域、位宽和哨兵前提。
4. 快排分区后验证区域关系并确保递归区间缩小；Hoare 分界不是枢轴最终位置。归并验证两输入段已有序；相等时先取左侧。堆删除后验证父子关系；桶排序不能把分桶完成当排序完成。
5. 性能实验固定规模、键类型、原输入、稳定要求、线程数和是否计入磁盘或设备传输。每轮用相同原输入的副本，记录多次运行的中位数，不把后一轮“已经排好”的输入拿来与第一轮比较。
6. 练习 1 用插入、Lomuto 快排和归并分别处理 $[5, 2, 4, 1, 3]$ ，记录每次比较后的动作。练习 2 用 LSD 排 $[31, 12, 21, 11]$ ，解释为何十位分配必须稳定。练习 3 为相同值添加身份，寻找选择排序改变相等顺序的例子。练习 4 检验所有四项零一输入都通过本书五比较器小网络。

练习提示

LSD 个位后为 $[31, 21, 11, 12]$ ，十位后为 $[11, 12, 21, 31]$ 。选择排序的反例可用 $[2a, 2b, 1]$ ，第一次交换即成 $[1, 2b, 2a]$ 。四项零一网络共检查 $2^4=16$ 个输入。

参考资料

来源核验日期 2026 年 10 月 5 日。定义与复杂度以所述版本为准；高级实现参考作者论文和源代码。下列标题和网址均可点击，正文引用编号可跳到对应条目。自制例子与图未复制来源的插图。

[1] NIST DADS: Counting sort

<https://xlinux.nist.gov/dads/HTML/countingsort.html>

[2] NIST DADS: Pigeonhole sort

<https://xlinux.nist.gov/dads/HTML/pigeonholeSort.html>

[3] Sedgewick and Wayne: String sorts

<https://algs4.cs.princeton.edu/51radix/>

[4] Princeton LSD implementation

<https://algs4.cs.princeton.edu/51radix/LSD.java.html>

[5] Princeton MSD implementation

<https://algs4.cs.princeton.edu/51radix/MSD.java.html>

[6] McIlroy, Bostic, McIlroy: Engineering radix sort

https://www.usenix.org/publications/compsystems/1993/win_mcilroy.pdf

[7] NIST Radix Sort

<https://xlinux.nist.gov/dads/HTML/radixsort.html>

[8] Princeton AmericanFlag implementation

<https://algs4.cs.princeton.edu/51radix/AmericanFlag.java.html>

[9] NIST DADS: Bucket sort

<https://xlinux.nist.gov/dads/HTML/bucketsort.html>

[10] MIT 6.006: Linear-time sorting

<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/resources/lecture-7-counting-sort-radix-sort-lower-bounds-for-sorting/>

[11] Karl-Dietrich Neubert: The FlashSort algorithm

<https://www.neubert.net/FSOIntro.html>

[12] Neubert: Flashsort implementations

<https://www.neubert.net/Flacodes/FLACodes.html>

[13] Sedgewick and Wayne: Tries

<https://algs4.cs.princeton.edu/52trie/>

[14] Princeton TrieST implementation

<https://algs4.cs.princeton.edu/52trie/TrieST.java.html>

[15] Sinha and Zobel: Cache-conscious sorting of large sets of strings with dynamic tries

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acmj04.pdf>

[16] Sinha and Zobel: Efficient trie-based sorting

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acsc03sz.pdf>

[17] Boost.Sort: Spreadsort documentation

https://www.boost.org/doc/libs/1_69_0/libs/sort/doc/html/sort/single_thread/spreadsort.html

[18] Steven Ross: Original Spreadsort paper

https://www.boost.org/doc/libs/1_61_0/libs/sort/doc/papers/original_spreadsort06_2002.pdf

[19] Bentley & Sedgewick: Fast Algorithms for Sorting and Searching Strings

<https://sedgewick.io/wp-content/themes/sedgewick/papers/1997StringsSODA.pdf>

[20] Princeton: Quick3string implementation

<https://algs4.cs.princeton.edu/51radix/Quick3string.java.html>

[21] NIST: multikey quicksort aliases

<https://xlinux.nist.gov/dads/HTML/multikeyQuicksort.html>

[22] Munro and Wild: Nearly-optimal mergesorts

<https://www.wild-inter.net/publications/munro-wild-2018.pdf>

[23] Princeton: Mergesort

<https://algs4.cs.princeton.edu/22mergesort/>

[24] Tim Peters and contributors: CPython listsort notes

<https://github.com/python/cpython/blob/main/Objects/listsort.txt>

[25] Auger, Jugé, Nicaud, Pivoteau: On the worst-case complexity of TimSort

<https://arxiv.org/abs/1805.08612>

[26] CPython 3.10 classic collapse implementation

<https://github.com/python/cpython/blob/3.10/Objects/listobject.c>

[27] Munro and Wild: Nearly-optimal mergesorts

<https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2018.63>

[28] Authors' Powersort implementations

<https://github.com/sebawild/powersort>

[29] Gelling, Nebel, Smith, Wild: Multiway Powersort

<https://arxiv.org/abs/2209.06909>

[30] Munro and Wild: Nearly-optimal mergesorts, Peeksort algorithm

<https://drops.dagstuhl.de/opus/volltexte/2018/9526/pdf/LIPIcs-ESA-2018-63.pdf>

[31] Princeton: Elementary sorts

<https://algs4.cs.princeton.edu/21elementary/>

[32] Princeton Insertion implementation

<https://algs4.cs.princeton.edu/21elementary/Insertion.java.html>

[33] NIST DADS: Binary insertion sort

<https://xlinux.nist.gov/dads/HTML/binaryinsort.html>

[34] Bender, Farach-Colton, Mosteiro: Insertion sort is $O(n \log n)$

<https://arxiv.org/abs/cs/0407003>

[35] Aldous and Diaconis: Longest increasing subsequences, from patience sorting

<https://escholarship.org/uc/item/09j2v8d0>

[36] Aldous and Diaconis: Patience sorting paper

<https://www.stat.berkeley.edu/users/aldous/Research/OP/patience.pdf>

[37] Stanford Aldous and Diaconis research report

<https://statistics.stanford.edu/technical-reports/longest-increasing-subsequences-patience-sorting-baik-deift-johansson-theorem>

- [38] Dijkstra: Smoothsort, an alternative for sorting in situ, EWD796a
<https://www.cs.utexas.edu/~EWD/transcriptions/EWD07xx/EWD796a.html>
- [39] musl libc qsort: Smoothsort implementation
<https://git.musl-libc.org/cgit/musl/tree/src/stdlib/qsrt.c>
- [40] NIST DADS: Strand sort
<https://xlinux.nist.gov/dads/HTML/strandSort.html>
- [41] Levcopoulos and Petersson: Adaptive heapsort
<https://www.sciencedirect.com/science/article/abs/pii/S0196677483710217>
- [42] Goodrich: Cartesian-tree priority-queue sort lecture
<https://ics.uci.edu/~goodrich/teach/cs263/notes/09-Maxima.pdf>
- [43] Merge.java
<https://algs4.cs.princeton.edu/22mergesort/Merge.java.html>
- [44] Sedgewick & Wayne: MergeBU implementation
<https://algs4.cs.princeton.edu/22mergesort/MergeBU.java.html>
- [45] MergeBU API and complexity
<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/MergeBU.html>
- [46] Sedgewick & Wayne: heaps and heapsort
<https://algs4.cs.princeton.edu/24pq/>
- [47] Princeton: Heap implementation
<https://algs4.cs.princeton.edu/24pq/Heap.java.html>
- [48] Edelkamp & Wegener: weak-heapsort analysis
<https://eccc.weizmann.ac.il/report/1999/028/download/>
- [49] Edelkamp et al.: Weak heaps engineered
<https://www.sciencedirect.com/science/article/pii/S1570866713000592>
- [50] Katajainen: weak-heap variants and applications
<https://hjemmesider.diku.dk/~jyrki/Myris/EEK2012bJ.html>
- [51] Sahni: winner-tree sorting implementation
<https://www.cise.ufl.edu/~sahni/dsaaj/instruct/exer/c14/e8.htm>
- [52] NIST: tournament sort and original report
<https://xlinux.nist.gov/dads/HTML/tournamentSort.html>
- [53] Musser: original introsort paper
[https://onlinelibrary.wiley.com/doi/10.1002/\(SICI\)1097-024X\(199708\)27:8%3C983::AID-SPE117%3E3.0.CO;2-%23](https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1097-024X(199708)27:8%3C983::AID-SPE117%3E3.0.CO;2-%23)
- [54] Musser: generic algorithms and introsort
https://www.cs.rpi.edu/~musser/gp/index_1.html
- [55] orlp: pdqsort design and guarantees
<https://github.com/orlp/pdqsort/blob/master/readme.md>
- [56] orlp: pdqsort source
<https://github.com/orlp/pdqsort/blob/master/pdqsort.h>
- [57] Lammich: verified introsort and pdqsort
<https://pmc.ncbi.nlm.nih.gov/articles/PMC7324064/>

[58] Moffat, Eddy & Petersson: Splaysort

[https://onlinelibrary.wiley.com/doi/10.1002/\(SICI\)1097-024X\(199607\)26:7%3C781::AID-SPE35%3E3.0.CO;2-B](https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1097-024X(199607)26:7%3C781::AID-SPE35%3E3.0.CO;2-B)

[59] Sleator & Tarjan: self-adjusting trees

<https://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf>

[60] Carnegie Mellon: zig, zig-zig, zig-zag

<https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15451-f97/www/splay.html>

[61] Atkinson Sack Santoro Strothotte Min-Max Heaps and Generalized Priority Queues

<https://people.scs.carleton.ca/~santoro/Reports/MinMaxHeap.pdf>

[62] Grailsort author implementation

<https://github.com/Mrrl/GrailSort>

[63] WikiSort author: in-place blocks, buffers, tags

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%203.%20In-Place.md>

[64] WikiSort author implementation

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/WikiSort.c>

[65] MIT OpenCourseWare: Quicksort lecture

<https://ocw.mit.edu/courses/6-046j-introduction-to-algorithms-sma-5503-fall-2005/resources/lecture-4-quicksort-randomized-algorithms/>

[66] Aumüller et al.: Lomuto-based partitioning analysis

<https://arxiv.org/abs/1810.12047>

[67] Princeton: original Hoare implementation

<https://algs4.cs.princeton.edu/23quicksort/QuickHoare.java.html>

[68] Sedgewick & Wayne: original Hoare exercise

<https://algs4.cs.princeton.edu/23quicksort/>

[69] Princeton: Quick3way implementation

<https://algs4.cs.princeton.edu/23quicksort/Quick3way.java.html>

[70] Wild: Quicksort with many equal keys

<https://arxiv.org/abs/1608.04906>

[71] Princeton: dual-pivot teaching implementation

<https://algs4.cs.princeton.edu/23quicksort/QuickDualPivot.java.html>

[72] Wild & Nebel: Yaroslavskiy analysis

<https://arxiv.org/abs/1310.7409>

[73] Frazer & McKellar: original samplesort paper

<https://research.ibm.com/publications/samplesort-a-sampling-approach-to-minimal-storage-tree-sorting>

[74] Axtmann et al.: in-place parallel superscalar samplesort

<https://arxiv.org/abs/1705.02257>

[75] Sedgewick & Wayne: binary search trees

<https://algs4.cs.princeton.edu/32bst/>

[76] Sedgewick & Wayne: balanced search trees

<https://algs4.cs.princeton.edu/33balanced/>

[77] Princeton: Shell implementation and Knuth bound

<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Shell.html>

[78] Poonen: Shellsort bounds and Pratt increments

<https://math.mit.edu/~poonen/papers/shell.pdf>

[79] Frigo et al.: Cache-Oblivious Algorithms, MIT

https://ocw.mit.edu/courses/6-895-theory-of-parallel-systems-sma-5509-fall-2003/7e5331aae765482e87251ba75d9aeabe_cache_oblivs_focs.pdf

[80] Prokop: Cache-Oblivious Algorithms thesis, MIT

https://ocw.mit.edu/courses/6-895-theory-of-parallel-systems-sma-5509-fall-2003/6dc7de52dcf13b53cebf2fe10ae6752a_cache_oblivs_thsis.pdf

[81] Stober Weiß On the Average Case of MergeInsertion

<https://arxiv.org/abs/1905.09656>

[82] Princeton Sorting Applications Index Sorting

<https://algs4.cs.princeton.edu/25applications/>

[83] OpenDSA External Sorting

<https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/ExternalSort.html>

[84] Yufei Tao External Memory Model and Sorting

<https://www.cse.cuhk.edu.hk/~taoyf/course/wst621/notes/lec1.pdf>

[85] NIST Polyphase Merge Sort

<https://xlinux.nist.gov/dads/HTML/polyphmrgsrt.html>

[86] NIST Optimal Polyphase Merge Sort

<https://xlinux.nist.gov/dads/HTML/optimalPolyphaseMergeSort.html>

[87] Carnegie Mellon Parallel Sorting

<https://www.cs.cmu.edu/afs/cs/academic/class/15210-f15/www/tapp.html>

[88] Authors IPS4o implementation

<https://github.com/ips4o/ips4o>

[89] Satish Harris Garland Designing Efficient Sorting Algorithms for Manycore GPUs

<https://www.nvidia.com/docs/io/67073/nvr-2008-001.pdf>

[90] NVIDIA CUB DeviceRadixSort official implementation and documentation

https://github.com/NVIDIA/cccl/blob/main/cub/cub/device/device_radix_sort.cuh

[91] NIST Bitonic Sort and Batcher Reference

<https://xlinux.nist.gov/dads/HTML/bitonicSort.html>

[92] Hans Werner Lang Odd-even mergesort

<https://www.inf.hs-flensburg.de/lang/algorithmen/sortieren/networks/oemen.htm>

[93] University of Pittsburgh Shear Sort assignment

<https://people.cs.pitt.edu/~bmills/docs/teaching/cs1645/hw6.pdf>

[94] Scherson Sen Shamir Two nearly optimal sorting algorithms using shear-sort

[https://doi.org/10.1016/0743-7315\(89\)90047-6](https://doi.org/10.1016/0743-7315(89)90047-6)

[95] NIST DADS: comb sort

<https://xlinux.nist.gov/dads/HTML/combSort.html>

[96] Hans Bezemer: Circle sort, original author discussion

<https://sourceforge.net/p/forth-4th/wiki/Circle%20sort/>

[97] Haddon: Cycle-Sort: A Linear Sorting Method (author-uploaded paper)

https://www.researchgate.net/publication/220459798_Cycle-Sort_A_Linear_Sorting_Method

[98] Manipal: Parallelization of Cycle Sort Algorithm

<https://researcher.manipal.edu/en/publications/parallelization-of-cycle-sort-algorithm/>

[99] Colorado State University: Enumeration Sort with duplicate tie breaking

<https://www.cs.colostate.edu/~cs475/f17/Lectures/Lec10Sorting.pdf>

[100] NIST DADS: selection sort

<https://xlinux.nist.gov/dads/HTML/selectionSort.html>

[101] NIST DADS: gnome sort

<https://xlinux.nist.gov/dads/HTML/gnomeSort.html>

[102] NIST DADS: bubble sort

<https://xlinux.nist.gov/dads/HTML/bubblesort.html>

[103] NIST DADS: bidirectional bubble sort

<https://xlinux.nist.gov/dads/HTML/bidirectionalBubbleSort.html>

[104] ETH Zürich: Principles of Distributed Computing, Odd/Even Sort

https://disco.ethz.ch/courses/fs12/podc/lecture/podc_2on1.pdf

[105] UCAR CESM official source: interchange_sort

https://docs.cesm.ucar.edu/models/cesm2/config/old/cesmBbrowser/html_code/cam/quicksort.F90.html

[106] WikiSort author: merging techniques

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%202.%20Merging.md>

[107] WikiSort author: rotation and other tools

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%201.%20Tools.md>

[108] Anupam Gupta: Sorting Pancakes lecture notes

<https://cs.nyu.edu/~anupamg/251-notes/pancakes.pdf>

[109] Cohen and Blum: On the problem of sorting burnt pancakes

<https://www.sciencedirect.com/science/article/pii/0166218X94000093>

[110] NIST DADS: stooge sort

<https://xlinux.nist.gov/dads/HTML/stoogesort.html>

[111] Broder and Stolfi: Pessimal Algorithms and Simplicity Analysis

<https://ic.unicamp.br/~stolfi/EXPORT/papers/by-tag/papers/math/pessimal-algorithms/download.pdf>

[112] Arulanandham, Calude and Dinneen: Bead-Sort: A Natural Sorting Algorithm

https://www.researchgate.net/publication/37987842_Bead--Sort_A_Natural_Sorting_Algorithm

[113] Calude: University of Auckland publication list

<https://www.cs.auckland.ac.nz/~cristian/papers.html>

[114] A. K. Dewdney: Computer Recreations, June 1984

<https://www.scientificamerican.com/article/computer-recreations-1984-06/>

[115] Kevlin Henney: Need Something Sorted? Sleep On It!

<https://accu.org/journals/overload/31/175/henney/>

[116] NIST DADS: permutation sort

<https://xlinux.nist.gov/dads/HTML/permutationSort.html>

[117] NIST DADS: bogosort

<https://xlinux.nist.gov/dads/HTML/bogosort.html>

[118] Gruber, Holzer and Ruepp: Sorting the Slow Way

https://link.springer.com/chapter/10.1007/978-3-540-72914-3_17

[119] NIST DADS: bozo sort

<https://xlinux.nist.gov/dads/HTML/bozoSort.html>

[120] David Morgan-Mar: Bogobogosort definition and discussion

<https://www.dangermouse.net/esoteric/bogobogosort.html>

[121] Lucas Werkmeister: recursive-check implementation

<https://github.com/lucaswerkmeister/Miscellaneous/blob/master/Sorting/src/Bogobogosort.java>